Doc List

Internal Doc System — Categories + Docs (Brain of Axion)

System Overview — Expanded Doc Set (Hardened, Non-Redundant)

*You're right: 3 docs won't carry the "brain" with enough precision. Here's an expanded **System Overview** category that stays non-redundant and locks the guarantees, boundaries, and operating model.*

SYS-01 System Purpose & Guarantees

- *What Axion is for, what it must optimize for (speed + quality + consistency)*
- *Non-negotiable guarantees (no guessing, traceability, deterministic outputs)*

SYS-02 Operating Principles (the rules behind every decision)

- *No duplicate truth*
- *Explicit unknowns*
- *Determinism & version pinning*
- *"No forward progress without compliance"*
- *Separation: internal-agent vs external-agent responsibilities*

SYS-03 End-to-End Architecture (pipeline stages + artifact flow)

- *Full pipeline from intake → kit → build → proof*
- *Inputs/outputs per stage (high-level)*

SYS-04 Artifact Taxonomy (what artifacts exist and why)

- *A1/A2/A3/A4/A5/A6/A7 summarized as the system's "parts list"*

- *What is raw vs normalized vs canonical vs derived*

SYS-05 Roles & Responsibilities (agents + user)

- *Internal agent responsibilities*
- *External agent responsibilities*
- *User responsibilities (what must be supplied/approved)*
- *Handoff boundaries (what each party can trust)*

SYS-06 Data & Traceability Model (provenance chain)

- *Submission → validation → normalization → spec → plan → kit → proofs*
- *IDs and linkage requirements at a conceptual level (not the detailed CAN docs)*

SYS-07 Compliance & Gate Model (how Axion prevents drift)

- *What a "gate" is*
- *Gate types (schema validation, template completeness, proof gates)*
- *How gates stop progression*

SYS-08 Configuration Model (what's fixed vs configurable)

- *Global fixed rules vs project-configurable settings*
- *Where configuration lives (standards snapshot vs system config)*

SYS-09 Terminology & Definitions (Core Terms)

- *Core terms and consistent definitions*
- *Links to deeper vocab library if needed*

SYS-10 System Boundaries (what Axion explicitly does NOT do)

- *What is out-of-scope for the system itself*

- *Prevents feature creep in the system design*
-

2) Intake & Validation

- INT-01 Intake Form Spec (field/page layout)
- INT-02 Intake Schema Spec (validity contract)
- INT-03 Intake Validation Rules (dependencies + thresholds)
- INT-04 Submission Record Format (traceability)
- INT-05 Validator Output Format (errors/warnings contract)

3) Canonical Data Model

- CAN-01 Canonical Spec Model (entities + relationships + IDs)
- CAN-02 ID & Reference Rules (no duplicate truth, referential integrity)
- CAN-03 Unknowns Model (how unknowns are represented and handled)

4) Standards System

- STD-01 Standards Library Structure (what standards exist)
- STD-02 Standards Resolution Rules (defaults → overrides → snapshot)
- STD-03 Standards Snapshot Format (what gets emitted into kits)

5) Template System

- TMP-01 Template Types & Template Index (registry)
- TMP-02 Template File Contract (required sections + placeholder rules)
- TMP-03 Template Selection Rules (routing/gates → which templates apply)
- TMP-04 Template Fill Rules (direct/derived/standards inserts; no guessing)
- TMP-05 Template Completeness Rules (what “valid” means)

6) Document Build Order & Gates

- ORD-01 Build Order Graph (doc dependency graph)
- ORD-02 Gate DSL / Gate Rules (how compliance is checked)
- ORD-03 Per-Doc Gate Checklist (standard gate format)

7) Planning & Decomposition System

- PLAN-01 Work Breakdown Rules (how spec becomes work units)

- PLAN-02 Acceptance Map Rules (how proof is attached to units)
- PLAN-03 Sequencing Heuristics (foundational first, vertical slices, etc.)

8) Verification & Proof System

- VER-01 Proof Types & Evidence Rules (what counts as proof)
- VER-02 Verification Command Policy (when/how to run checks)
- VER-03 Completion Criteria (global “done” rules)

9) Packaging & Kit Format

- KIT-01 Kit Folder Structure Contract
- KIT-02 Manifest/Index Format (IDs → paths, reading order)
- KIT-03 Entrypoint Contract (single start file rules)
- KIT-04 Version Stamping Rules (what versions get recorded)

10) Continuity & State

- STATE-01 State Snapshot Format
- STATE-02 Resume Rules (how to restart without drift)
- STATE-03 Handoff Rules (agent-to-agent continuity)

11) Governance & Evolution

- GOV-01 Versioning Policy (templates/standards/system)
- GOV-02 Change Control Rules (what can change, approval requirements)
- GOV-03 Deprecation & Migration Rules
- GOV-04 Audit / Traceability Rules

12) Execution Instructions

- EXEC-01 Internal Agent Runbook (how internal agent runs the pipeline)
- EXEC-02 External Agent Prompt Template *(the one external-facing piece)*
- EXEC-03 Failure Handling Playbook (what to do when blocked)

System Docs (SYS)

System Overview — Locked Doc Set

1. SYS-01 System Purpose & Guarantees
2. SYS-02 Operating Principles
3. SYS-03 End-to-End Architecture
4. SYS-04 Artifact Taxonomy
5. SYS-05 Roles & Responsibilities
6. SYS-06 Data & Traceability Model
7. SYS-07 Compliance & Gate Model
8. SYS-08 Configuration Model
9. SYS-09 Terminology & Definitions
10. SYS-10 System Boundaries

Locked.

SYS-01

SYS-01 — System Purpose & Guarantees

1) Purpose

Axion exists to convert a project idea into an execution-ready Agent Kit that can be used by an external build agent to produce working software with proof—not claims—while minimizing drift, contradictions, and missed requirements.

Axion is not “an agent.” Axion is the system that **constrains and equips** agents.

---

2) System Purpose (what Axion is trying to optimize)

Axion optimizes for:

- **Consistency:** two runs of the same input + same versions should yield the same kit structure and constraints.
- **Completeness:** the system should not miss standard senior-level project requirements (security, QA, data, ops where applicable).
- **Traceability:** every output can be traced back to user-provided input and the resolved ruleset.
- **Execution-readiness:** the kit must tell an agent exactly what to build, in what order, and how to verify.

---

3) Guarantees (non-negotiable)

Axion must guarantee the following:

G-01 No Creative Guessing

If required information is missing, Axion must:

- mark it as **UNKNOWN** explicitly, or
- fail validation / fail a gate (depending on the rule)

Axion must never invent requirements, data, or constraints.

G-02 Single Source of Truth

Axion must maintain one authoritative truth model per project (Canonical Spec).
Templates and plans must reference truth by ID, not rewrite it.

G-03 Deterministic Constraints

Axion must produce a Resolved Standards Snapshot that:

- is self-contained
- pins versions
- records defaults + overrides
- defines fixed vs configurable rules

G-04 Traceability Chain

Axion must preserve a provenance chain:
Intake Submission → Validation Result → Normalized Input → Canonical Spec → Work Breakdown → Acceptance Map → Kit Manifest → Proof Log pointers.

Every derived artifact must link to its inputs by ID.

G-05 No Forward Progress Without Compliance

Axion must not progress to downstream artifacts if upstream gates fail.
Compliance is enforced at these layers:

- intake validity (schema)
- template completeness
- reference integrity
- acceptance/proof requirements

G-06 Portable, Predictable Kit Output

Axion must output an Agent Kit with:

- fixed file tree contract
- manifest/index
- entrypoint
- version stamps
- required artifacts included (or explicitly N/A with reason)

---

4) Success Criteria (how we know SYS-01 is true)

Axion is succeeding if:

- external agent can build from kit without asking foundational questions

- drift incidents are traceable to a specific missing/unknown or override
- the kit can be audited (what rules were applied, what proofs exist)
- repeated runs are structurally consistent

---

5) Failure Modes (what SYS-01 prevents)

- kits that “look good” but don’t build
- contradictory docs that cause agent confusion
- hidden assumptions that later break the build
- missing security/QA/ops requirements that should have been standard

---

6) Definition of Done (for SYS-01 doc)

SYS-01 is complete when:

- all guarantees are explicit, testable, and unambiguous
- terms used here are defined in SYS-09
- guarantees map cleanly to the gate model (SYS-07) and artifact taxonomy (SYS-04)

SYS-02

SYS-02 — Operating Principles

1) Purpose

Define the universal principles Axion must follow in every pipeline stage, across every artifact, to prevent drift and maintain senior-level consistency.

These principles are **system rules**, not suggestions.

---

2) Principles (locked)

P-01 Separation of Responsibilities (internal vs external)

- **Internal agent** manufactures the kit (fills templates, enforces gates, produces truth artifacts).
- **External agent** builds software using the kit.
- The external agent should never need to reason about Axion internals; it should follow the kit.

P-02 Single Source of Truth

- The **Canonical Spec** is the only place where product truth is defined.
- All other docs must reference spec IDs; no redefining features/roles/workflows/data elsewhere.

P-03 Explicit Unknowns

- Unknowns must be represented as explicit `UNKNOWN` items, not silently inferred.
- Unknowns must be consolidated in a dedicated section in each doc that allows them.
- Unknowns must be labeled with **impact** and **blocking** status.

P-04 Determinism

Given:

- the same validated submission
- the same template library version
- the same standards library version
- the same system version

  …the outputs must be reproducible in structure and constraints.

P-05 “No Forward Progress Without Compliance”

- Every stage has gates.
- If a gate fails, the internal agent must stop and report failures with actionable pointers (field paths, template sections, rule IDs).

P-06 Traceability by Default

Every artifact must include:

- IDs of upstream artifacts it depends on
- version stamps for template/standards/system
- a stable identifier for itself

P-07 Minimum Senior Coverage

The system must cover the senior-level build areas:

- product scope, UX/UI, architecture, implementation, security, quality, ops (where applicable)
  Missing a critical area is treated as a system failure, not a “nice-to-have omission.”

P-08 Constraint First, Creativity Last

Where the build requires decisions:

- prefer resolved standards and explicit user constraints first
- only then allow discretionary choices, and those must be documented as decisions

P-09 Controlled Evolution

System evolution must be governed:

- versioned libraries
- change control for locked contracts
- compatibility notes for older kits

---

3) What these principles constrain (system-wide)

These principles constrain:

- normalization behavior
- standards resolution behavior
- spec building behavior
- template filling behavior
- planning/decomposition behavior
- verification/proof behavior

- kit packaging behavior

---

4) Failure Modes these principles prevent

- duplicating truth across docs
- agent improvisation due to missing constraints
- inconsistent outputs across runs
- silent drift from untracked changes
- un-auditable builds

---

5) Definition of Done

SYS-02 is complete when:

- each principle is enforceable via SYS-07 (gates) or via artifact schema/contract rules
- no principle overlaps redundantly with SYS-01 guarantees (SYS-01 = what must be true; SYS-02 = how we operate to keep it true)
- terms are defined in SYS-09

SYS-03

SYS-03 — End-to-End Architecture (Hardened Draft)

1) Purpose

Define the end-to-end Axion pipeline as a deterministic system: **what stages exist, what each stage consumes, what it produces, and what gates control progression**.

---

2) Pipeline Stages (locked)

Stage 0 — Intake

Consumes: user form inputs + uploads
Produces: Intake Submission Record
Gate: submission is recorded immutably (record creation success)

Stage 1 — Validation

Consumes: Intake Submission Record + Intake Schema
Produces: Validation Result (pass/fail + violations)
Gate: must pass schema + dependency rules + profile thresholds to proceed

Stage 2 — Normalization

Consumes: Validated submission + schema version + normalization rules
Produces: Normalized Input Record (+ normalization report)
Gate: normalization completes deterministically; no invented data; unknowns explicit

Stage 3 — Standards Resolution

Consumes: Normalized input + Standards Library (+ permitted overrides)
Produces: Resolved Standards Snapshot
Gate: snapshot is complete, version-pinned, and conflict-free (or conflicts explicitly blocked)

Stage 4 — Spec Building

Consumes: Normalized input + Resolved standards + vocabulary rules (if used)
Produces: Canonical Spec (with stable IDs + cross-maps + unknowns)
Gate: referential integrity, no duplicate truth, required spec sections present

Stage 5 — Planning / Decomposition

Consumes: Canonical spec + standards constraints
Produces: Work Breakdown (units + deps)
Gate: every unit traces to spec IDs; no cycles; unit size discipline passes

Stage 6 — Acceptance Mapping

Consumes: Work Breakdown + proof rules + quality standards
Produces: Acceptance Map (criteria + proof requirements per unit)
Gate: every unit has minimum acceptance + proof; high-risk units have required checks

Stage 7 — Template Selection & Filling

Consumes: Template Library + selection rules + Canonical spec + Standards snapshot + plans
Produces: Filled document set (templates populated)
Gate: each filled template passes its completeness gate; unknowns handled correctly

Stage 8 — Packaging

Consumes: core artifacts + filled docs + kit layout contract
Produces: Agent Kit folder + zipped kit + manifest/index + entrypoint
Gate: kit structure contract satisfied; required files present; N/A files present where applicable

Stage 9 — Execution (External Agent)

Consumes: Agent Kit
Produces: Built software + proof artifacts + completion report
Gate: acceptance checks and proof requirements satisfied (verification system)

Stage 10 — Continuity (Resumption/Handoff)

Consumes: State Snapshot + work breakdown + acceptance map
Produces: Updated State Snapshot + audit trail
Gate: last-known-good recorded; no skipping gates without override

---

3) Stage Inputs/Outputs Summary (compressed)

- Intake → Submission Record
- Validation → Validation Result
- Normalization → Normalized Input Record
- Standards → Resolved Standards Snapshot
- Spec → Canonical Spec
- Planning → Work Breakdown
- Acceptance → Acceptance Map
- Templates → Filled Docs

- Packaging → Agent Kit
- Execution → Proof + Completion
- Continuity → State Snapshot updates

---

4) System Gates (where drift is blocked)

- **Schema gate** (INT): requiredness, types, dependencies, thresholds
- **Normalization gate** (A2): no invention, deterministic transforms
- **Standards gate** (STD): resolved constraints, version pinned
- **Spec gate** (CAN): referential integrity, no duplicate truth, explicit unknowns
- **Planning gate** (PLAN): traceability, dependency correctness
- **Acceptance gate** (QA/VER): proof required per unit
- **Template gate** (TMP): completeness and consistency of filled docs
- **Packaging gate** (KIT): layout contract, manifest completeness

---

5) Failure Modes (system-level)

- Valid input but non-deterministic output (breaks trust)
- Templates filled with invented content (breaks guarantees)
- Kit missing required artifacts (external agent drift)
- Acceptance lacks proof requirements (claims without evidence)

---

6) Definition of Done

SYS-03 is complete when:

- every stage has clear inputs, outputs, and gating conditions
- stages map to the Artifact Taxonomy (SYS-04)
- no stage depends on undefined artifacts or undefined rules

SYS-04

SYS-04 — Artifact Taxonomy (Hardened Draft)

1) Purpose

Define the complete set of Axion artifacts as a **parts list**, categorized by what they are and how they are used, so the internal agent always knows:

- what artifacts must exist
- which are raw vs derived vs canonical
- which are reusable libraries vs project-specific outputs

---

2) Taxonomy Categories (locked)

T-01 Intake Assets (raw inputs + validity boundary)

Artifacts that capture and validate user intent.

- Intake Form (user-facing)
- Intake Schema (valid submission contract)
- Intake Submission Record (raw, immutable)
- Intake Validator Output (pass/fail + issues)

Property: raw/traceable; not “truth model,” just input boundary.

T-02 Core Data Artifacts (project truth objects)

Artifacts that become the system’s internal and kit-delivered truth.

- Normalized Input Record
- Resolved Standards Snapshot
- Canonical Spec
- Work Breakdown
- Acceptance Map
- State Snapshot

Property: project-specific; version-stamped; referenced by IDs.

T-03 Libraries (reusable stores)

Reusable assets used across projects.

- Template Library
- Standards Library
- Vocabulary/Definitions Library
- Example/Reference Library (optional)

Property: versioned; selected and pinned per project.

T-04 Generators (transforms that produce artifacts)

System processes that manufacture artifacts.

- Normalizer
- Standards Resolver
- Spec Builder
- Planner/Decomposer
- Acceptance Mapper
- Template Selector
- Template Filler
- Packager

Property: deterministic transforms; gated.

T-05 Kit Deliverable (handoff package)

Portable, predictable output package.

- Agent Kit folder (structured)
- Zipped kit
- Entrypoint document
- Kit manifest/index
- Version stamp document

Property: must satisfy layout contract; external-agent facing.

T-06 Verification Assets (proof system)

Artifacts that define and store proof.

- Verification plan/checklist

- Proof log
- Commands list / verification policy
- Completion report format

Property: prevents “claims without evidence.”

T-07 Control Assets (stability over time)

Artifacts that keep the system stable and evolvable.

- Version registry
- Changelog
- Compatibility notes
- Lock rules / change control rules

Property: governs evolution without junkyard growth.

---

3) Artifact Properties (how to classify any artifact)

Every artifact must declare:

- **Type category** (T-01..T-07)
- **Mutability** (immutable / append-only / regenerable)
- **Authority level**
    - raw input / normalized / canonical / derived / library / contract
- **Version stamps** required fields
- **Upstream dependencies**
- **Downstream consumers**

---

4) What artifacts are “authoritative truth”

Only these are authoritative truth objects:

- Canonical Spec
- Resolved Standards Snapshot
- Work Breakdown
- Acceptance Map
- State Snapshot (truth of progress, not product)

Everything else must reference these, not redefine them.

---

5) Failure Modes prevented by this taxonomy

- treating templates as truth (causes contradictions)
- mixing raw submission with normalized/canonical data
- unversioned standards drifting across runs
- kits missing required authoritative artifacts

---

6) Definition of Done

SYS-04 is complete when:

- every artifact used in the pipeline is mapped into exactly one taxonomy category
- “authoritative truth artifacts” are explicitly identified and enforced
- this doc aligns with SYS-03 stage outputs and SYS-07 gating model

SYS-05

SYS-05 — Roles & Responsibilities (Hardened Draft)

1) Purpose

Define who does what in Axion so responsibility boundaries are explicit and the system does not rely on implied behavior. This prevents drift caused by role confusion.

---

2) Primary Actors (locked)

R-01 User (Project Owner)

Responsibilities

- Provide accurate project intent via intake form
- Provide required artifacts (uploads/links) when applicable
- Resolve blocking unknowns when gates require it
- Approve overrides that change locked constraints (if allowed)

What the user can trust

- The kit reflects what they submitted + resolved standards snapshot
- The system will surface missing/contradictory inputs as validation/gate failures

---

R-02 Internal Agent (Axion Runner)

Responsibilities

- Execute the pipeline stages deterministically:
  - validate submission
  - normalize input
  - resolve standards
  - build canonical spec
  - generate work breakdown + acceptance map
  - select and fill templates
  - package agent kit
- Enforce gates:
  - stop on failures
  - produce actionable errors referencing field paths/template sections/rule IDs
- Preserve traceability:
  - version stamps

    - IDs linking upstream artifacts to downstream artifacts

What the internal agent must never do

- invent requirements
- silently fix contradictions
- proceed past a failed gate without a recorded override decision

---

R-03 External Agent (Builder)

Responsibilities

- Read the kit entrypoint and follow the documented build order
- Execute work units and satisfy acceptance/proof requirements
- Record proof artifacts (command output, screenshots, logs) into the proof log
- Update state snapshot (or provide the data required to update it)

What the external agent must never do

- deviate from locked constraints without generating a decision record
- claim completion without proof artifacts

---

3) System Owner / Maintainer (optional but recommended)

R-04 Maintainer (Axion Maintainer)

Responsibilities

- Manage versions of templates, standards, vocab
- Approve system changes under change control
- Maintain compatibility notes and migrations

---

4) Handoff Boundaries (what can be trusted at each boundary)

Boundary B-01: Validated Submission

- Trust: schema compliance, dependencies satisfied, profile thresholds met
- Do not trust: that input is “complete enough” for a perfect build—unknowns may remain

Boundary B-02: Canonical Spec

- Trust: single truth model with stable IDs and referential integrity
- Do not trust: that unknowns are resolved—unknowns must remain explicit

Boundary B-03: Agent Kit

- Trust: contains all required artifacts and filled docs per layout contract
- Do not trust: that external agent will follow it unless verified through proofs

Boundary B-04: Completion

- Trust: only if acceptance criteria and proof requirements are met and logged

---

5) Failure Modes prevented by clear roles

- internal agent acting like a creative builder
- external agent improvising because kit is incomplete
- user expecting system to infer missing requirements silently

---

6) Definition of Done

SYS-05 is complete when:

- responsibilities and “must never do” constraints are explicit for each actor
- handoff boundaries define what is trusted and what is not
- roles align with SYS-03 stage ownership and SYS-07 gating model

SYS-06

SYS-06 — Data & Traceability Model (Hardened Draft)

1) Purpose

Define the required provenance chain so every decision and output in Axion can be traced back to:

- the user's raw submission
- the validated/normalized inputs
- the resolved standards snapshot
- the canonical spec
- the generated plans, filled docs, and proofs

This is the system's audit spine.

---

2) Traceability Chain (locked)

Every project must maintain this linked chain:

1. **Intake Submission Record** (raw, immutable)
2. **Validation Result** (pass/fail + issues; schema version)
3. **Normalized Input Record** (cleaned/standardized + normalization report)
4. **Resolved Standards Snapshot** (pinned ruleset + versions + overrides applied/blocked)
5. **Canonical Spec** (stable IDs + relationships + unknowns)
6. **Work Breakdown** (unit IDs + dependencies + spec mappings)
7. **Acceptance Map** (acceptance IDs + proof requirements per unit)
8. **Agent Kit Manifest/Index** (IDs → file paths + reading order)
9. **Proof Log** (proof IDs linked to acceptance IDs)
10. **State Snapshot** (unit progress + acceptance status + proof refs)

---

3) Required IDs (minimum linkage keys)

Every artifact must include:

- `submission_id` (root of provenance)
- `schema_version` and `form_version` (for intake-related artifacts)
- `standards_version` (for standards snapshot and downstream artifacts)
- `template_library_version` + template versions used (for filled docs)
- `spec_id` (for spec and downstream artifacts)

- `work_breakdown_id` (for planning and downstream)
- `acceptance_map_id` (for verification and downstream)
- `kit_id` (for packaged output)
- `state_id` (for continuity record)

---

4) Link Rules (must always be true)

TR-01 No orphan artifacts

No artifact may exist without a pointer to its upstream parent(s). Minimum:

- everything must link to `submission_id`

TR-02 No silent transformations

Normalization and standards resolution must record:

- what changed
- why it changed
- before/after (or hashed previews)

TR-03 Override traceability

Any override must record:

- what changed (field_path)
- who/what requested it (source)
- reason
- whether it was allowed
- timestamp

TR-04 Proof traceability

Every proof artifact must link to:

- an `acceptance_id`
- which links to a `unit_id`
- which links to spec IDs
  So proof always ties back to “what requirement did this prove?”

---

5) Minimal Trace Fields per Artifact Type (summary)

- **Submission Record:** submission_id, timestamp, submitter, form/schema versions, raw payload, upload refs
- **Validation Result:** submission_id, schema_version, is_valid, issues[]
- **Normalized Input:** submission_id, schema_version, normalized_payload, normalization_report
- **Standards Snapshot:** submission_id, standards_version, resolved_ruleset, overrides[]
- **Canonical Spec:** submission_id, spec_id, standards_version, stable ID index
- **Work Breakdown:** spec_id, units[], scope_refs[]
- **Acceptance Map:** work_breakdown_id, acceptance_items[], proof requirements
- **Kit Manifest:** kit_id, spec_id, template versions, IDs→paths
- **Proof Log:** proof_id, acceptance_id, type, location, timestamp
- **State Snapshot:** state_id, unit_status, acceptance_status, proof refs, last_known_good

---

6) Failure Modes prevented

- “we don’t know where this requirement came from”
- templates drifting away from canonical spec
- proofs that can’t be tied to any acceptance criterion
- re-running the pipeline and getting inconsistent outputs without explanation

---

7) Definition of Done

SYS-06 is complete when:

- the traceability chain is explicit and unbroken
- required IDs are enumerated and unambiguous
- link rules are strict enough to enforce auditability
- this model aligns with artifact specs in A1/A2/A5/A6

SYS-07

SYS-07 — Compliance & Gate Model (Hardened Draft)

1) Purpose

Define how Axion prevents drift: **gates** are the enforcement mechanism that blocks progression until artifacts are valid, consistent, and auditable.

This document defines:

- what a gate is
- where gates occur
- what gate outcomes look like
- what “override” means (and what it must record)

---

2) Gate Definition (locked)

A **Gate** is a deterministic pass/fail check applied to an artifact or stage output.

A gate must specify:

- **Target** (what artifact/output is being checked)
- **Rules** (what must be true)
- **Failure output** (issues with pointers)
- **Severity** (hard stop vs warning)
- **Allowed remediation** (how it can be fixed)
- **Override policy** (whether bypass is possible)

---

3) Gate Types (locked)

GATE-01 Schema Gate (Intake Validity)

Checks submissions against the Intake Schema:

- required fields
- types/enums/formats
- dependencies (if X then Y)
- skill-level thresholds

Hard stop if failing.

GATE-02 Normalization Gate (Transform Integrity)

Checks that normalization:

- did not invent content
- recorded changes in a normalization report
- produced schema-consistent types and canonical naming

Hard stop if invention or invalid types; warnings allowed for minor cleanup.

GATE-03 Standards Gate (Resolved Ruleset Integrity)

Checks standards snapshot:

- version pinned
- defaults + overrides recorded
- fixed vs configurable flags set
- conflicts resolved or explicitly blocked

Hard stop if unresolved conflicts.

GATE-04 Spec Gate (Truth Integrity)

Checks canonical spec:

- stable IDs exist
- referential integrity (no broken references)
- no duplicate truth
- unknowns explicit and indexed

Hard stop if broken references or duplicate truth.

GATE-05 Planning Gate (Work Breakdown Integrity)

Checks work breakdown:

- each unit maps to spec IDs
- dependency graph is acyclic
- units are within size discipline (not “whole app in one unit”)

Hard stop if missing mappings or cycles; warnings for size discipline if you allow.

GATE-06 Acceptance Gate (Proof Completeness)

Checks acceptance map:

- every unit has acceptance items

- hard gates defined
- proof types and verification instructions present

Hard stop if any unit lacks hard-gate acceptance.

GATE-07 Template Gate (Filled Doc Completeness)

Checks filled templates:

- required fields populated (or valid UNKNOWN policy)
- no contradictions inside doc
- cross-references resolve to canonical spec IDs
- template completeness gate passes

Hard stop if required fields missing or contradictions.

GATE-08 Packaging Gate (Kit Contract)

Checks kit:

- file tree contract satisfied
- manifest/index present and correct
- version stamps present
- N/A rule followed (no silent omissions)

Hard stop if contract violated.

GATE-09 Execution Gate (Proof & Completion)

Checks build progress:

- acceptance items pass
- proofs recorded and linked
- state snapshot updated

Hard stop for completion claim without proof.

---

4) Gate Output Contract (what a gate returns)

Every gate produces a report:

- `gate_id`
- `target_artifact_id`
- `status` (pass/fail)

- `issues[]`:
  - `issue_id`
  - `severity` (error/warning)
  - `rule_id` (which rule failed)
  - `pointer` (field_path/template section/unit_id)
  - `message`
  - `remediation` (what to change)
- `timestamp`

---

5) Override Model (controlled bypass)

Overrides are allowed only if policy permits.

An override must record:

- `override_id`
- `gate_id` and failing `rule_id`
- `who/what approved` (user/maintainer/system policy)
- `reason`
- `risk_acknowledged`
- `timestamp`
- downstream impacts (optional but recommended)

Rule: overrides never delete the original failure; they annotate it.

---

6) “No Forward Progress Without Compliance” (system enforcement)

- If a **hard stop** gate fails, pipeline must stop.
- If a **warning** gate fails, pipeline may continue but must record warnings in:
  - kit manifest
  - state snapshot (if execution phase)

---

7) Failure Modes prevented

- moving forward with missing or contradictory requirements
- producing kits with silent omissions
- claiming completion without proof

- untracked bypass decisions

---

8) Definition of Done

SYS-07 is complete when:

- all gate types map to stages in SYS-03
- output contract is consistent across all gates
- override model is unambiguous and auditable

SYS-08

SYS-08 — Configuration Model (Hardened Draft)

1) Purpose

Define how Axion separates:

- what is **fixed system law**
- what is **configurable per project**
- what is **versioned library content**
  so projects are deterministic and the system stays clean over time.

---

2) Configuration Layers (locked)

Layer 1 — System Contracts (Fixed)

These are non-negotiable contracts that must not vary by project:

- intake schema structure contracts (field paths must remain stable across versions unless migrated)
- canonical spec model contracts (ID rules, referential integrity)
- kit file tree contract
- gate output contract

Storage: internal system docs + version registry
Change policy: change control required

Layer 2 — Libraries (Versioned Defaults)

Reusable stores that define defaults and patterns:

- Standards Library (design/security/quality/engineering rules)
- Template Library (template definitions + completeness rules)
- Vocabulary/Definitions Library
- Examples/Reference Library (optional)

Storage: library folders + version stamps
Change policy: version bump + compatibility notes

Layer 3 — Project-Resolved Configuration (Snapshot)

Project-specific resolved results:

- Resolved Standards Snapshot (defaults + overrides, pinned versions)

- Template selection result (which templates apply to this project)
- Any allowed project overrides recorded explicitly

Storage: inside the kit under core artifacts / stamps
Change policy: regenerated or updated via recorded override decisions

Layer 4 — Runtime/Execution State (Mutable)

Per-project, time-varying state:

- State Snapshot
- Proof Log

Storage: kit runtime area or external storage linked by ID
Change policy: append/update allowed; must remain auditable

---

3) Fixed vs Configurable (what belongs where)

Fixed (system law)

- gating model and gate contracts
- ID/reference rules
- required artifact chain for traceability
- kit layout structure and required root files
- “unknown must be explicit” rule
- “no duplicate truth” rule

Configurable (project-level, via standards snapshot)

- tech stack choices (when not fixed globally)
- UI theme/styling rules (within allowed standards)
- auth method selection (within allowed security baseline)
- testing depth expectations (by skill level and standards)
- operational requirements (depending on deployment context)

Library-defined defaults (should not be hand-edited per project)

- standard presets for each category/type/audience
- template structures and required sections
- vocabulary dictionaries

---

4) Override Rules (how config changes legally happen)

An override can only change fields marked configurable.

Each override must record:

- field_path
- before/after
- reason
- who approved
- timestamp
- version stamps

Overrides that attempt to change fixed rules must be blocked or routed to change control.

---

5) Configuration Identification (how a project is “pinned”)

A project run is defined by:

- `schema_version`
- `standards_library_version`
- `template_library_version`
- `system_version`
- resolved standards snapshot hash/id

If these match, the kit should be reproducible (determinism principle).

---

6) Failure Modes prevented

- hidden configuration that changes behavior silently
- “same input, different output” due to unpinned defaults
- project-level edits corrupting global standards
- templates drifting without version tracking

---

7) Definition of Done

SYS-08 is complete when:

- configuration layers are unambiguous
- fixed vs configurable boundaries are explicit
- override rules are aligned with SYS-07 gate override model

- version pinning requirements align with SYS-06 traceability model

SYS-09

SYS-09 — Terminology & Definitions (Post-Completion Update)

1) Purpose

Provide the single authoritative glossary for Axion so all internal docs, gates, and kit artifacts use consistent meaning. SYS-09 is the canonical reference for terms used across SYS/INT/CAN/STD/TMP/ORD/PLAN/VER/KIT/STATE/GOV/EXEC.

---

2) Core Terms (expanded to match finished doc set)

Axion

System that converts user intent into an execution-ready Agent Kit by producing canonical truth artifacts, enforcing standards, and blocking progress via gates.

Internal Agent

Runs Axion. Generates artifacts and the kit; enforces gates; must not build product code; must not modify system assets during normal runs.

External Agent

Builds the product using the Agent Kit. Must follow work breakdown + acceptance map; must log proof; must update state snapshot.

System Update Mode

Maintainer-controlled mode where system assets (templates/standards/contracts/models) may be changed with change control, versioning, and compatibility notes (GOV-01..04). Not permitted during normal kit generation.

Kit Generation Mode

Default mode. Produces an Agent Kit from a submission using pinned versions; system assets are read-only.

---

3) Artifact & Contract Terms

Intake Form (INT-01)

User-facing pages/fields and conditional modules that collect project info.

Intake Schema (INT-02)

Formal validity contract: field paths, types, base constraints, enums. Does not contain conditional requiredness logic.

Validation Rules (INT-03)

Conditional requiredness, thresholds, dependencies, and reference integrity rules.

Submission Record (INT-04)

Immutable stored raw submission + metadata + version stamps + upload refs.

Validation Result (INT-05)

Standard pass/fail output: issues include rule_id + field_path + error_code.

Canonical Spec (CAN-01)

Single authoritative product truth model with entities, constraints, unknowns, and indexes/cross-maps.

ID & Reference Rules (CAN-02)

Rules for stable IDs, uniqueness, reference-by-ID discipline, allowed edges, referential integrity, and no duplicate truth.

Unknown (CAN-03)

First-class object representing missing/ambiguous/contradictory information; includes blocking policy and resolution requirements.

Standards Library (STD-01)

Versioned store of declarative rules organized into packs with applies_when filters and fixed vs configurable flags.

Standards Resolver (STD-02)

Deterministic rules for selecting packs, ordering them, merging conflicts, and applying/denying overrides.

Resolved Standards Snapshot (STD-03)

Project-specific, version-pinned resolved ruleset emitted into the kit; conflict-free; includes applied/blocked overrides.

Template Library / Template Index (TMP-01)

Versioned template inventory registry (`template_index.json`) listing templates, applicability, requiredness, inputs, outputs, dependencies.

Template File Contract (TMP-02)

Required template structure + placeholder syntax + allowed sources + unknown block format.

Template Selection (TMP-03)

Deterministic selection of templates based on routing and gate flags; produces selected/omitted lists with reasons and ordering.

Template Fill Rules (TMP-04)

Deterministic placeholder resolution from canonical artifacts; derived inserts; standards injection; unknown creation policy.

Template Completeness Rules (TMP-05)

Template Gate pass/fail rules: structure, resolved placeholders, reference integrity, no invention, standards compliance, skill-level requiredness.

Build Order Graph (ORD-01)

Doc/artifact dependency graph + valid topological build order + critical path; must be acyclic.

Gate DSL / Gate Rules (ORD-02)

Minimal deterministic rule language for expressing compliance checks and generating gate reports.

Per-Doc Gate Checklist (ORD-03)

Standard per-doc gate checklist format using the gate DSL; maps DoD to enforceable checks.

Work Breakdown (PLAN-01)

Ordered work units with unit_ids, scope_refs (spec IDs), dependencies, deliverables, risk flags.

Acceptance Map (PLAN-02)

Acceptance criteria per unit with hard/soft gating and proof requirements; links acceptance_id ↔ unit_id ↔ scope_refs.

Sequencing Heuristics (PLAN-03)

Allowed deterministic tie-breakers for ordering eligible work units; dependencies always dominate.

Proof (VER-01)

Acceptable evidence types (command output/test result/screenshot/log excerpt/diff ref/checklist) with required metadata and linkage.

Verification Command Policy (VER-02)

Rules for when/how checks must be run; capture requirements; failure handling; acceptance linkage.

Completion Criteria (VER-03)

Global rules for unit completion and project completion based on hard-gate acceptance + proof + no blocking unknowns.

Agent Kit (KIT-01)

Portable deliverable with locked folder structure, required root files, core artifacts, pack folders, and N/A stubs.

Manifest / Index (KIT-02)

`00_KIT_MANIFEST.md` contains authoritative JSON mapping IDs→paths and reading order; `00_KIT_INDEX.md` mirrors it for humans.

Entrypoint (KIT-03)

`00_START_HERE.md` single start file; defines execution loop; declares truth sources; prohibits skipping/proofless claims.

Version Stamping (KIT-04)

`00_VERSIONS.md` includes machine-readable JSON of pinned versions for system, intake, standards, templates, model, planning, proof, kit contracts.

State Snapshot (STATE-01)

Machine-readable continuity record: unit_status, acceptance_status, proof_refs, blockers, unknown statuses, last-known-good, history/events.

Resume Rules (STATE-02)

Deterministic procedure to restart work using artifacts and state snapshot; no chat-memory authority.

Handoff Rules (STATE-03)

What must be recorded before switching agents/sessions and how the receiving agent resumes deterministically.

Versioning Policy (GOV-01)

What is versioned and when versions must bump; breaking vs non-breaking definitions.

Change Control (GOV-02)

Required change record, approval rules, governance gates, and activation rules for system updates.

Deprecation & Migration (GOV-03)

Deprecation records, replacement mappings, migration guides, compatibility notes required for breaking changes.

Audit / Traceability Rules (GOV-04)

Enforceable provenance requirements for artifacts, versions, proofs, and system changes; defines audit failure conditions.

Internal Agent Runbook (EXEC-01)

Deterministic end-to-end procedure internal agent follows to produce artifacts + kit and enforce gates.

External Agent Prompt Template (EXEC-02)

Standard builder-agent prompt that enforces truth sources, execution loop, proof/state discipline, and stop conditions.

Failure Handling Playbook (EXEC-03)

Standardized blocker/gate failure handling, what to record, and how to request missing input.

---

4) Consistency Rules (how terms must be used)

- “Canonical Spec” is the only product truth model.
- “Template” is guidance output and must not redefine truth.
- “Gate” implies deterministic pass/fail with a gate report and rule_id.
- “Override” implies a recorded before/after + approval + timestamp.
- “Proof” must link to acceptance_id and unit_id; otherwise it is not proof.
- “Manifest JSON is authoritative” for paths; index mirrors it.

---

5) Definition of Done (SYS-09 update)

SYS-09 is complete when:

- every term used as a title or key concept in the finalized doc set appears here
- GOV-04 naming is consistent (“Audit / Traceability Rules”)
- no duplicate/conflicting definitions exist across docs (SYS-09 wins)

SYS-10

SYS-10 — System Boundaries (Hardened Draft)

1) Purpose (Expanded + Hardened)

1.1 Purpose Statement (locked)

SYS-10 exists to define **hard boundaries** for Axion so the internal agent cannot expand scope implicitly. These boundaries are enforced through gates, overrides, and versioning rules so the system remains:

- **Stable:** behavior doesn't change run-to-run without a recorded version change
- **Non-bloated:** new "helpful" behaviors don't accumulate as untracked features
- **Predictable:** every action the internal agent takes is authorized by system docs
- **Auditable:** boundary decisions are traceable and reviewable
- **Deterministic:** same inputs + same versions produce the same outputs and constraints

1.2 What this document controls (authority)

SYS-10 is a **system law document**. It controls:

- internal agent behavior limits (what actions are prohibited)
- pipeline scope limits (what stages can and cannot do)
- what belongs in kits vs what does not
- what requires change control versus normal configuration

SYS-10 supersedes any "helpful" behavior found elsewhere if there is conflict.

1.3 Why SYS-10 exists (failure prevention)

This document exists to prevent these specific system failure classes:

1. **Scope Creep Drift**

- internal agent begins doing extra work “to be helpful” (e.g., generating code, inventing requirements, deciding stack choices not authorized)

2. **Junkyard Growth**

- repeated small additions create inconsistent and messy repo/system behavior (new docs, new rules, new exceptions with no governance)

3. **Hidden Assumption Injection**

- internal agent fills gaps silently, producing kits that cannot be audited or reproduced

4. **Contract Erosion**

- internal agent bypasses gates, changes file layout, or shifts responsibilities between internal/external agents without recorded decision and versioning

5. **User Trust Breakage**

- users can’t tell what Axion guarantees, what it does, or why outputs changed

1.4 Enforcement Requirement (non-negotiable)

Every boundary in SYS-10 must have an enforcement mechanism:

- **Gate enforcement:** Packaging Gate / Template Gate / Spec Gate / etc. must fail on violations

- **Override enforcement:** if any boundary can ever be bypassed, the bypass must require an override record

- **Version enforcement:** boundary changes require versioning + change control

Rule: A boundary without enforcement is not a boundary. It must be treated as incomplete.

1.5 Output of SYS-10 (what it enables)

SYS-10 enables:

- strict internal-agent operating constraints (“what it must never do”)

- a clean separation between internal manufacturing vs external building

- a stable baseline for governance: change control can only act on defined boundaries
- consistent maintenance: the system can evolve without accumulating contradictions

1.6 Definition of Done (for Section 1)

Section 1 is complete when:

- it explicitly states SYS-10's authority and enforcement requirement
- it defines the failure classes SYS-10 prevents
- it states that boundaries must be tied to gates/overrides/versioning
- it contains no ambiguous language ("should," "usually," "ideally") in any enforceable claim

---

SYS-10 — Section 2) Axion Boundaries (Locked)

Below is the **hardened** version of Section 2. I'm keeping your boundary list exactly, but upgrading each boundary into **enforceable law**: scope, prohibitions, gate enforcement, override policy, and pass/fail checks.

B-01 Axion does not build the product (Hardened)

Boundary statement: Axion manufactures the Agent Kit; the external agent builds the software.

B-01.1 Allowed outputs (Axion may produce)

- truth artifacts (normalized input, standards snapshot, canonical spec, work breakdown, acceptance map, initial state snapshot)
- filled templates (guidance only)
- packaging artifacts (kit structure, manifest/index, entrypoint, version stamps)
- validation and gate reports
- proof requirements and proof formats (not proofs)

B-01.2 Prohibited actions (Axion must not)

- produce product source code as "deliverables"

- claim feature implementation is complete
- “test-run” the target product build
- make ad-hoc implementation decisions not derived from standards or user inputs

B-01.3 Enforcement (hard gate)

- **KIT Packaging Gate** must fail if:
    - code deliverables exist beyond explicitly allowed scaffolding policy, OR
    - any doc claims build completion, OR
    - acceptance/proof requirements are missing
- **Issue format:** `SYS10-B01-VIOLATION` with file path pointers

B-01.4 Override policy

- **Not overridable**. This is a system boundary.

B-01.5 Pass/Fail checklist

- Kit contains no implementation deliverables (unless explicitly allowed scaffolding and labeled).
- No docs claim “implemented” work.
- Work Breakdown + Acceptance Map exist and are referenced from entrypoint.

B-02 Axion does not invent requirements (Hardened)

Boundary statement: Axion never guesses missing requirements.

B-02.1 Allowed handling of missing info (only)

1. explicit `UNKNOWN` with impact and blocking flag
2. gate failure requiring user input

B-02.2 Prohibited actions

- inferring features, entities, roles, permissions, workflows, constraints not provided
- filling placeholders with “reasonable assumptions” without marking UNKNOWN
- creating “default requirements” beyond the resolved standards snapshot

B-02.3 Enforcement (hard gates)

- **Normalization Gate** fails if it introduces non-traceable new requirements

- **Spec Gate** fails if canonical spec contains invented objects not traceable to normalized input or standards snapshot
- **Template Gate** fails if required placeholders are filled with invented content rather than UNKNOWN

B-02.4 Override policy

- Overridable only by explicit user decision **as new input**, not as a validator bypass.
    - i.e., you can update the submission; you cannot “override” invention.

B-02.5 Pass/Fail checklist

- Every non-trivial requirement traces to submission or standards snapshot.
- Unknowns are explicit, indexed, and never silently resolved.

B-03 Axion does not replace human intent (Hardened)

Boundary statement: Axion cannot decide priorities/scope/brand identity beyond captured intent.

B-03.1 Allowed actions

- structure and normalize user input
- apply standards defaults where permitted
- ask for resolution when intent is missing/contradictory (via gate failures)

B-03.2 Prohibited actions

- inventing business priorities or scope tradeoffs
- redefining “success” beyond user definition
- generating brand identity beyond user-provided direction for consumer-facing products

B-03.3 Enforcement (gates)

- **Spec Gate** fails if it introduces scope/priorities not in input
- **Template Gate** fails if PRD/Design templates contain “decisions” not traceable
- **Decision Log requirement:** any discretionary choice must be marked as a decision with source and rationale (and may be blocked depending on policy)

B-03.4 Override policy

- Overridable only by user-provided inputs or approved overrides in configurable areas.

B-03.5 Pass/Fail checklist

- Priorities, success criteria, and scope are identical to user inputs unless explicitly overridden and logged.
- Consumer-facing brand direction exists or is explicitly UNKNOWN (and treated per gates).

B-04 Axion does not override locked rules silently (Hardened)

Boundary statement: Deviations from fixed contracts/resolved standards require explicit override.

B-04.1 Prohibited actions

- changing any locked contract (kit tree, canonical spec schema, gate rules) without change control
- applying user preferences that conflict with fixed standards silently

B-04.2 Enforcement (hard gates)

- **Standards Gate** fails if an override is applied without:
  - field_path
  - before/after
  - source
  - permission check
  - timestamp
- **Packaging Gate** fails if version stamps don't record standards/template/system versions

B-04.3 Override policy (strict)

Overrides must include:

- `override_id`, `field_path`, `before`, `after`, `reason`, `approved_by`, `approved_at`, `permission_basis`

B-04.4 Pass/Fail checklist

- All overrides are recorded and permission-validated.
- No silent changes exist between snapshot and output docs.

B-05 Axion does not guarantee correctness of external execution (Hardened)

Boundary statement: Axion guarantees kit quality/traceability, not external implementation correctness.

B-05.1 What Axion guarantees

- acceptance criteria exist for each work unit
- proof requirements exist for each acceptance criterion
- verification commands and/or verification procedures are specified

B-05.2 What Axion must not claim

- “the build works”
- “the feature is implemented”
- “tests pass”
  unless accompanied by proof artifacts produced during execution (external phase)

B-05.3 Enforcement

- **Completion Gate** fails if state snapshot marks unit done without passing hard-gate acceptance + proof refs.
- **Report Gate** fails if completion report contains claims without referenced proof IDs.

B-05.4 Pass/Fail checklist

- Completion claims always reference proof IDs.
- Proof IDs map to acceptance IDs and unit IDs.

B-06 Axion does not handle production operations by default (Hardened)

Boundary statement: Ops is included only when required by scope/routing.

B-06.1 Allowed behavior

- include Ops templates and constraints when:

    - project requires deployment/runtime operations, OR
    - user explicitly asks for ops coverage, OR
    - preset implies operational complexity

B-06.2 Prohibited behavior

- generating heavy ops requirements for simple builds without need
- running deployments or managing infrastructure

B-06.3 Enforcement

- **Template Selection Gate** must justify inclusion/exclusion of ops templates.
- Ops slot must be either filled or `00_NA.md` (no silent omission).

B-06.4 Pass/Fail checklist

- Ops coverage is present only when justified, otherwise N/A with reason.
- No deployment execution occurs inside Axion.

B-07 Axion does not store “hidden state” as truth (Hardened)

Boundary statement: Only recorded artifacts are authoritative; no implicit memory.

B-07.1 Authoritative state sources (only)

- submission record
- validation report
- normalized input
- standards snapshot
- canonical spec
- work breakdown
- acceptance map
- kit manifest/index
- state snapshot + proof log

B-07.2 Prohibited behavior

- relying on agent conversation memory as authoritative
- making decisions based on “prior runs” without recorded state snapshot links
- storing untracked state in ad-hoc notes

B-07.3 Enforcement

- **Traceability Gate** fails if an artifact references an untracked dependency ("as discussed" with no ID link).
- **Resume Gate** fails if current run cannot be reconstructed from stored artifacts.

B-07.4 Pass/Fail checklist

- Every decision references an artifact ID, not conversational history.
- State snapshot is the only continuity anchor.

B-08 Axion does not permit uncontrolled evolution (Hardened)

Boundary statement: System changes require change control and versioning.

B-08.1 Prohibited behavior

- editing templates/standards/contracts without version bump
- changing kit layout without compatibility notes
- breaking older kits without migration policy

B-08.2 Enforcement

- **Governance Gate** fails if:
  - a locked artifact changes without changelog entry
  - version registry not updated
  - compatibility notes missing when required

B-08.3 Pass/Fail checklist

- Every change has a version bump and changelog entry.
- Compatibility notes exist for breaking changes.
- Old kits remain interpretable via pinned versions.

Section 3) Allowed Extensions (Rewritten + Extra Hardened)

3.0 Non-Negotiable Rule: No Self-Modifying Runs

During **normal Agent Kit generation**, the internal agent must treat all system assets as **read-only**:

- Template Library
- Standards Library
- Gate rules
- Canonical Spec model/schema
- Kit layout contracts
- System Overview docs

If any of these change during a normal run: the run must **fail immediately** as a boundary violation.

3.1 Purpose

This section defines the **only safe lanes** for Axion to evolve **between runs** under explicit governance. It is a maintainer-controlled capability, not a runtime behavior.

3.2 When Extensions Are Allowed (only these contexts)

Extensions are allowed only when the system is explicitly in **System Update Mode**, meaning:

- a maintainer has initiated an update operation
- a new system version is being authored
- changes will be versioned, reviewed, and recorded

Extensions are **not allowed** in:

- normal kit generation mode

- external agent execution mode
- resume/handoff mode

3.3 Safe Extension Lanes (the only allowed change types)

E-01 Template Library Changes (add/modify/remove templates)

Allowed only if:

- template has a single clear purpose
- no duplicate truth (references Canonical Spec IDs)
- required fields are fillable without guessing (UNKNOWN or gate fail)
- completeness gate exists and is objective
- Template Index updated
- template library version bumped
- compatibility notes written if any kit outputs change materially

E-02 Standards Library Changes (add/modify standards)

Allowed only if:

- standards are versioned
- fixed vs configurable is explicit per rule
- resolution order is deterministic (defaults → presets → overrides → snapshot)
- standards snapshot format remains compatible or has migration notes
- standards library version bumped

- compatibility notes written if outputs change

E-03 Gate Model Changes (add/modify gates)

Allowed only if:

- gate is deterministic
- gate target + rule IDs are explicit
- gate report output contract is unchanged or versioned
- override policy is explicit
- change does not duplicate an existing gate unless it tightens enforcement
- system version bumped
- compatibility notes written if behavior changes

E-04 Canonical Spec Model Changes (schema evolution)

Allowed only if:

- spec model version bumped
- migration path exists (old → new mapping rules)
- ID/reference invariants are preserved or explicitly migrated
- impacted templates updated + versioned
- compatibility notes written for old kits/specs

3.4 Enforcement Rules (must be true for any extension)

Any extension must produce:

- version bump (system/library/schema as applicable)
- changelog entry (what changed + why)
- compatibility notes (what breaks/what migrates)
- updated indexes/registries (template index, standards index, etc.)
- a governance gate pass report confirming compliance

Rule: If any of these are missing, the extension is invalid.

3.5 Explicitly Disallowed Changes (unsafe growth)

Disallowed in all contexts:

- self-modifying behavior during normal runs
- unversioned edits to templates/standards/contracts
- changes that introduce hidden state as truth
- changes that allow guessing/invention as a fallback
- changes that break the kit layout without migration/compat notes

---

3.6 Definition of Done (for Section 3)

Section 3 is complete when:

- System Update Mode is explicitly the only place extensions occur
- normal runs are explicitly read-only for system assets
- each safe lane has enforceable requirements
- unsafe changes are explicitly prohibited

Section 4) Boundary Violation Examples (Expanded + Hardened)

4.1 Purpose

This section defines **concrete, unambiguous examples** of what boundary violations look like in practice so the internal agent can detect them and gates can fail deterministically.

Rule: Every violation example must map to:

- a boundary (B-01..B-08)
- a gate that must fail (schema/spec/template/packaging/completion)
- a required remediation action

4.2 Violation Examples (locked set)

V-01 Invented Features in Filled Templates

Boundary: B-02 (does not invent requirements) + B-03 (does not replace intent)
Violation pattern: a filled template includes a feature/capability not present in Canonical Spec or user submission.

Concrete examples

- PRD-03 lists “Team collaboration” but Canonical Spec has no such capability ID.
- DES-04 includes a “Shared Calendar” screen not referenced by any workflow or feature.
- IMP guide instructs building “push notifications” when not requested and not in standards snapshot as required.

Gate enforcement

- **Template Gate** fails with:
    - `rule_id: SYS10-V01-INVENTION`

    - pointers to the template section and the missing canonical spec reference

- **Spec Gate** must fail earlier if the invention entered the Canonical Spec.

Required remediation

- Either:
    - mark as UNKNOWN (only if it is truly a missing requirement), **or**
    - remove the invented content, **or**
    - update intake input explicitly and re-run pipeline.

V-02 Progressing Past Failed Intake Validation

Boundary: B-04 (no silent override) + SYS-07 “No forward progress without compliance”
Violation pattern: pipeline continues to normalization/spec/templates when intake validation failed.

Concrete examples

- Validation report includes missing required fields but Normalized Input Record is generated anyway.
- Intake schema thresholds not met (workflows/out-of-scope) but canonical spec is built.
- Dependency rule violated (build_target=existing but repo link missing) but planner runs.

Gate enforcement

- **Intake Schema Gate** is a hard stop. If failed:
    - pipeline must terminate and output validation issues.
- Any downstream artifact created after a failed intake gate triggers:
    - `rule_id: SYS10-V02-FORWARD-PROGRESS-WITH-FAIL`

    - immediate run invalidation.

Required remediation

- user corrects submission or provides missing info
- revalidate successfully before continuing

V-03 Kit Layout Changed Without Version Bump + Compatibility Notes

Boundary: B-08 (no uncontrolled evolution) + configuration/versioning laws
Violation pattern: kit file tree or required file placements change without version governance.

Concrete examples

- `00_START_HERE.md` renamed or moved with no kit contract version bump.
- `01_core_artifacts/03_canonical_spec.json` path changes without manifest contract update.
- pack folder layout changed (e.g., `30_features/` renamed) and older kits become unreadable.

Gate enforcement

- **Packaging Gate** fails with:
    - `rule_id: SYS10-V03-UNVERSIONED-KIT-CONTRACT-CHANGE`
    - pointers to violated required paths
- **Governance Gate** fails if:
    - version registry not updated
    - compatibility notes missing

Required remediation

- revert layout change, or
- bump kit contract version + add compatibility notes + migration guidance

---

V-04 Allowing External Agent to Interpret Missing Constraints

Boundary: B-03 (does not replace intent) + B-07 (no hidden state)
Violation pattern: kit contains ambiguous or missing constraints and instructs external agent to "decide" core requirements.

Concrete examples

- Security templates say "choose an auth method" with no standards constraints and no user input.
- Design templates say "pick a UI style" without any user direction or standards snapshot.
- Implementation docs instruct "decide data model as needed" without canonical spec entities.

Gate enforcement

- **Template Gate** fails if any required decision area is left as "agent decides" instead of:
    - resolved standard
    - explicit user requirement
    - explicit UNKNOWN (with gate behavior)
- **Spec Gate** fails if canonical spec lacks required constrained decisions while downstream docs proceed.

Required remediation

- capture the missing constraint via intake input or standards resolution
- represent as UNKNOWN if allowed and non-blocking

- regenerate affected artifacts

4.3 Additional Mandatory Violation Classes (to make this strict)

These are also boundary violations and must be treated as failures:

V-05 Silent Override (changes without override record)

- Boundary: B-04
- Gate: Standards Gate / Packaging Gate

V-06 Hidden State Reliance (“as discussed earlier” with no artifact reference)

- Boundary: B-07
- Gate: Traceability Gate

V-07 Proofless Completion Claims

- Boundary: B-05
- Gate: Completion Gate

V-08 Self-Modifying Run (libraries/contracts edited during kit generation)

- Boundary: Section 3 (No self-modifying runs) + B-08
- Gate: Governance Gate / Packaging Gate

4.4 Definition of Done (for Section 4)

Section 4 is complete when:

- every violation maps to a boundary and a gate
- each has a required remediation
- violation patterns are concrete enough to detect automatically
-

Section 5) Definition of Done (Expanded + Hardened)

5.1 Purpose

Define what it means for **SYS-10** to be complete, enforceable, and safe to lock as a system-law document.

Rule: SYS-10 cannot be considered “done” unless its boundaries are enforceable through gates, versioning rules, or explicit run-mode constraints.

5.2 Required Completion Conditions (must all be true)

DOD-01 Boundary Coverage Completeness

- All boundaries **B-01 through B-08** are present.
- Each boundary includes:
    - prohibited actions
    - allowed behaviors (where relevant)
    - enforcement mechanism (gate + rule_id)
    - override policy (overridable vs non-overridable)
    - pass/fail checklist

Fail condition: any boundary lacks enforcement.

DOD-02 Enforcement Mapping Exists

For every boundary, SYS-10 must map to at least one enforcement point:

- Intake/Schema Gate
- Spec Gate
- Template Gate
- Packaging Gate
- Completion/Proof Gate
- Governance Gate
- Traceability Gate

Each boundary must declare:

- `rule_id` format: `SYS10-B0X-*` or `SYS10-V0X-*`
- pointers required in gate reports (field_path / file_path / template_section / unit_id)

Fail condition: boundary exists only as narrative without a gate mapping.

DOD-03 Run-Mode Separation Is Explicit

SYS-10 must explicitly define:

- **Normal Kit Generation Mode** = read-only system assets
- **System Update Mode** = allowed extension lanes under change control

And must state:

- any system-asset change during normal runs is a hard failure

Fail condition: ambiguity about when extensions are permitted.

DOD-04 Traceability Compliance

SYS-10 must align with SYS-06 by requiring:

- no reliance on chat memory as truth
- all decisions must reference artifact IDs
- proof must link acceptance_id → unit_id → spec scope refs

Fail condition: any boundary implies “implicit memory” as acceptable.

DOD-05 Versioning & Compatibility Alignment

SYS-10 must require:

- version pinning for kits (system/templates/standards/schema)
- compatibility notes for breaking changes
- prohibition of unversioned layout/contract changes

Fail condition: kit contract changes can occur without version bump + notes.

DOD-06 Concrete Violation Examples Included

SYS-10 must include a Boundary Violation Examples section where:

- each example maps to a boundary + gate + remediation
- examples are specific enough to be detected (not vague)

Fail condition: examples are too abstract to enforce.

DOD-07 No Conflicting Language

SYS-10 must not contain:

- “should,” “usually,” “ideally” for enforceable statements
- conflicting guidance with SYS-01 guarantees or SYS-07 gate model

Fail condition: any enforceable rule is expressed as optional.

5.3 Lock Criteria (when SYS-10 becomes LOCKED)

SYS-10 becomes **LOCKED** when:

- all DOD conditions above are met
- governance registry records SYS-10 version
- change control policy marks SYS-10 as system-law

After lock:

- changes require change control + version bump + changelog entry

---

5.4 Outputs of Completion (what “done SYS-10” enables)

Once complete, SYS-10 enables:

- deterministic stopping conditions (no drift past boundaries)
- safe system evolution (via System Update Mode only)
- enforceable separation between internal manufacturing and external building

---

5.5 Definition of Done Checklist (short form)

- B-01..B-08 expanded with enforcement + override policy + checklist
- Each boundary mapped to a gate with rule IDs
- Normal vs Update mode explicitly enforced
- Traceability and proof linkage requirements align with SYS-06
- Versioning/compatibility requirements align with governance
- Concrete violation examples map to boundaries + remediation
- No optional language in enforceable rules
- SYS-10 marked LOCKED in version registry (once governance docs exist

Intake & Validation

Intake & Validation

- INT-01 Intake Form Spec (field/page layout)
- INT-02 Intake Schema Spec (validity contract)
- INT-03 Intake Validation Rules (dependencies + thresholds)
- INT-04 Submission Record Format (traceability)
- INT-05 Validator Output Format (errors/warnings contract)

INT-01

INT-01 — Intake Form Spec (Field/Page Layout)

(Hardened Draft — Full)

1) Purpose

Define the **exact intake surface** the user submits through, including:

- page order and page intent
- every field presented to the user
- field UI types (text, dropdown, chips, uploads, etc.)
- which sections are conditional based on routing (matrix axes)
- how “autofill draft” is offered (optional, user-controlled)

This spec exists so the internal agent can treat intake as a **stable contract surface** and so the schema/validator can remain aligned to what the user can actually submit.

---

2) Inputs

- Template Matrix axes (routing):
  - `skill_level` (beginner/intermediate/expert)
  - `category` (application/service/infrastructure/library_sdk)
  - `type_preset` (category-scoped)
  - `build_target` (new/existing)
  - `audience_context` (consumer/internal/developer)
  - `autofill` (user choice)
- Intake Inventory (all possible intake fields and modules)
- File upload capabilities (zip, logo, etc.)

---

3) Outputs

- A user submission payload (raw answers) that conforms to:
  - INT-02 Intake Schema Spec (field paths + types)
  - INT-03 Validation Rules (dependencies + thresholds)
- A predictable “what the user saw” mapping, by:
  - form_version
  - routing snapshot
  - included modules list

---

4) Rules / Invariants (must always be true)

1. **Schema alignment:** every form field must map to a schema field path (INT-02). No orphan fields.
2. **No unreachable schema:** schema must not require fields the form cannot collect under that routing.
3. **Routing first:** Page 0 routing choices must occur before any conditional modules appear.
4. **Autofill is opt-in:** autofill drafting must never run automatically; the user explicitly toggles it.
5. **Conditional clarity:** if a section is omitted due to routing, it must be truly non-applicable (not silently skipping required info).
6. **Version stamping:** every submission stamps the exact `form_version`.

---

5) Page Structure (Locked Page List)

Page 0 — Routing (Matrix Selector)

Goal: determine what form modules should appear.

Fields:

- Skill level (radio)
- Category (dropdown)
- Type preset (dropdown/cards; filtered by category)
- Build target (radio)
- Audience context (radio)
- Autofill draft answers (toggle; default OFF)

Page 1 — Project Basics + Inputs

Goal: capture project identity + baseline inputs.

Sections:

1. Project Identity
- Project name (text)

- Project overview (large textarea)
- One-liner (optional text)
- Core experience paragraph (optional textarea)

2. Source Materials

- Upload zip (file upload .zip)
- Reference links (repeatable URL + label)

3. Existing Project Context (conditional: build_target=existing)

- Existing repo link (URL)
- Current state summary (textarea)
- Must-not-change list (list builder)
- Known issues list (repeatable title + description)

4. Delivery Preference

- Desired scope (radio: mvp/full/unsure)
- Priority bias (radio: speed/balanced/quality)

Page 2 — Product Intent + Brand Ideology

Goal: capture the “why/for who/success” and consumer ideology.

Always-present sections:

- Problem statement (textarea)
- Why now (optional textarea)
- Alternatives/workarounds (repeatable name + optional link + what's missing)
- Audience description (textarea)
- Primary user label (text)
- Secondary users (repeatable text)
- Primary goals (repeatable text)
- Success definition (textarea)
- Success metrics (repeatable metric + optional target/timeframe)
- Out of scope (repeatable text)

Consumer-only blocks (conditional: audience_context=consumer):

- What it stands for (textarea)
- Change created (optional textarea)
- Core values (chips + custom)
- Beliefs shaping decisions (chips + custom)
- Refusal guardrails (textarea)
- Psychographics (textarea)
- User cares about / fears avoids (textarea)
- Aligned communities (repeatable)
- Status signal (dropdown)

- Brand promise (text)
- Positioning statement (optional textarea)
- Unique advantage (optional textarea)
- Voice adjectives (chips + custom)
- Tone boundaries (optional textarea)
- Example phrases (repeatable)
- Content maturity (radio)
- Emotional journey questions (optional textareas)

Page 3 — Design Direction (New Page)

Goal: ensure agents don't guess UI direction.

Sections:

- Style adjectives (chips + custom) *(required for consumer)*
- Visual preset (dropdown)
- Avoid list (list builder)
- Logo upload (optional)
- Brand colors (repeatable name + hex input)
- Fonts (repeatable name + use-case)
- Brand references (repeatable URL + notes)
- UI density (radio)
- Navigation preference (dropdown)
- Component vibe (chips)
- Motion preference (radio)
- Accessibility expectations (checklist + notes)
- Content presentation preference (radio)
- Showcase screens (repeatable: name + priority + notes + optional ref URL/file)

Page 4 — Functional Spec

Goal: convert intent into buildable structure.

Sections:

1. Features
- Must-have features (repeatable name + description)
- Nice-to-have features (repeatable)
- Future backlog (repeatable)
- Priority ranking (drag/drop list of must-have)

2. Roles
- Roles list (repeatable role subform)
3. Permissions
- Role permissions builder (allowed/restricted/approval-required)
4. Workflows
- Core workflows (repeatable workflow builder; steps list)
- Edge workflows (optional)
5. Business rules
- Business rules exist toggle
- Must-always rules (repeatable; required if enabled)
- Must-never rules (repeatable)
- Validation rules (repeatable)
- Lifecycle rules (repeatable)

Page 5 — Data Model (Conditional gate)

Goal: capture data truth when applicable.

Gate field:

- Does the product store/manage data? (yes/no)

If yes:

- Data objects/entities (repeatable subform)
    - fields (required/optional)
    - relationships
    - lifecycle states
- Sensitive data flags (multi-select)
- Import/export needs (multi-select)
- Retention expectations (textarea)
- Ownership/tenancy notes (textarea)

Page 6 — Auth & Authorization (Conditional gate)

Goal: capture auth requirements when applicable.

Gate field:

- Authentication required? (yes/no)

If yes:

- Auth methods (multi-select)
- Account lifecycle (multi-select)
- 2FA required? (yes/no)
- Session rules (multi-select)
- Authorization model (dropdown)
- Approval flows needed? (yes/no)
    - Approval flow description (textarea)

Page 7 — Integrations (Conditional gate)

Goal: capture external dependencies.

Gate field:

- Any external integrations? (yes/no)

If yes:

- Integrations list (repeatable subform):
    - service name, purpose, data in/out, triggers, secrets handling

---

Page 8 — Non-Functional Requirements (NFR)

Goal: capture performance/reliability/compliance expectations.

- Performance targets (repeatable metric + target)
- Scale assumptions (expected users, concurrent users, notes, unsure)
- Reliability expectation (dropdown)
- Offline required? (yes/no)
- Compliance flags (multi-select + notes)

Page 9 — Category/Type Specific Page (conditional)

Only one of these is shown based on routing.category:

9A) Application UI Structure

- Screens list (repeatable: name/purpose/roles/data/actions/states)
- Navigation summary (sections + role diffs)
- UX requirements checklist

9B) Service Structure

- Responsibilities (textarea)
- Operations (repeatable)
- API consumers (repeatable)
- Endpoints/actions (repeatable)
- Webhooks (repeatable)

9C) Infrastructure Structure

- Infra goal (textarea)
- Environments (multi-select)
- Runtime expectations (multi-select)
- Observability needs (multi-select)
- Backup/recovery notes (textarea)

9D) Library/SDK Structure

- Target languages (multi-select + custom)
- Developer persona (textarea)
- API surface (repeatable)
- Init/config pattern (textarea)
- Packaging/distribution (text)
- Docs/examples expectations (multi-select)

Page 10 — Final Verification + Confirmations

Goal: prevent “agree later” drift.

- Definition of done (textarea)
- Must-pass checks (multi-select + custom)
- Acceptance criteria statements (repeatable)
- Rejection conditions (textarea)
- Confirm priorities are correct (checkbox must be true)
- Confirm out-of-scope is real (checkbox must be true)
- Confirm constraints are accurate (checkbox must be true)

---

6) Failure Modes

- Form collects fields not in schema (orphan fields)
- Schema requires fields that are never shown (unreachable requirements)
- Autofill runs without user opt-in

- Routing choices conflict with shown modules (wrong module set)
- Conditional pages omitted when required by routing (missing foundations)

---

7) Definition of Done (INT-01)

INT-01 is complete when:

- every page and field is enumerated
- every field maps to a schema path (INT-02)
- conditional inclusion rules are explicit (by routing/gates)
- autofill behavior is explicitly opt-in
- form_versioning rules are defined and enforceable

INT-02

INT-02 — Intake Schema Spec (Validity Contract)

(Hardened Draft — Full)

1) Purpose

Define the **formal validity contract** for intake submissions so the system can deterministically decide whether a submission is usable.

INT-02 defines:

- canonical field paths and data types
- required vs optional fields
- allowed values (enums)
- constraints (min/max length, min items, formats)
- which sections may appear (schema shape)

INT-02 does not define dependency logic ("if X then Y") or skill thresholds; that is INT-03.

---

2) Inputs

- INT-01 Intake Form Spec (what can be submitted)
- Locked routing axes and field inventory

---

3) Outputs

- A stable schema contract that:
  - the Validator enforces
  - downstream stages trust
- `schema_version` required in all submissions and validation reports

---

4) Rules / Invariants (must always be true)

1. **One canonical path per field** (no aliases).
2. **Form-aligned**: schema must not require fields that INT-01 cannot collect.
3. **Backwards-aware**: changes require `schema_version` bump and compatibility notes (governance).

4. **No dependency logic here**: conditional requirements live in INT-03.
5. **No guessing**: schema supports explicit UNKNOWN where policy allows (handled by templates/INT-03 thresholds, not by changing schema types).

---

5) Schema Shape (Top-Level Objects)

A submission payload is an object with these sections:

- `routing` (required)
- `project` (required)
- `inputs` (optional)
- `existing` (optional; used when build_target=existing)
- `delivery` (required)
- `intent` (required)
- `brand` (optional; consumer-facing)
- `design` (optional; consumer-facing)
- `spec` (required)
- `data` (optional)
- `auth` (optional)
- `integrations` (optional)
- `nfr` (optional)
- `app_ui` (optional)
- `service` (optional)
- `infra` (optional)
- `sdk` (optional)
- `final` (required)

---

6) Field Definitions (Types + Base Constraints)

6.1 routing (required)

- `routing.skill_level`: enum `SkillLevel` (required)

- `routing.category`: enum `ProjectCategory` (required)
- `routing.type_preset`: enum `TypePreset` (required)
- `routing.build_target`: enum `BuildTarget` (required)
- `routing.audience_context`: enum `AudienceContext` (required)
- `routing.autofill`: boolean (required)

6.2 project (required)

- `project.project_name`: string (required; min/max length)
- `project.project_overview`: string (required; min/max length)
- `project.project_oneliner`: string (optional; max length)
- `project.core_experience_paragraph`: string (optional; max length)

6.3 inputs (optional)

- `inputs.zip_upload`: fileRef (optional; allowed filetypes rule)
- `inputs.reference_links[]`: array (optional)
    - item: `{ url: string, label?: string }`

6.4 existing (optional section)

- `existing.existing_repo_link`: url (optional)
- `existing.current_state_summary`: string (optional; min/max length)
- `existing.must_not_change[]`: array (optional)
- `existing.known_issues[]`: array (optional)
    - item: `{ title: string, description?: string }`

6.5 delivery (required)

- `delivery.desired_scope`: enum `DesiredScope` (required)
- `delivery.priority_bias`: enum `PriorityBias` (required)

6.6 intent (required)

- `intent.problem_statement`: string (required; min/max length)
- `intent.why_now`: string (optional)
- `intent.alternatives[]`: array (optional)
    - item: `{ name: string, link?: url, whats_missing?: string }`

- `intent.audience_description`: string (required)
- `intent.primary_user_label`: string (required)
- `intent.secondary_users[]`: array (optional)
- `intent.primary_goals[]`: array (required; min 1)
- `intent.success_definition`: string (required)
- `intent.success_metrics[]`: array (optional)
  - item: `{ metric: string, target?: string, timeframe?: string }`
- `intent.out_of_scope[]`: array (required; min 1)

6.7 brand (optional section)

- `brand.stands_for`: string (optional)
- `brand.change_created`: string (optional)
- `brand.core_values[]`: array (optional)
- `brand.decision_beliefs[]`: array (optional)
- `brand.refusal_guardrails`: string (optional)
- `brand.psychographics`: string (optional)
- `brand.user_cares_about`: string (optional)
- `brand.user_fears_avoids`: string (optional)
- `brand.aligned_communities[]`: array (optional)
- `brand.status_signal`: enum `StatusSignal` (optional)
- `brand.brand_promise`: string (optional)
- `brand.positioning_statement`: string (optional)
- `brand.unique_advantage`: string (optional)
- `brand.voice_adjectives[]`: array (optional)
- `brand.tone_boundaries`: string (optional)
- `brand.example_phrases[]`: array (optional)
- `brand.content_maturity`: enum `ContentMaturity` (optional)
- `brand.first_open_emotion`: string (optional)
- `brand.post_success_emotion`: string (optional)
- `brand.avoid_emotions`: string (optional)

6.8 design (optional section)

- `design.style_adjectives[]`: array (optional)
- `design.visual_preset`: enum `VisualPreset` (optional)
- `design.avoid_list[]`: array (optional)
- `design.logo_file`: fileRef (optional)

- `design.brand_colors[]`: array (optional)
  - item: `{ name?: string, hex: string }`
- `design.fonts[]`: array (optional)
  - item: `{ name: string, use_case?: enum FontUseCase }`
- `design.brand_references[]`: array (optional)
  - item: `{ url: string, notes?: string }`
- `design.ui_density`: enum `UIDensity` (optional)
- `design.nav_preference`: enum `NavPreference` (optional)
- `design.component_vibe[]`: array (optional)
- `design.motion_preference`: enum `MotionPreference` (optional)
- `design.a11y_expectations[]`: array (optional)
- `design.a11y_notes`: string (optional)
- `design.content_presentation`: enum `ContentPresentation` (optional)
- `design.showcase_screens[]`: array (optional)
  - item: `{ name: string, visual_priority?: enum ShowcasePriority, notes?: string, ref_url?: url, ref_file?: fileRef }`

6.9 spec (required)

- `spec.must_have_features[]`: array (required; min 1)
  - item: `{ name: string, description?: string }`
- `spec.nice_to_have_features[]`: array (optional)
- `spec.future_features[]`: array (optional)
- `spec.feature_priority_rank[]`: array (required; references must-haves)
- `spec.roles[]`: array (required; min 1)
  - item: `{ name: string, description?: string, primary_goal?: string }`
- `spec.role_permissions[]`: array (required)
  - item: `{ role_name: string, allowed_capabilities: array<string>, restricted_actions?: array<string>, approval_required_actions?: array<string> }`
- `spec.workflows[]`: array (required; min 3)
  - item: `{ name: string, actor_role: string, steps: array<string>, success_outcome: string, failure_states?: string, priority?: enum WorkflowPriority }`
- `spec.edge_workflows[]`: array (optional)
- `spec.business_rules_enabled`: boolean (optional)

- `spec.must_always_rules[]`: array (optional)
- `spec.must_never_rules[]`: array (optional)
- `spec.validation_rules[]`: array (optional)
  - item: `{ fields?: string, rule: string }`
- `spec.lifecycle_rules[]`: array (optional)

6.10 data (optional)

- `data.enabled`: boolean (optional)
- `data.objects[]`: array (optional)
  - item: `{ name: string, description?: string, required_fields?: array<object>, optional_fields?: array<object>, relationships?: array<object>, lifecycle_states?: array<string> }`

6.11 auth (optional)

- `auth.required`: boolean (optional)
- `auth.methods[]`: array (optional)
- `auth.account_lifecycle[]`: array (optional)
- `auth.two_fa_required`: boolean (optional)
- `auth.session_rules[]`: array (optional)
- `auth.authorization_model`: enum `AuthorizationModel` (optional)
- `auth.approval_flows_needed`: boolean (optional)
- `auth.approval_flows_description`: string (optional)

6.12 integrations (optional)

- `integrations.enabled`: boolean (optional)
- `integrations.items[]`: array (optional)
  - item: `{ service_name: string, purpose: string, data_in?: string, data_out?: string, triggers?: array<enum IntegrationTrigger>, secrets_handling?: string }`

6.13 nfr (optional)

- `nfr.performance_targets[]`: array (optional)
- `nfr.scale_assumptions`: object (optional)

- `nfr.reliability_expectation`: enum `ReliabilityLevel` (optional)
- `nfr.offline_required`: boolean (optional)
- `nfr.compliance_flags[]`: array (optional)

6.14 category-specific optional sections

- `app_ui.*` (optional)
- `service.*` (optional)
- `infra.*` (optional)
- `sdk.*` (optional)

6.15 final (required)

- `final.definition_of_done`: string (required)
- `final.must_pass_checks[]`: array (optional)
- `final.acceptance_criteria[]`: array (optional)
- `final.rejection_conditions`: string (optional)
- `final.confirm_priorities`: boolean (required; must be true)
- `final.confirm_out_of_scope`: boolean (required; must be true)
- `final.confirm_constraints`: boolean (required; must be true)

---

7) Enum Registry (Referenced)

INT-02 references enums defined in the schema's enum registry:

- SkillLevel, ProjectCategory, TypePreset, BuildTarget, AudienceContext
- DesiredScope, PriorityBias
- StatusSignal, ContentMaturity
- VisualPreset, FontUseCase, UIDensity, NavPreference, MotionPreference, ContentPresentation, ShowcasePriority
- WorkflowPriority
- AuthMethod, AccountLifecycle, SessionRule, AuthorizationModel
- IntegrationTrigger
- ReliabilityLevel, ComplianceFlag
  (Exact value lists are maintained in the Enum Registry for this schema version.)

---

8) Failure Modes

- schema drift from form (field paths mismatch)
- enums not aligned to UI choices
- constraints too strict or too loose for real submissions
- allowing undefined sections without control (fixed by INT-03)

---

9) Definition of Done (INT-02)

INT-02 is complete when:

- every INT-01 field has a matching schema field path
- all field types are explicit and stable
- base constraints (min/max, formats) are defined where needed
- enum references are complete for this schema version
- changes require `schema_version` bump and compatibility notes

INT-03

INT-03 — Intake Validation Rules (Dependencies + Thresholds)

(Hardened Draft — Full)

1) Purpose

Define all validation logic that goes beyond basic types:

- dependency rules ("if X then Y required")
- contextual validity (category→type preset legality)
- referential integrity rules (references must resolve)
- skill-level thresholds (Beginner/Intermediate/Expert depth rules)
- format checks that are conditional (file requiredness, URL requirements, etc.)

INT-03 is the **logic layer** that turns INT-02's schema into an enforceable gate.

---

2) Inputs

- INT-02 Intake Schema Spec (field paths + types)
- Routing snapshot (`routing.*`)
- Policy choices for warnings vs errors (if warnings exist)

---

3) Outputs

- A deterministic set of validation rules consumed by:
  - Intake Validator
  - Gate reports (rule_id + pointers)
- Validation result must reference rule IDs from this document.

---

4) Rule Format (locked)

Every rule must be written in this structure:

- `rule_id`
- `description`
- `when` (condition)

- `then` (requirements)
- `error_code`
- `pointer_paths[]` (what fields to point at)
- `severity` (error | warning) *(default: error)*
- `notes` *(optional)*

---

5) Rule Sets

5.1 Routing Validity Rules (category ↔ type preset)

INT3-ROUTING-01

- **when:** always
- **then:** `routing.type_preset` must be valid for `routing.category`
- **error_code:** `INVALID_ENUM_FOR_CONTEXT`
- **pointers:** `routing.category`, `routing.type_preset`

(Same pattern repeats for each category mapping; enforced through the preset map table.)

5.2 Build Target Rules (existing project requirements)

INT3-EXIST-01

- **when:** `routing.build_target == "existing"`
- **then require:**
  - `existing.existing_repo_link` present and valid URL
  - `existing.current_state_summary` present
- **error_code:** `DEPENDENCY_MISSING` / `INVALID_URL`
- **pointers:** `existing.existing_repo_link`, `existing.current_state_summary`

5.3 Audience Context Rules (consumer-facing minimums)

INT3-CONS-01

- **when:** `routing.audience_context == "consumer"`
- **then require:**
  - `brand.stands_for`
  - `brand.brand_promise`
  - `brand.voice_adjectives[]` length ≥ 1
- **error_code:** `DEPENDENCY_MISSING` / `MIN_ITEMS`
- **pointers:** `brand.stands_for`, `brand.brand_promise`, `brand.voice_adjectives`

INT3-CONS-02

- **when:** `routing.audience_context == "consumer"`
- **then require:** `design.style_adjectives[]` length ≥ 1
- **error_code:** `MIN_ITEMS`
- **pointers:** `design.style_adjectives`

5.4 Gate-Enabled Section Rules (data/auth/integrations)

INT3-DATA-01

- **when:** `data.enabled == true`
- **then require:** `data.objects[]` length ≥ 1
- **error_code:** `MIN_ITEMS`
- **pointers:** `data.objects`

INT3-AUTH-01

- **when:** `auth.required == true`
- **then require:** `auth.methods[]` length ≥ 1
- **error_code:** `MIN_ITEMS`
- **pointers:** `auth.methods`

INT3-AUTH-02

- **when:** `auth.approval_flows_needed == true`
- **then require:** `auth.approval_flows_description` present

- **error_code:** `DEPENDENCY_MISSING`
- **pointers:** `auth.approval_flows_description`

INT3-INTEG-01

- **when:** `integrations.enabled == true`
- **then require:** `integrations.items[]` length ≥ 1
- **error_code:** `MIN_ITEMS`
- **pointers:** `integrations.items`

5.5 Feature/Workflow/Role Integrity Rules (cross-references)

INT3-REF-01 (Role names unique)

- **when:** `spec.roles[]` present
- **then require:** all `spec.roles[].name` values unique
- **error_code:** `DUPLICATE_VALUE`
- **pointers:** `spec.roles`

INT3-REF-02 (Permissions reference valid roles)

- **when:** `spec.role_permissions[]` present
- **then require:** each `role_name` exists in `spec.roles[].name`
- **error_code:** `INVALID_REFERENCE`
- **pointers:** `spec.role_permissions`, `spec.roles`

INT3-REF-03 (Workflows reference valid roles)

- **when:** `spec.workflows[]` present
- **then require:** each `actor_role` exists in `spec.roles[].name`
- **error_code:** `INVALID_REFERENCE`
- **pointers:** `spec.workflows`, `spec.roles`

INT3-REF-04 (Feature priority rank references must-haves)

- **when:** `spec.feature_priority_rank[]` present
- **then require:** every entry exists in `spec.must_have_features[].name`

- **error_code:** `INVALID_REFERENCE`
- **pointers:** `spec.feature_priority_rank`, `spec.must_have_features`

5.6 Business Rules Toggle Integrity

INT3-RULES-01

- **when:** `spec.business_rules_enabled == true`
- **then require:** `spec.must_always_rules[]` length ≥ 1
- **error_code:** `MIN_ITEMS`
- **pointers:** `spec.must_always_rules`

5.7 Format Validations (conditional)

INT3-FMT-01 (Zip file type)

- **when:** `inputs.zip_upload` present
- **then require:** file type is `.zip`
- **error_code:** `INVALID_FILETYPE`
- **pointers:** `inputs.zip_upload`

INT3-FMT-02 (URL validity for reference links)

- **when:** `inputs.reference_links[]` present
- **then require:** each `url` is valid URL
- **error_code:** `INVALID_URL`
- **pointers:** `inputs.reference_links`

INT3-FMT-03 (Hex validity for brand colors)

- **when:** `design.brand_colors[]` present
- **then require:** each `hex` is valid hex color format
- **error_code:** `INVALID_FORMAT`
- **pointers:** `design.brand_colors`

---

6) Skill Level Threshold Rules (Depth Profile)

6.1 Core minimums (always)

INT3-SKILL-01 (Must-have features count)

- **when:** `routing.skill_level == beginner`
- **then:** `spec.must_have_features.length >= 1`
- **error_code:** `MIN_ITEMS`
- **pointers:** `spec.must_have_features`

INT3-SKILL-02

- **when:** `routing.skill_level == intermediate`
- **then:** `spec.must_have_features.length >= 3`
- **error_code:** `MIN_ITEMS`
- **pointers:** `spec.must_have_features`

INT3-SKILL-03

- **when:** `routing.skill_level == expert`
- **then:** `spec.must_have_features.length >= 5`
- **error_code:** `MIN_ITEMS`
- **pointers:** `spec.must_have_features`

INT3-SKILL-04 (Workflows count)

- **when:** beginner or intermediate
- **then:** `spec.workflows.length >= 3`
- **error_code:** `MIN_ITEMS`
- **pointers:** `spec.workflows`

INT3-SKILL-05 (Expert workflows alternative)

- **when:** expert
- **then:** either
    - `spec.workflows.length >= 5`

    - OR (`spec.workflows.length >= 3` AND `spec.edge_workflows.length >= 2`)
- **error_code:** `MIN_ITEMS`
- **pointers:** `spec.workflows`, `spec.edge_workflows`

INT3-SKILL-06 (Out of scope count)

- **when:** beginner
- **then:** `intent.out_of_scope.length >= 1`
- **error_code:** `MIN_ITEMS`
- **pointers:** `intent.out_of_scope`

INT3-SKILL-07

- **when:** intermediate
- **then:** `intent.out_of_scope.length >= 2`
- **error_code:** `MIN_ITEMS`
- **pointers:** `intent.out_of_scope`

INT3-SKILL-08

- **when:** expert
- **then:** `intent.out_of_scope.length >= 3`
- **error_code:** `MIN_ITEMS`
- **pointers:** `intent.out_of_scope`

---

7) Final Confirmation Rules (must be true)

INT3-FINAL-01

- **when:** always
- **then require:**
    - `final.confirm_priorities == true`
    - `final.confirm_out_of_scope == true`
    - `final.confirm_constraints == true`
- **error_code:** `REQUIRED`
- **pointers:** those three fields

---

8) Failure Modes

- rules contradict schema (wrong field paths)
- thresholds too strict for the selected skill profile
- missing rule coverage for key gates (consumer, existing, auth, data)
- non-deterministic or ambiguous conditions (must be avoided)

---

9) Definition of Done (INT-03)

INT-03 is complete when:

- every conditional requiredness in INT-01 is represented as a rule here
- every rule has rule_id, when/then, error_code, pointers
- skill-level thresholds cover features/workflows/out-of-scope minimums
- reference integrity rules cover roles, permissions, workflows, priority rank
- rules are deterministic (no vague conditions)

INT-04

INT-04 — Submission Record Format (Traceability)

(Hardened Draft — Full)

1) Purpose

Define the immutable **Intake Submission Record** so the system can preserve exactly what the user provided, with enough metadata to:

- reproduce validation behavior (form/schema versions)
- trace every downstream artifact back to the original input
- audit changes across re-submissions without ambiguity

This record is the **raw truth** of intake. It is never “normalized” or rewritten.

---

2) Inputs

- Raw submission payload (from INT-01 form)
- Upload references (zip/logo/etc.)
- Runtime metadata captured at submit time:
  - timestamp
  - submitter identity (if available)
  - source channel

---

3) Outputs

A single immutable record:

- `submission_id` (stable)
- raw payload (as provided)
- version stamps (form/schema)
- upload pointers
- routing snapshot (at submit time)

This record becomes the authoritative input for INT-05 validation.

---

4) Record Contract (Canonical Structure)

4.1 Required Metadata (always present)

- `submission_id` (string, unique)
- `created_at` (timestamp)
- `created_by` (string; user id/email/anonymous id)
- `source` (enum/string; e.g., web/app/api)
- `form_version` (string)
- `schema_version` (string)

4.2 Routing Snapshot (required)

- `routing_snapshot` (object)
  - `skill_level`
  - `category`
  - `type_preset`
  - `build_target`
  - `audience_context`
  - `autofill`

(This snapshot is required even if routing also exists inside raw payload. This prevents ambiguity if payload changes later.)

4.3 Raw Submission Payload (required)

- `raw_submission` (object)
  - The full submission object exactly as provided by the user.
  - No normalization, no enum coercion, no trimming, no reformatting.

4.4 Upload References (optional but canonicalized)

- `uploads[]` (array)
  - `upload_id` or `storage_ref`
  - `file_name`
  - `file_type`
  - `file_size`
  - `field_path` (where it came from, e.g. `inputs.zip_upload`, `design.logo_file`)
  - `uploaded_at`

4.5 Integrity Fields (recommended)

- `payload_hash` (hash of raw_submission + uploads list)

- `record_version` (version of the submission record format)

4.6 Status (optional, but if present must be controlled)

- `status` (enum/string): received | validated | rejected
- `status_updated_at`
- **Rule:** status can change; raw_submission cannot.

---

5) Invariants (must always be true)

INT4-INV-01 Immutability of raw payload

- `raw_submission` is immutable after record creation.
- Any correction requires a **new submission record** with a new `submission_id`.

INT4-INV-02 Version pinning

- `form_version` and `schema_version` must reflect what was in effect at submit time.
- Validator must use `schema_version` from the record.

INT4-INV-03 Upload traceability

- Every upload entry must include `field_path`.
- Upload references in raw submission must resolve to a recorded upload entry (or be invalid).

INT4-INV-04 Hash integrity (if used)

- `payload_hash` must match the stored payload and uploads references.
- Any mismatch is a corruption/alteration signal.

---

6) Failure Modes

- missing version stamps → cannot reproduce validation behavior
- raw payload mutated → breaks audit chain
- uploads not linked to field_path → cannot trace source
- hash mismatch → indicates record tampering or storage error

---

7) Definition of Done (INT-04)

INT-04 is complete when:

- all required metadata fields are defined
- immutability is explicit and non-negotiable
- version stamps are required and enforced
- upload reference structure is explicit
- integrity handling (hash policy) is specified clearly

INT-05

INT-05 — Validator Output Format (Errors/Warnings Contract)

(Hardened Draft — Full)

1) Purpose

Define the **single standard output contract** for intake validation so:

- the pipeline can deterministically branch (pass vs fail)
- UI can render field-specific issues consistently
- gates can reference exact rule IDs and pointers
- downstream stages never need to “interpret” validation results

INT-05 defines the format only. The rules live in INT-03.

---

2) Inputs

- `submission_id`
- `schema_version` (from INT-04)
- INT-02 schema (field paths/types)
- INT-03 rules (dependencies/thresholds/reference integrity)
- Raw submission payload (INT-04)

---

3) Outputs

A single **Validation Result** object:

- pass/fail decision
- errors (blocking)
- warnings (non-blocking, optional)
- versions used

This result must be stored and linked to `submission_id`.

---

4) Output Contract (Canonical Structure)

4.1 Required top-level fields

- `submission_id` (string)
- `validated_at` (timestamp)
- `is_valid` (boolean)
- `schema_version_used` (string)
- `form_version_used` (string)
- `ruleset_version_used` (string) *(INT-03 version or hash; required for determinism)*
- `errors[]` (array of ValidationIssue; empty if valid)
- `warnings[]` (array of ValidationIssue; may be empty or omitted if warnings not supported)
- `summary` (object)
  - `error_count`
  - `warning_count`
  - `blocking_rule_ids[]` *(unique list)*

4.2 ValidationIssue structure

Each issue must include:

- `issue_id` (string, unique)
- `severity` (enum: error | warning)
- `error_code` (enum/string; from error code catalog)
- `rule_id` (string; from INT-03)
- `field_path` (string; schema path)
- `message` (string; user-readable)
- `meta` (object; optional but recommended)
  - examples:
    - `{ "min_items": 3, "actual": 2 }`
    - `{ "allowed_values": ["x","y"], "actual": "z" }`
    - `{ "expected_type": "url" }`

---

5) Error Code Catalog (validation-level)

Validator output may only use these codes (locked list):

- `REQUIRED`
- `DEPENDENCY_MISSING`
- `MIN_ITEMS`

- `INVALID_ENUM`
- `INVALID_ENUM_FOR_CONTEXT`
- `INVALID_FORMAT`
- `INVALID_URL`
- `INVALID_FILETYPE`
- `INVALID_REFERENCE`
- `DUPLICATE_VALUE`
- `WARNING_INCOMPLETE` *(only if warnings are enabled)*

Rule: If a new code is needed, it must be added via change control.

---

6) Determinism Rules (must always be true)

INT5-DET-01 Stable ordering

- Issues should be ordered deterministically:
    1. routing errors
    2. requiredness/dependencies
    3. type/format
    4. reference integrity
    5. thresholds

INT5-DET-02 Stable pointers

- `field_path` must always point to a real schema path.
- If the issue is about a relationship (e.g., invalid reference), `field_path` points to the referencing field.

INT5-DET-03 Rule ID required

- Every issue must include `rule_id` linking it to INT-03.
- No “freeform” validator errors without a rule anchor.

---

7) Pass/Fail Semantics (locked)

- `is_valid == true` **only if** `errors.length == 0`
- warnings never flip validity (unless you choose a strict policy; default is warnings are non-blocking)

---

8) Failure Modes

- missing rule_id makes remediation impossible
- inconsistent field_path breaks UI highlighting
- non-deterministic ordering makes debugging difficult
- validator uses undefined error codes (breaks automation)

---

9) Definition of Done (INT-05)

INT-05 is complete when:

- validation result format is fully specified
- issue structure includes rule_id + field_path + error_code
- error code catalog is locked and aligned with INT-03
- determinism rules for ordering and pointers are explicit

Canonical Data Model

Canonical Data Model

- CAN-01 Canonical Spec Model (entities + relationships + IDs)
- CAN-02 ID & Reference Rules (no duplicate truth, referential integrity)
- CAN-03 Unknowns Model (how unknowns are represented and handled)

CAN-01

CAN-01 — Canonical Spec Model (Entities + Relationships + IDs)

(Hardened Draft — Full)

1) Purpose

Define the **Canonical Spec** as the single authoritative structured truth model for a project. This model is what every downstream artifact references to prevent drift, contradictions, and missing foundations.

CAN-01 defines:

- what entities exist in the canonical spec
- required fields for each entity type
- allowed relationships between entities
- the required indexes/maps that make the model navigable

ID semantics and enforcement rules are defined in CAN-02.

Unknowns object model is defined in CAN-03.

---

2) Inputs

- Normalized Input Record (A2.1)
- Resolved Standards Snapshot (A2.2)
- Intake routing snapshot (skill/category/type/build_target/audience)
- (Optional) vocabulary/definitions mappings

---

3) Outputs

A single **Canonical Spec** artifact:

- `spec_id`
- `submission_id`
- `standards_snapshot_id`
- `spec_version`
- `spec` object (entities)

- `index` object (lookup tables + cross-maps)
- `unknowns` object (list of unknown objects; canonical)

---

4) Core Invariants (must always be true)

1. **Single source of truth:** Canonical Spec is the only place where project truth is defined.
2. **Stable IDs:** every entity has a stable ID (rules in CAN-02).
3. **Referential integrity:** all references resolve (rules in CAN-02).
4. **Explicit unknowns:** unknowns are explicit entities (CAN-03), not missing fields.
5. **Standards binding:** constraints from standards snapshot are represented as constraints in the spec, not re-decided later.

---

5) Canonical Spec Top-Level Shape

```
canonical_spec = {
  meta: {...},
  routing: {...},
  constraints: {...},
  entities: {...},
  rules: {...},
  unknowns: {...},
  index: {...}
}
```

5.1 meta (required)

- `spec_id` (string)
- `submission_id` (string)
- `schema_version_used` (string)
- `standards_snapshot_id` (string)
- `standards_version_used` (string)
- `spec_version` (string)
- `created_at` (timestamp)

5.2 routing snapshot (required)

- `skill_level`
- `category`
- `type_preset`
- `build_target`
- `audience_context`

5.3 constraints (required)

Constraints are facts the system must obey, derived from:

- Resolved Standards Snapshot
- Intake constraints (scope bias, consumer design direction, etc.)

Minimum fields:

- `stack_constraints` (object or list)
- `security_constraints` (object or list)
- `quality_constraints` (object or list)
- `design_constraints` (object or list; if consumer)
- `fixed_vs_configurable` (map; reference standards snapshot)

---

6) Entity Groups (canonical “truth entities”)

6.1 Role (required if any permissions/workflows exist)

Role entity

- `role_id`
- `name`
- `description` (optional)
- `primary_goal` (optional)

6.2 Capability / Feature (required)

Feature entity

- `feature_id`
- `name`
- `description` (optional)
- `priority_tier` (must / nice / future)
- `priority_rank` (optional numeric; derived from rank list)
- `scope_boundaries`:
  - `in_scope[]` (optional)
  - `out_of_scope[]` (optional)
- `workflow_refs[]` (optional; links to workflows)

6.3 Workflow (required)

Workflow entity

- `workflow_id`
- `name`
- `actor_role_ref` (role_id)
- `steps[]` (ordered)
- `success_outcome`
- `failure_states` (optional)
- `priority` (optional)

6.4 Permission / Capability Grant (required)

Permissions are represented as a relationship entity so they're referenceable:

- `perm_id`
- `role_ref` (role_id)
- `allowed_capabilities[]` (strings or capability IDs; choose one and lock in CAN-02)
- `restricted_actions[]` (optional)
- `approval_required_actions[]` (optional)

(If you later formalize capabilities as IDs, this becomes references.)

6.5 Screen (conditional: if UI app)

Screen entity

- `screen_id`
- `name`

- `purpose`
- `roles_allowed[]` (role_id refs)
- `data_shown` (optional)
- `actions[]` (optional)
- `state_notes` (optional: empty/loading/error states)

6.6 UI Component (optional)

Only for “promoted” reusable components:

- `component_id`
- `name`
- `purpose`
- `variants[]` (optional)
- `usage_rules_ref` (template or standards pointer; optional)

6.7 Data Object / Entity (conditional: if data.enabled)

Data entity

- `data_object_id`
- `name`
- `description` (optional)
- `fields_required[]` (list of field objects)
- `fields_optional[]` (list of field objects)
- `relationships[]` (relationship objects)
- `lifecycle_states[]` (optional)
- `sensitive_flags[]` (optional)

Field object (minimal):

- `name`
- `type`
- `notes` (optional)

Relationship object (minimal):

- `relation_type`
- `target_data_object_ref` (data_object_id)
- `notes` (optional)

6.8 Operation (required for service/backend builds; optional for pure UI)

Operations represent system actions (often API endpoints or internal operations):

- `operation_id`
- `name`
- `purpose`
- `inputs_summary` (optional)
- `outputs_summary` (optional)
- `auth_required` (optional boolean)
- `error_behavior` (optional)
- `feature_refs[]` (feature_id refs)
- `workflow_refs[]` (workflow_id refs)
- `data_refs[]` (data_object_id refs)

6.9 Integration (conditional: if integrations.enabled)

- `integration_id`
- `service_name`
- `purpose`
- `data_in` (optional)
- `data_out` (optional)
- `triggers[]` (optional)
- `secrets_handling` (optional)

6.10 Non-Functional Requirement (optional)

Represent NFRs as structured constraints:

- `nfr_id`
- `category` (performance/reliability/compliance/etc.)
- `statement`
- `targets` (optional structured)
- `applies_to_refs[]` (optional: feature/screen/operation refs)

---

7) Rules Group (canonical rules and constraints)

7.1 Business rules (optional)

- `must_always[]`
- `must_never[]`
- `validation_rules[]`
- `lifecycle_rules[]`

7.2 Definition of Done (required)

- `definition_of_done` (string)
- `must_pass_checks[]` (optional)
- `acceptance_criteria[]` (optional)
- `rejection_conditions` (optional)

---

8) Unknowns Group (present even if empty)

- `unknowns[]` (list of unknown objects; defined in CAN-03)
- `unknown_index` (map by unknown_id)

---

9) Index Group (required)

The canonical spec must include stable indexes to prevent re-parsing.

Minimum indexes:

- `roles_by_id`, `roles_by_name`
- `features_by_id`, `features_by_name`

- `workflows_by_id`, `workflows_by_name`
- `screens_by_id` (if any)
- `data_objects_by_id` (if any)
- `operations_by_id` (if any)
- `integrations_by_id` (if any)
- `cross_maps`:
    - `workflow_to_features`
    - `feature_to_workflows`
    - `feature_to_operations`
    - `screen_to_workflows` (if UI)
    - `role_to_workflows`
    - `role_to_permissions`

---

10) Failure Modes

- missing entity IDs (breaks referenceability)
- duplicated truth (same feature described differently in multiple places)
- missing indexes (forces re-parsing, increases drift)
- references that don't resolve (broken integrity)
- unknowns handled as empty fields rather than explicit objects

---

11) Definition of Done (CAN-01)

CAN-01 is complete when:

- canonical spec top-level shape is locked
- entity groups are enumerated with required fields
- required index tables and cross-maps are defined
- constraints and rules groups are represented explicitly
- CAN-01 cleanly defers ID enforcement to CAN-02 and unknown policy to CAN-03

CAN-02

CAN-02 — ID & Reference Rules (No Duplicate Truth, Referential Integrity)

(Hardened Draft — Full)

1) Purpose

Define the enforceable rules that make the Canonical Spec trustworthy:

- how IDs are formed and assigned
- what must be unique
- how references are represented and validated
- how “no duplicate truth” is enforced across entities and templates
- what integrity checks must pass before downstream steps

CAN-02 is the **integrity law** for the canonical model.

---

2) Inputs

- Canonical Spec (CAN-01 structure)
- (Optional) vocabulary/definitions library for name normalization rules
- Governance rules for versioning/migrations (when model changes)

---

3) Outputs

- A locked set of integrity rules used by:
    - Spec Gate (SYS-07)
    - Planner/Decomposer
    - Template Filler
    - Packager (manifest integrity)

---

4) ID Rules (locked)

4.1 ID Requiredness

Every entity instance must have an ID field:

- role_id, feature_id, workflow_id, perm_id
- screen_id (if present), component_id (if present)
- data_object_id (if present), operation_id (if present)
- integration_id (if present), nfr_id (if present)
- unknown_id (defined in CAN-03)

Rule: no entity is valid without its ID.

4.2 ID Format

IDs must be:

- globally unique within the spec
- stable across regeneration unless the underlying entity is truly new

Canonical format (recommended lock):

- `rol_<slug>`
- `feat_<slug>`
- `wf_<slug>`
- `perm_<slug>`
- `scr_<slug>`
- `cmp_<slug>`
- `dat_<slug>`
- `op_<slug>`
- `int_<slug>`
- `nfr_<slug>`
- `unk_<slug>`

Slug rules:

- lowercase
- alphanumeric + underscore
- deterministic from normalized name (with collision handling)

4.3 Collision Handling (deterministic)

If two entities normalize to the same slug:

- append deterministic suffix: `_2`, `_3`, etc. based on stable ordering rules.

Stable ordering rule: order by original appearance in normalized input record.

4.4 ID Stability Rules

- If an entity name changes only by whitespace/case normalization, ID must remain unchanged.
- If entity meaning changes (feature replaced), a new ID must be created and the old one deprecated via governance notes (if applicable).

---

5) Uniqueness Rules (locked)

5.1 Name uniqueness requirements

These names must be unique **within their entity type**:

- role.name
- feature.name (within must/nice/future tiers can still collide—disallow collisions globally to simplify)
- workflow.name
- screen.name (if present)
- data_object.name (if present)
- operation.name (if present)

Violation results in:

- error_code: `DUPLICATE_VALUE`
- rule_id: `CAN02-UNIQ-*`

---

6) Reference Model (how entities refer to each other)

6.1 Reference type

Rule: References must be by ID, not by free-text names, whenever the target entity has an ID.

Canonical usage:

- workflow.actor_role_ref must be role_id
- screen.roles_allowed[] must be role_id
- permission.role_ref must be role_id
- operation.feature_refs[] must be feature_id
- operation.workflow_refs[] must be workflow_id
- operation.data_refs[] must be data_object_id
- data.relationships.target_data_object_ref must be data_object_id
- nfr.applies_to_refs[] must be entity IDs (typed ref; see 6.3)

6.2 Allowed reference edges (locked)

Allowed relationships:

- Role → Workflows (as actor)
- Role → Permissions (grant)
- Workflow → Features (coverage)
- Feature → Workflows (coverage)
- Feature → Operations
- Workflow → Screens (optional mapping)
- Screen → Workflows (optional mapping)
- DataObject → DataObject (relationships)
- Operation → DataObject(s)
- Operation → Feature(s)
- Operation → Workflow(s)
- Integration → Operation(s) *(optional if formalized)*
- NFR → {Feature|Workflow|Screen|Operation|DataObject} *(typed ref)*

No other edges are allowed unless the model is extended via versioned migration.

6.3 Typed reference object (for mixed-target refs)

For fields like `nfr.applies_to_refs[]`, use a typed ref structure:

- `{ ref_type: "feature"|"workflow"|"screen"|"operation"|"data_object", ref_id: "<id>" }`

This prevents ambiguity.

---

7) Referential Integrity Checks (must pass)

7.1 Resolve checks

Every reference must resolve to an existing entity ID in the spec.

Violations produce:

- error_code: `INVALID_REFERENCE`
- pointer: referencing field path

7.2 Cross-map consistency checks

If cross-maps exist, they must agree with entity refs. Example:

- if `operation.feature_refs` includes feat_123 then `feature_to_operations[feat_123]` must include op_456 (and vice versa)

Violations produce:

- error_code: `INVALID_REFERENCE` (or `INVALID_FORMAT` if you want distinct)
- rule_id: `CAN02-XMAP-*`

---

8) No Duplicate Truth (strict rules)

8.1 Canonical Spec is the only truth store

- Product truth can only be defined in entities.
- Templates, plans, and docs may summarize but must reference IDs and not redefine.

8.2 Duplicate truth detection rules

Treat as violation if:

- same feature appears as two different feature entities with overlapping meaning
- same workflow is described under different IDs with same steps and actor
- permissions are repeated inconsistently (two perm objects for same role with conflicting capability grants)

Enforcement:

- Spec Gate fails with `CAN02-DUPTRUTH-*`

---

9) Downstream Requirements (why these rules exist)

Downstream artifacts depend on CAN-02 guarantees:

- Work Breakdown must map to IDs
- Acceptance Map must map to IDs
- Template filler must reference IDs
- Pack manifest must index IDs

If CAN-02 fails, the pipeline must stop.

---

10) Failure Modes

- name-based references cause drift when names change
- missing typed refs causes ambiguity
- collisions handled inconsistently breaks determinism
- duplicate truth leads to contradictory templates

11) Definition of Done (CAN-02)

CAN-02 is complete when:

- ID format and collision rules are explicit and deterministic
- uniqueness rules are defined per entity type
- reference model is ID-based and typed where needed
- allowed edges are explicitly listed
- referential integrity checks and cross-map consistency checks are defined
- no-duplicate-truth enforcement is explicit and gateable

CAN-03

CAN-03 — Unknowns Model (Representation + Handling)

(Hardened Draft — Full)

1) Purpose

Define how Axion represents missing/uncertain information as explicit **Unknown objects** so the system never:

- guesses requirements
- hides gaps as empty fields
- relies on chat memory
- silently proceeds with ambiguous decisions

Unknowns are first-class entities that can be:

- non-blocking (allowed to proceed)
- blocking (must be resolved before downstream steps)

---

2) Inputs

- Normalized Input Record (fields missing, ambiguous, contradictory)
- Validation warnings (if supported)
- Canonical Spec (where unknowns may attach)
- Standards Snapshot (what is fixed vs configurable; affects blocking status)

---

3) Outputs

A canonical `unknowns` collection inside the Canonical Spec:

- `unknowns[]` (list of Unknown objects)
- `unknown_index` (map: unknown_id → object)
- references to unknowns from affected entities/sections (as needed)

---

4) Unknown Object Contract (Locked Structure)

Each Unknown must have:

4.1 Identity

- `unknown_id` (string; follows CAN-02 ID conventions, prefix `unk_`)
- `created_at` (timestamp)
- `status` (enum): `open` | `resolved`
- `resolution_ref` (optional; pointer to decision/override/input update)

4.2 Classification

- `unknown_type` (enum):
  - `missing_requirement`
  - `ambiguous_requirement`
  - `contradiction`
  - `unresolved_decision`
  - `missing_reference`
  - `missing_constraint`
- `severity` (enum): `low` | `medium` | `high`
- `blocking` (boolean)

4.3 Scope Attachment (what it affects)

At least one of:

- `applies_to` (typed ref): `{ ref_type, ref_id }` *(CAN-02 typed ref)*
- `field_path` (schema path) *(for intake-derived unknowns)*
- `template_ref` (template_id + section_id) *(for template fill unknowns)*

4.4 Description (human-readable, non-ambiguous)

- `summary` (one sentence)
- `detail` (longer explanation)
- `impact` (what breaks or becomes risky if unresolved)

4.5 Resolution Requirements

- `what_is_needed_to_resolve` (explicit question(s) or data required)
- `allowed_resolution_methods[]` (enum list):
  - `user_input_update`
  - `standards_override`
  - `explicit_decision_record`
  - `de_scope_feature`
  - `mark_not_applicable`
- `default_resolution_policy` (enum):
  - `must_resolve_before_spec_final`
  - `must_resolve_before_planning`
  - `must_resolve_before_template_fill`
  - `may_defer_to_execution`
  - `may_remain_unknown` *(rare; only for non-core areas)*

---

5) Unknown Creation Rules (when to create unknowns)

CAN03-NEW-01 Missing required-but-unknown input

If a required concept exists but lacks details (e.g., “auth required” but no methods), create an unknown if it’s allowed to proceed; otherwise gate fail at INT-03.

CAN03-NEW-02 Ambiguity

If user input can be interpreted in multiple incompatible ways, create an unknown of type `ambiguous_requirement`.

CAN03-NEW-03 Contradiction

If two statements conflict (e.g., out-of-scope includes “no auth” but auth.required true), create an unknown of type `contradiction` and mark blocking by default.

CAN03-NEW-04 Missing reference

If a reference cannot be resolved during spec build (should usually be prevented by INT-03), create unknown `missing_reference` and block.

---

6) Blocking Policy (strict)

6.1 Default blocking rules

Unknowns are **blocking** if they affect any of these core decision areas:

- roles/permissions model
- auth requirements
- core workflows
- must-have features list
- data model existence when data.enabled
- API contract existence when service/API build
- standards snapshot constraints that must be obeyed

Unknowns may be **non-blocking** when they affect:

- cosmetic design preferences
- optional NFR details (unless required by preset)
- optional integrations (when integrations disabled)
- optional copy tone examples

6.2 Skill-level influence (allowed, but bounded)

Skill level can affect tolerance:

- Beginner: more unknowns allowed, but not in core areas
- Intermediate: fewer unknowns allowed
- Expert: unknowns should be rare; core unknowns should trigger gates

Rule: skill level may reduce required detail, but it does not allow unknowns in core decision areas.

---

7) Unknown Storage & Referencing Rules

7.1 Centralized list is required

All unknowns must exist in `canonical_spec.unknowns[]`. No unknowns stored only inside templates.

7.2 References to unknowns

Entities/sections may reference unknowns by `unknown_id` in a simple list:

- `entity.unknown_refs[]: ["unk_..."]`

7.3 Resolution must be traceable

When resolved:

- set `status = resolved`
- set `resolution_ref` to:
  - updated submission_id, OR
  - decision/override record ID, OR
  - standards snapshot override record ID

---

8) Gates Related to Unknowns

Unknowns interact with gates as follows:

- **Spec Gate** fails if any blocking unknown exists at “spec finalization.”
- **Planning Gate** fails if blocking unknowns exist that affect unit decomposition.
- **Template Gate** fails if a template’s required fields would be UNKNOWN in a core area without allowed policy.

---

9) Failure Modes prevented

- silent guessing to “fill in blanks”
- agent relying on chat memory for missing decisions
- templates embedding unknowns without traceability
- contradictions ignored until late in build

---

10) Definition of Done (CAN-03)

CAN-03 is complete when:

- Unknown object contract is fully specified
- creation rules cover missing/ambiguous/contradictory/reference cases
- blocking policy is strict and tied to core decision areas
- resolution method and traceability requirements are explicit
- unknown interactions with gates are specified

Standards System

Standards System

- STD-01 Standards Library Structure (what standards exist)
- STD-02 Standards Resolution Rules (defaults → overrides → snapshot)
- STD-03 Standards Snapshot Format (what gets emitted into kits)

STD-01

STD-01 — Standards Library Structure (What Standards Exist)

(Hardened Draft — Full)

1) Purpose

Define the **Standards Library** as a versioned, reusable store of rules that Axion can apply deterministically across projects. This library exists to:

- prevent re-deciding fundamentals each build
- enforce senior-level consistency
- make constraints reproducible via version pinning

STD-01 defines **what standards exist** and how they are organized. Resolution logic is STD-02.

---

2) Inputs

- System guarantees and boundaries (SYS-01..SYS-10)
- Template type coverage requirements (Template Library needs standards inserts)
- Supported project categories and routing axes (INT/CAN outputs)

---

3) Outputs

A Standards Library that provides:

- a **standards index/registry** (machine-readable)
- **versioned standards packs** (by category and domain)
- a fixed set of **standard categories** (non-redundant)

---

4) Rules / Invariants (must always be true)

1. **Versioned:** every standards pack has a version; the library has a library version.
2. **Deterministic selection:** standards must be selectable by routing + gates (no ambiguity).
3. **Fixed vs configurable:** every rule declares whether it is fixed or configurable.
4. **Non-overlap:** each rule belongs to exactly one standards category.
5. **Snapshot-able:** every rule must be representable in STD-03 snapshot format without interpretation.
6. **No hidden logic:** standards must be declarative, not “do what seems best.”

---

5) Standards Categories (Locked Set)

STD-CAT-01 Engineering Standards

Covers:

- language/framework/tooling constraints
- repo structure conventions
- coding conventions (naming, error handling discipline)
- dependency policy (allowed/disallowed)

STD-CAT-02 Design Standards

Covers:

- typography/layout rules

- component styling conventions
- spacing/radius/elevation guidance
- accessibility baseline (design-side constraints)
- consumer-facing brand application rules (if applicable)

STD-CAT-03 Security Standards

Covers:

- auth/session baseline requirements
- authorization enforcement expectations
- sensitive data handling/logging constraints
- abuse controls baseline when public-facing

STD-CAT-04 Quality Standards

Covers:

- required verification checks (build/lint/tests)
- minimum coverage expectations (if used)
- acceptance/proof discipline
- performance/reliability checks when triggered

STD-CAT-05 Operations Standards (conditional)

Covers:

- environment expectations (dev/stage/prod)
- deployment constraints
- secrets/config management expectations

- observability baseline (logs/metrics/alerts)
- backup/recovery baseline when required

STD-CAT-06 Interface/Contract Standards (conditional)

Covers:

- API error model conventions
- versioning/compatibility conventions
- event/webhook conventions
- contract naming discipline

STD-CAT-07 Analytics/Telemetry Standards (conditional)

Covers:

- event naming conventions
- required event metadata
- metric definition discipline
- privacy constraints for telemetry

---

6) Standards Pack Structure (How rules are stored)

A **Standards Pack** is a versioned unit that applies under explicit conditions.

6.1 Standards Pack Metadata (required)

- `pack_id`

- `pack_version`
- `category` (one of STD-CAT-01..07)
- `applies_when`
    - routing constraints (category/type_preset/audience/build_target)
    - gates (data enabled, auth required, integrations enabled, etc.)
- `priority` (for tie-breaking during merge; deterministic)
- `fixed_rules[]`
- `configurable_rules[]`

6.2 Rule Structure (required)

Each rule entry must include:

- `rule_id` (stable)
- `name`
- `description`
- `rule_type` (enum: requirement | constraint | default | prohibition)
- `value` (structured; must be snapshot-able)
- `fixed` (boolean)
- `source_pack` (pack_id@version)

---

7) Standards Index / Registry (Machine-readable)

The library must include an index listing:

- all packs and versions
- their applies_when filters
- categories covered
- their rule IDs

Minimum index fields:

- `standards_library_version`
- `packs[]: { pack_id, pack_version, category, applies_when, priority, rule_count }`

---

8) Failure Modes

- rules overlap categories (conflicting truths)
- rules not versioned (non-reproducible outputs)
- “configurable” not declared (overrides become unsafe)
- packs apply ambiguously (non-deterministic resolution)

---

9) Definition of Done (STD-01)

STD-01 is complete when:

- standards categories are locked and non-overlapping
- standards pack structure is defined (metadata + rule entries)
- standards index requirements are defined
- fixed vs configurable is enforceable at rule level
- all rules are declarative and snapshot-able

STD-02

STD-02 — Standards Resolution Rules (Defaults → Overrides → Snapshot)

(Hardened Draft — Full)

1) Purpose

Define the deterministic algorithm that resolves:

- which standards packs apply to a project
- which rules win when multiple packs provide rules
- how overrides are applied or blocked
- how the final ruleset becomes a **Resolved Standards Snapshot** (STD-03)

STD-02 is the “resolver law.” No discretionary behavior is allowed.

---

2) Inputs

- Normalized Input Record (routing + flags + constraints)
- Standards Library Index (packs, applies_when, priorities)
- Standards Packs (rule entries)
- User/project overrides (only for configurable rules)
- System fixed contracts (cannot be overridden)

---

3) Outputs

- A deterministic **Resolved Ruleset**:
    - final merged rule set

    - list of packs selected
    - explicit merge decisions (when conflicts exist)
    - overrides applied/blocked
    - version stamps
- This becomes the content of STD-03 snapshot.

---

4) Resolver Invariants (must always be true)

1. **Deterministic:** same inputs + same library version ⇒ same resolved snapshot.
2. **No silent conflict resolution:** conflicts must be resolved by explicit precedence rules.
3. **Fixed rules cannot be overridden.**
4. **Every resolved rule records its origin pack + version.**
5. **Blocked overrides are recorded, not discarded.**

---

5) Resolution Pipeline (Locked Steps)

Step 1 — Compute Project Context

Build a resolver context from normalized input:

- routing: category/type_preset/audience/build_target/skill_level
- gates: data.enabled/auth.required/integrations.enabled
- compliance flags (if any)
- delivery preferences (scope, bias)

- any user-stated constraints that are allowed to influence standards

Output: `resolver_context`

Step 2 — Select Applicable Packs

From the Standards Index, select packs where `applies_when` matches resolver_context.

Rules:

- Matching is strict (boolean logic).
- If a pack's applies_when is ambiguous or incomplete, it is invalid.

Output: `selected_packs[]`

Step 3 — Order Packs Deterministically

Sort selected packs by:

1. **specificity score** (more specific applies_when wins)
2. **priority** (declared in pack metadata)
3. **pack_id** (lexical tie-breaker)
4. **pack_version** (highest version wins only if identical pack_id and both selected by policy)

Output: `ordered_packs[]`

Step 4 — Merge Rules

Initialize empty `resolved_rules`.

For each pack in `ordered_packs`:

- For each rule in pack:
    - If rule_id not present → add it
    - If rule_id already present:
        - if the **value is identical** → keep existing, record both sources
        - if the **value differs** → resolve conflict using precedence below

Conflict Precedence (Locked)

1. **System fixed contract rules** (always highest; cannot be overwritten)
2. **More specific pack** wins (higher specificity score)
3. **Higher priority** wins
4. **If still tied** → **conflict must be flagged as unresolved** and resolver fails

Output: merged `resolved_rules` + `conflict_log`

Step 5 — Apply Overrides (Configurable Only)

For each override request:

- locate target rule_id or field_path in resolved_rules
- check `fixed == false`
- if fixed: block override; record blocked override
- if configurable: apply override; record applied override

Override record must include:

- field_path/rule_id
- before/after

- source (user/admin/system)
- reason (if provided)
- status (applied/blocked)
- timestamp

Output: `overrides_applied[]`, `overrides_blocked[]`

Step 6 — Validate Resolved Ruleset

Run resolver-level checks:

- no unresolved conflicts remain
- all required baseline rules exist for the project context
    - e.g., if auth.required then security baseline rules must exist
- every rule has origin metadata

If any fail → resolver fails (hard stop).

Step 7 — Emit Snapshot Inputs

Prepare the data that STD-03 will store:

- selected packs + versions
- resolved rules
- overrides applied/blocked
- fixed vs configurable map
- resolver context

---

6) Specificity Scoring (to make precedence enforceable)

Specificity score increases with each explicit selector in `applies_when`, for example:

- category specified
- type_preset specified
- audience_context specified
- build_target specified
- gate flags specified (auth/data/integrations)
- compliance flags specified

Rule: specificity scoring must be numeric and consistent across runs.

---

7) Failure Modes

- two packs conflict with no deterministic winner → resolver must fail
- override attempts fixed rule → must be blocked and recorded
- missing baseline rules for context → fail (no partial standards)
- ambiguous applies_when → invalid pack

---

8) Definition of Done (STD-02)

STD-02 is complete when:

- resolver steps are explicit and deterministic
- precedence rules are locked and cover ties/conflicts
- override application rules are strict and auditable
- failure conditions are explicit (fail vs proceed is unambiguous)
- output data required by STD-03 is fully specified

STD-03

STD-03 — Standards Snapshot Format (Emitted Into Kits)

(Hardened Draft — Full)

1) Purpose

Define the **Resolved Standards Snapshot** artifact that is emitted into every Agent Kit. This snapshot is the project-specific, version-pinned ruleset that all downstream artifacts must follow.

The snapshot exists so:

- the build is deterministic
- the external agent can follow constraints without guessing
- audits can trace exactly which standards were applied and why

---

2) Inputs

- STD-02 resolver outputs:
  - resolver_context
  - selected packs (ordered)
  - merged resolved rules
  - applied/blocked overrides
  - conflict log (must be empty if success)
- Version stamps (system/template/standards/schema)

---

3) Outputs

A single snapshot file in the kit (canonical location under core artifacts):

- `resolved_standards_snapshot.json` *(or .md + embedded JSON, but must be machine-readable)*

---

4) Snapshot Contract (Canonical Structure)

4.1 Snapshot Metadata (required)

- `resolved_standards_id` (string)

- `submission_id` (string)
- `spec_id` (string, if already assigned; otherwise optional until spec exists)
- `created_at` (timestamp)

4.2 Version Stamps (required)

- `system_version`
- `schema_version_used`
- `standards_library_version_used`
- `template_library_version_used` *(if available at snapshot time; otherwise fill later and update stamp chain)*
- `resolver_ruleset_version_used` *(STD-02 version/hash)*

4.3 Resolver Context (required)

- `resolver_context` (object)
  - routing snapshot:
    - skill_level/category/type_preset/build_target/audience_context
  - gate flags:
    - data.enabled/auth.required/integrations.enabled
  - compliance flags (if any)
  - delivery preferences (scope/priority bias)

4.4 Selected Packs (required)

- `selected_packs[]` (ordered list)
  - each item:
    - `pack_id`
    - `pack_version`
    - `category`
    - `applies_when` (stored for audit)
    - `specificity_score`
    - `priority`

4.5 Resolved Ruleset (required)

- `rules[]`
  - each rule:
    - `rule_id`
    - `category`
    - `name`
    - `description`

        - `rule_type` (requirement | constraint | default | prohibition)
        - `value` (structured)
        - `fixed` (boolean)
        - `sources[]` (list of pack_id@version that contributed)
        - `resolved_by` (enum): exact_match | precedence | override
        - `effective_scope` (optional: which parts of system it governs)

4.6 Fixed vs Configurable Map (required)

- `fixed_vs_configurable` (map)
    - key: `rule_id`
    - value: `fixed` | `configurable`

4.7 Overrides (required audit section)

- `overrides_applied[]`
- `overrides_blocked[]`

Each override record includes:

- `override_id`
- `rule_id` *(or field_path if you allow field-path overrides; choose one and lock)*
- `before`
- `after`
- `source` (user/admin/system)
- `reason` (optional)
- `status` (applied|blocked)
- `timestamp`

4.8 Conflict Log (required)

- `conflicts[]`
    - must be empty for successful snapshots
    - if not empty → snapshot is invalid and resolver should have failed

4.9 Integrity (recommended)

- `snapshot_hash` (hash of the snapshot content)
- `signature` (optional)

---

5) Snapshot Invariants (must always be true)

1. **Self-contained:** downstream steps must not need to consult the live standards library to interpret rules.
2. **Version-pinned:** every pack and library version is recorded.
3. **Conflict-free:** conflicts list must be empty for a valid snapshot.
4. **Override-auditable:** applied and blocked overrides are recorded.
5. **Fixed rules immutable:** fixed rules appear as fixed in the map and are never overridden.

---

6) Failure Modes

- snapshot missing versions → non-reproducible builds
- rules missing sources → no audit trail
- overrides applied without before/after → silent drift
- conflicts present but snapshot marked valid → breaks determinism

---

7) Definition of Done (STD-03)

STD-03 is complete when:

- snapshot structure is fully specified and machine-readable
- all resolver outputs are represented (context, selected packs, rules, overrides)
- required version stamps are included
- invariants (self-contained, conflict-free, auditable) are explicit and enforceable

STD-04

STD-04 — Plagiarism & IP-Safe Use Standard (Agents)

1) Purpose

Prevent agents (internal and external) from reproducing **copyrighted text, proprietary content, or code** they are not authorized to copy—especially from AXION's **Knowledge/Standards Library**—while still allowing legitimate reuse of approved internal assets and properly licensed material.

2) Scope

Applies to:

- Any **Agent** that reads AXION artifacts, Knowledge Library, Standards Library, previous project repos, or external sources.
- All outputs: code, documentation, copywriting, diagrams, prompts, tests, schemas, UI text.

3) Definitions

- **Knowledge Library (KSL):** internal repository of reference material, notes, examples, prior code, standards, and guidance.
- **Standards Library (STD):** AXION rules/constraints documents (may include examples).
- **Approved Internal Asset:** a file/module/template explicitly marked reusable for the current build (see "Allowed Sources").
- **Restricted Content:** any content not explicitly authorized for reuse in this build.
- **Verbatim Copy:** exact or near-exact reproduction of text/code beyond allowed limits.
- **Derived Work:** output that substantially mirrors a restricted source's structure or expression even if renamed.

4) Policy Summary

Agents **must generate original work** by default. Reuse is allowed **only** when:

1. the source is explicitly approved for reuse **for this build**, and
2. licensing/ownership permits it, and
3. attribution requirements are met.

---

5) Allowed Sources (what agents MAY reuse)

Agents may reuse content only from these categories:

A) Project-Owned Code in the Current Repo

- Reuse allowed if it already exists in the target repo.
- Must preserve internal attribution headers if present.

B) “Reusable” Internal Assets (explicitly whitelisted)

Only assets listed in the build’s **REUSE_ALLOWLIST** are reusable.
Allowlist entries must include:

- `path`
- `license/ownership` (e.g., “Company owned”, “MIT”, “Apache-2.0”)
- `reuse_scope` (exact copy allowed vs pattern-only)
- `required_attribution` (if any)

C) Open-Source Code with Compatible Licenses

Allowed only if:

- the license is compatible with the project's distribution model, and
- the required notice/attribution is included, and
- the reused portion is tracked in a reuse log.

D) Standards / Knowledge Library Content as "Rules", Not "Copy"

- Agents may use KSL/STD content for **concepts, constraints, checklists, and patterns**.
- Agents must **not copy** long passages of text or whole code blocks from KSL/STD unless the exact excerpt is whitelisted.

---

6) Prohibited Behavior (hard rules)

Agents MUST NOT:

1. **Copy code from KSL/STD examples into the build output** unless that specific example is in the allowlist.
2. **Reproduce large verbatim sections** of text from KSL/STD or external sources.
3. Use external sources as a "code donor" (copy/paste) without verifying license + logging reuse.
4. Produce output that is a **thin rewrite** of restricted content (rename variables, reorder lines, minimal edits).
5. Include proprietary vendor code, SDK internals, or leaked source content.

---

7) Safe Generation Rules (how agents should work)

Agents MUST follow this workflow:

7.1 Pattern extraction, not copying

When referencing KSL/STD:

- extract **requirements** (bullets, constraints, acceptance criteria)
- re-express in new words
- implement code **from scratch** using the requirements

7.2 “Small excerpt” rule (docs)

Unless whitelisted:

- no more than **25 words** verbatim from any single non-licensed text source
- excerpts must be quoted and attributed

7.3 “No large blocks” rule (code)

Unless whitelisted:

- do not reproduce a function/class/module copied from elsewhere
- re-implement using the spec, not the source code’s structure

7.4 Attribution and reuse logging

If any reuse occurs, the agent must add a record to `axion/reuse_log.json` (or equivalent) including:

- source (repo URL/file path)
- license

- what was reused (file, function names, line ranges)
- attribution included (yes/no, where)

---

8) Required Artifacts

8.1 REUSE_ALLOWLIST (required for every build)

Location: `axion/reuse_allowlist.json`

Minimum fields:

- `schema_version`
- `items[]: { path, owner, license, reuse_scope, allowed_excerpt_max_lines, required_attribution }`

8.2 REUSE_LOG (required if any reuse occurs)

Location: `axion/reuse_log.json`

Minimum fields:

- `schema_version`
- `entries[]: { timestamp, source, license, reused_parts, attribution_path }`

---

9) Enforcement Gates (AXION)

Add these gates to your gate set (names are placeholders; keep your naming convention):

GATE-PLAG-01 — Allowlist exists

- fail if `axion/reuse_allowlist.json` missing

GATE-PLAG-02 — Reuse requires logging

- if `reuse_allowlist` has any items OR agent declares reuse, require `axion/reuse_log.json`

GATE-PLAG-03 — Knowledge Library copy ban (default)

- fail if build outputs contain verbatim blocks matching KSL/STD example snippets **unless** the snippet is listed in allowlist
    - (Implementation: hash-based snippet registry or simple “known example” detection)

GATE-PLAG-04 — License notice enforcement

- if reuse_log includes OSS sources requiring NOTICE, fail if NOTICE file not updated

Proof required

- `PROOF: reuse_allowlist_present`
- `PROOF: reuse_log_present_if_reuse`
- `PROOF: license_notice_updated_if_required`
- `PROOF: plagiarism_scan_pass` (if you add a scanner)

---

10) Agent Instructions (drop-in prompt block)

Use this verbatim in your agent role prompt:

- Do not copy code or text from AXION Knowledge Library or Standards Library into outputs unless the exact source path is listed in `axion/reuse_allowlist.json`.
- Use KSL/STD for requirements and patterns only; implement from scratch.
- If you reuse anything (internal reusable assets or OSS), you must add an entry to `axion/reuse_log.json` and include required attribution/NOTICE changes.
- If unsure about license/permission, do not reuse; re-implement.

---

11) Human Review Checklist (fast)

- reuse_allowlist exists and is intentional
- reuse_log exists if any reuse happened
- NOTICE/attribution updated where needed
- outputs do not contain pasted KSL/STD example code
- any excerpts are short + attributed

Template System

Template System

- TMP-01 Template Types & Template Index (registry)
- TMP-02 Template File Contract (required sections + placeholder rules)
- TMP-03 Template Selection Rules (routing/gates → which templates apply)
- TMP-04 Template Fill Rules (direct/derived/standards inserts; no guessing)
- TMP-05 Template Completeness Rules (what “valid” means)

TMP-01

TMP-01 — Template Types & Template Index (Registry)

(Hardened Draft — Full)

1) Purpose

Define the Template System registry so Axion can:

- enumerate every template that exists
- select templates deterministically based on routing + gates
- support safe growth via versioning and non-redundant inventory
- stamp exactly which templates/versions were used in a kit

TMP-01 defines:

- the locked **Template Types**
- the **Template Index** format (machine-readable registry)
- minimum metadata required for selection, filling, gating, and traceability

Template file structure is TMP-02. Selection logic is TMP-03. Fill rules are TMP-04. Completeness is TMP-05.

---

2) Inputs

- Locked template types list (merged types)
- Standards snapshot contract (what templates can reference)
- Canonical spec model (what templates can reference)
- Governance/versioning policy (library versioning discipline)

---

3) Outputs

- A versioned **Template Library** containing:
    - template files (markdown templates)
    - a machine-readable **Template Index** registry
    - a library version stamp

The Template Index is the authoritative inventory the internal agent uses.

---

4) Template Types (Locked Set)

These are the only allowed template types:

1. Product / Requirements
2. Design
3. Architecture
4. Implementation *(includes planning)*
5. Security & Access
6. Quality *(includes a11y verification + performance checks)*
7. Operations *(includes runbooks/incident process)*
8. Data *(conditional)*
9. API / Interface Contracts *(conditional)*
10. Release *(conditional)*
11. Governance / Change Control *(conditional)*
12. Analytics / Telemetry *(conditional)*

Rule: New types require change control and Template Index schema version bump.

---

5) Template Index Contract (Machine-Readable Registry)

5.1 Template Index File

- File name (canonical): `template_index.json`
- Required top-level fields:
  - `template_library_version`
  - `template_index_version`
  - `generated_at`
  - `templates[]`

5.2 Template Entry Contract

Each template entry in `templates[]` must include:

Identity

- `template_id` (stable, unique; e.g., `PRD-01`, `DES-04`)
- `title`
- `type` (one of the 12 template types)
- `template_version` (semantic or numeric)
- `file_path` (within template library)
- `status` (enum): draft | active | deprecated

Applicability (selection filters)

- `applies_when` (object; deterministic filter set)
  - `routing` filters:
    - `category[]` (optional list)
    - `type_preset[]` (optional list)
    - `audience_context[]` (optional list)
    - `build_target[]` (optional list)
    - `skill_level[]` (optional list) *(rare; typically affects requiredness, not selection)*
  - `gates` filters:
    - `requires_data_enabled` (boolean|null)
    - `requires_auth_required` (boolean|null)
    - `requires_integrations_enabled` (boolean|null)
  - `conditions` (optional explicit boolean expressions if you support; otherwise omit)

Requiredness (how strongly it is enforced)

- `requiredness` (enum):
  - `always`
  - `conditional`
  - `optional`
- `required_by_skill_level` (object):
  - `{ beginner: required|optional|omit, intermediate: required|optional|omit, expert: required|optional|omit }`

Inputs (what it needs to fill)

- `inputs_required[]` (list of data sources/paths)
  - must reference known sources:
    - canonical spec paths
    - standards snapshot paths

        - work breakdown paths
        - acceptance map paths
- `derived_inputs[]` (optional list)
    - computed relationships needed (feature→workflows, etc.)

Outputs (what it produces in the kit)

- `output_path` (where the filled doc goes in kit)
- `output_artifact_id` (optional stable id for manifest indexing)

Upstream Dependencies (doc gating)

- `upstream_dependencies[]` (list of artifact/template IDs that must be valid first)
    - examples: `STD-03` snapshot must exist, canonical spec must exist, etc.

Compliance Gate Metadata

- `compliance_gate_id` (string; points to TMP-05 rule set)
- `compliance_requirements` (optional structured list of checks)

Cross-References (linking discipline)

- `references_entities[]` (list of canonical entity types it references: roles, features, workflows, etc.)
- `no_duplicate_truth` (boolean; must be true for all templates)

---

6) Template Library Versioning Rules (registry-level)

6.1 Library version

- `template_library_version` must change when:
    - a template is added/removed
    - a template structure changes materially
    - requiredness rules change materially
    - selection filters change materially

6.2 Template version

- `template_version` must change when:
  - template content structure changes
  - required fields change
  - placeholder variables change

6.3 Deprecation

- Deprecated templates remain in index with:
  - `status: deprecated`
  - `deprecated_at`
  - `replacement_template_id` (if any)

---

7) Invariants (must always be true)

1. Every template file must have a Template Index entry.
2. No Template Index entry may point to a missing file.
3. `template_id` is unique across the library.
4. `type` must be one of the locked types.
5. applies_when must be deterministic and not ambiguous.
6. required_by_skill_level must be present for every template.
7. output_path must conform to the kit file tree contract (KIT docs).
8. no_duplicate_truth must be true for all templates.

---

8) Failure Modes

- templates exist but aren't indexed (unselectable, inconsistent kits)
- ambiguous applies_when filters (non-deterministic selection)
- requiredness missing (agents don't know what to enforce)
- output_path inconsistent with kit layout (packaging break)
- stale index version with changed files (drift)

---

9) Definition of Done (TMP-01)

TMP-01 is complete when:

- template types are locked
- Template Index JSON contract is fully specified
- template entry schema covers: identity, applicability, requiredness, inputs/outputs, dependencies, compliance metadata
- versioning/deprecation rules are explicit
- invariants are enforceable via gates

TMP-02

TMP-02 — Template File Contract (Required Sections + Placeholder Rules)

(Hardened Draft — Full)

1) Purpose

Define the **required structure** of every template file so templates are:

- predictable for agents to read and fill
- machine-navigable (consistent headings, stable section IDs)
- gateable (completeness checks are possible)
- safe for growth (new templates follow the same contract)

TMP-02 defines template file shape and placeholder conventions. Filling behavior is TMP-04; completeness is TMP-05.

---

2) Inputs

- TMP-01 Template Index entry fields (template_id, type, version, etc.)
- Canonical Spec model (CAN-01) and ID rules (CAN-02)
- Standards snapshot contract (STD-03)
- Skill level rules (INT-03 and template requiredness metadata)

---

3) Outputs

A standardized template file format used across the entire library.

---

4) Template File Structure (Locked Order)

Every template file must contain these top-level sections **in this exact order**:

1. **Header Block** (required)
2. **Purpose** (required)
3. **Inputs Required** (required)
4. **Required Fields** (required)

5. **Optional Fields** (required)
6. **Rules** (required)
7. **Output Format** (required)
8. **Cross-References** (required)
9. **Skill Level Requiredness Rules** (required)
10. **Unknown Handling** (required)
11. **Completeness Gate** (required)

Rule: Missing any of these sections is a template contract violation.

4.1 Header Block (required fields)

The header must include:

- `Template ID:` (matches Template Index `template_id`)
- `Template Type:` (matches Template Index `type`)
- `Template Version:` (matches Template Index `template_version`)
- `Applies:` (brief; must not contradict Template Index filters)
- `Filled By:` (internal agent)
- `Consumes:` (canonical sources: spec/snapshot/plan/acceptance)
- `Produces:` (filled doc purpose)

---

5) Placeholder Rules (How variables are written)

5.1 Placeholder Syntax (locked)

All placeholders must follow one of these forms:

Direct insert

- `{{path.to.value}}`

Array insert (list/table)

- `{{path.to.array[]}}`

Derived insert (computed)

- `{{derive:NAME(args...)}}`

Optional placeholder

- `{{path.to.value | OPTIONAL}}`

Unknown-allowed placeholder (explicitly allowed)

- `{{path.to.value | UNKNOWN_ALLOWED}}`

Rule: No other placeholder syntax is allowed.

5.2 Allowed Data Sources (locked)

Placeholders may reference only:

1. **Canonical Spec**
- `{{spec.*}}` paths (entities and indexes)
2. **Resolved Standards Snapshot**
- `{{standards.*}}` paths (rules, constraints)
3. **Work Breakdown**
- `{{work.*}}` paths (units, dependencies)
4. **Acceptance Map**
- `{{acceptance.*}}` paths (criteria, proof)
5. **Submission/Trace IDs**
- `{{submission_id}}`, `{{spec_id}}`, `{{standards_id}}`, etc.

Rule: Templates must never pull from “chat memory” or freeform agent history.

5.3 No Creative Guessing (locked)

If a required placeholder cannot be resolved:

- if marked `UNKNOWN_ALLOWED` → fill with explicit UNKNOWN format (see section 10)
- otherwise → the template fails completeness (TMP-05) and blocks progression

---

6) Required Fields vs Optional Fields (rules)

6.1 Required Fields section

Must enumerate what fields must be populated for this template to be valid, with:

- field name
- expected source (spec/standards/work/acceptance)
- whether UNKNOWN is allowed

6.2 Optional Fields section

Must list optional enrichments, but must not create new truth.

---

7) Rules Section (template-local invariants)

Must include at minimum:

- **No duplicate truth rule** (reference canonical IDs)
- **No invention rule** (unknown or block)
- Any template-specific integrity rules

Rules must be checkable (not subjective).

---

8) Output Format Section (machine-navigable)

Must define:

- required headings (and their order)
- required tables/lists formats

- any stable subsection identifiers (e.g., “### 2.1 Roles Index”)

Rule: output format must be deterministic.

---

9) Cross-References Section

Must list:

- upstream artifacts it depends on (by ID)
- downstream templates that will reference this template
- canonical entity types referenced (roles/features/workflows/etc.)

---

10) Skill Level Requiredness Rules Section

Must define:

- which sections become required/optional/omitted by skill level
- minimum counts if relevant (e.g., expert requires deeper detail)

Rule: this must align with Template Index `required_by_skill_level`.

---

11) Unknown Handling Section (locked format)

Unknowns must be written inside templates in this exact block format:

UNKNOWN-<NNN>: [Area] <summary>
Impact: Low|Med|High
Blocking: Yes|No
Needs: <what input resolves it>
Refs: <spec_id/entity_id/field_path>

Rule: Unknowns in templates must also map back to Canonical Spec unknown objects (CAN-03) or be emitted into them during fill (TMP-04 rule).

---

12) Completeness Gate Section

Every template must end with a checklist that is objectively verifiable and maps to TMP-05 checks, including:

- required fields populated
- references resolve
- no contradictions inside template
- unknowns handled per policy

---

13) Failure Modes

- inconsistent placeholder syntax breaks filler
- pulling from non-authoritative sources causes drift
- optional fields become hidden requirements
- unknowns not structured prevents gating and resolution

---

14) Definition of Done (TMP-02)

TMP-02 is complete when:

- required section order is locked
- placeholder syntax and allowed sources are locked
- unknown block format is locked
- the contract is strict enough for automated completeness checks and safe growth

TMP-03

TMP-03 — Template Selection Rules (Routing/Gates → Which Templates Apply)

(Hardened Draft — Full)

1) Purpose

Define how the internal agent deterministically selects the **exact set of templates** to fill for a given project, based on:

- routing snapshot (category/type preset/audience/build_target/skill_level)
- gate flags (data/auth/integrations enabled)
- template requiredness rules

Selection must be reproducible and auditable.

---

2) Inputs

- Template Index (TMP-01)
- Resolver context:
  - routing snapshot (from intake/normalized input)
  - gate flags: `data.enabled`, `auth.required`, `integrations.enabled`
  - optional compliance flags (from NFR/compliance)
- Template library version (pinned)

---

3) Outputs

A **Template Selection Result** used by the Template Filler and Packager:

- `selection_id`
- `template_library_version_used`
- `resolver_context` (routing + gates)
- `selected_templates[]` (ordered list)
- `omitted_templates[]` (with omission reasons)
- `na_slots[]` (where templates are N/A; used by packaging)
- `created_at`

---

4) Selection Invariants (must always be true)

1. **Deterministic:** same context + same template library version ⇒ same selection result.
2. **No ambiguity:** a template is either selected or omitted with explicit reason.
3. **Requiredness enforced:** templates marked required must be selected when applicable.
4. **No silent omission:** if a template type is present in kit layout but no template applies, create `00_NA.md` with reason.
5. **No cross-contract conflicts:** selection must not require templates that cannot be filled due to missing upstream artifacts.

---

5) Selection Algorithm (Locked Steps)

Step 1 — Build Selection Context

Create `context` from canonical sources:

- routing: skill_level, category, type_preset, build_target, audience_context
- gates: data.enabled, auth.required, integrations.enabled
- optional: compliance flags (if used)

Step 2 — Filter Templates by Applies_When

For each template in Template Index:

- Evaluate `applies_when` against context.

Rules:

- If template's `applies_when` is empty → treat as applies to all.
- If `requires_data_enabled == true` then require `data.enabled == true`
- If `requires_data_enabled == false` then require `data.enabled == false` *(rare; use only when needed)*
- Same for auth/integrations.

If routing filters exist, require match within allowed lists.

Step 3 — Apply Requiredness by Skill Level

For each template that passes applies_when:

- Determine requiredness for this skill level using:

    - template entry `required_by_skill_level`

Interpretation:

- `required` → must select
- `optional` → select if applicable and not excluded by policy
- `omit` → do not select

Step 4 — Enforce Baseline Coverage (Senior Coverage)

Selection must ensure baseline template coverage exists for a project:

Baseline required template types (default):

- Product/Requirements
- Architecture
- Implementation
- Security
- Quality

Conditional baseline:

- Design (if UI/consumer)
- Data (if data.enabled)
- API Contracts (if service/API build or integrations present)
- Ops (if deployable/runtime operations required by preset)
- Release/Governance/Analytics only if context triggers them

If baseline cannot be satisfied (no template exists), selection fails (hard stop) because the library is incomplete for that context.

Step 5 — Determine Ordering

Order selected templates deterministically using:

1. upstream_dependencies graph (topological order)
2. template type ordering (global stable order list)
3. template_id lexical tie-breaker

Global type order (locked):

1. Product/Requirements
2. Design
3. Architecture
4. Data

5. API/Interface Contracts
6. Security & Access
7. Implementation
8. Quality
9. Operations
10. Release
11. Governance
12. Analytics

Step 6 — Emit Selection Result

Record:

- selected templates
- omitted templates with reasons:
  - `not_applicable` (fails applies_when)
  - `skill_level_omit`
  - `policy_excluded`
  - `missing_upstream_dependency` *(should typically fail instead of omit)*
- N/A slots required by kit packaging (if no templates selected for slot)

---

6) Omission Reasons (Locked Enum)

- `not_applicable`
- `skill_level_omit`
- `policy_excluded`
- `missing_dependency` *(treated as error by default)*

---

7) Failure Modes

- ambiguous applies_when logic creates non-deterministic selection
- baseline coverage missing causes “senior gaps”
- ordering not consistent causes build drift
- selection includes templates whose upstream artifacts are not generated yet

---

8) Definition of Done (TMP-03)

TMP-03 is complete when:

- selection steps are explicit and deterministic
- requiredness by skill level is enforced
- baseline coverage rules are defined and gateable
- selection produces a recorded list of selected + omitted templates with reasons
- ordering rules are explicit and stable

TMP-04

TMP-04 — Template Fill Rules (Direct/Derived/Standards Inserts; No Guessing)

(Hardened Draft — Full)

1) Purpose

Define how the internal agent fills templates into completed kit documents by:

- resolving placeholders from authoritative artifacts
- computing derived inserts deterministically
- injecting standards constraints from the snapshot
- preventing invention/guessing
- emitting unknowns as structured UNKNOWN blocks and canonical unknown objects

TMP-04 is the “filler law.” It must be deterministic.

---

2) Inputs

- Template Selection Result (TMP-03)
- Template files (TMP-02 contract)
- Canonical Spec (CAN-01)
- Resolved Standards Snapshot (STD-03)
- Work Breakdown (A2.4)
- Acceptance Map (A2.5)
- Unknowns list (CAN-03; may be appended by filler under strict rules)

---

3) Outputs

For each selected template:

- a **filled document** at its output_path
- a **fill report** (optional but recommended for audit)
- any new canonical unknowns created (strictly controlled)

---

4) Fill Invariants (must always be true)

1. **Authoritative sources only:** fill uses only CAN/STD/PLAN/QA artifacts (no chat memory).
2. **No invention:** missing required values become UNKNOWN or gate failure.
3. **Deterministic derived values:** derived inserts must be reproducible.
4. **No duplicate truth:** filled docs reference canonical IDs; they don't redefine.
5. **Unknown traceability:** any UNKNOWN in a filled doc must map to a canonical unknown object.

---

5) Placeholder Resolution Rules (Locked)

5.1 Direct Inserts

For placeholder `{{path}}`:

- resolve path against permitted sources (in this precedence order):
    1. Canonical Spec
    2. Standards Snapshot
    3. Work Breakdown
    4. Acceptance Map
    5. Trace IDs (submission_id/spec_id/etc.)

If not found:

- if placeholder marked `| OPTIONAL` → render empty or "N/A"
- if placeholder marked `| UNKNOWN_ALLOWED` → emit UNKNOWN block
- else → template fill fails (hard stop)

5.2 Array Inserts

For `{{path.to.array[]}}`:

- render using the output format specified by the template:
    - list or table format must be deterministic
- ordering must be stable:
    - by canonical ID order, or by Work Breakdown order if relevant

5.3 Derived Inserts

For `{{derive:NAME(args)}}`:

- derived functions must be from an approved catalog (see 6)
- derived outputs must be deterministic and traceable

---

6) Derived Insert Catalog (Approved Functions)

Derived functions exist to compute relationships already present in canonical spec indexes/cross-maps. Examples of allowed functions:

- `derive:WORKFLOW_TO_FEATURES(workflow_id)`
- `derive:FEATURE_TO_WORKFLOWS(feature_id)`
- `derive:FEATURE_TO_OPERATIONS(feature_id)`
- `derive:ROLE_TO_WORKFLOWS(role_id)`
- `derive:SCREEN_TO_WORKFLOWS(screen_id)`
- `derive:UNIT_SCOPE_SUMMARY(unit_id)` *(from work breakdown scope_refs)*
- `derive:UNIT_ACCEPTANCE_ITEMS(unit_id)` *(from acceptance map)*

Rules:

- Derived functions cannot invent new entities.
- Derived functions must operate only on existing canonical indexes/cross-maps.

---

7) Standards Inserts (Ruleset Injection)

Templates may request standards content in two forms:

7.1 Direct rule inserts

- `{{standards.rules[rule_id]}}` *(conceptual; actual path depends on snapshot structure)*

7.2 Category summary inserts

- Insert a deterministic summary of a standards category:
  - Engineering constraints summary
  - Security baseline summary

    - Design system constraints summary

Rule:

- summaries must be derived from the snapshot, not written from memory.

---

8) Unknown Handling During Fill (Strict)

8.1 When filler may create unknowns

Only when:

- a placeholder is marked `UNKNOWN_ALLOWED`
- and the missing value cannot be resolved from authoritative sources

8.2 Unknown creation must produce CAN-03 objects

For every UNKNOWN block inserted into a filled doc, the filler must:

- create (or reference) a canonical unknown object:
    - `unknown_type`
    - `blocking` policy per CAN-03
    - attachment ref: field_path or entity_id + template_id/section
- include the unknown_id in the doc's unknown block

8.3 Unknown reuse

If an unknown already exists for the same gap:

- reuse the existing unknown_id (no duplicates)

---

9) Fill Report (Recommended Output)

For audit and debugging, each filled doc may have a fill report entry:

- template_id + version

- inputs used (artifact IDs)
- placeholders resolved count
- unknowns created/reused count
- unresolved placeholders (if any; should be none in successful fills)

---

10) Failure Modes

- placeholders resolve from non-authoritative sources → drift
- derived inserts not stable → non-deterministic docs
- unknowns inserted without canonical unknown IDs → untraceable gaps
- templates rewrite truth instead of referencing IDs → contradictions

---

11) Definition of Done (TMP-04)

TMP-04 is complete when:

- placeholder resolution precedence and missing-value policy are locked
- derived insert catalog is defined and restricted
- standards injection is defined as snapshot-derived only
- unknown insertion creates/reuses canonical unknown objects
- fill behavior is deterministic and gateable

TMP-05

TMP-05 — Template Completeness Rules (What “Valid” Means)

(Hardened Draft — Full)

1) Purpose

Define the deterministic rules that decide whether a **filled template** is valid and can be included in the kit as a trusted guidance document.

TMP-05 is the **Template Gate**: it enforces completeness, integrity, and non-contradiction so downstream agents don’t drift.

---

2) Inputs

- Filled template document (output of TMP-04)
- The template definition (TMP-02 contract)
- Canonical Spec + Standards Snapshot (for reference checks)
- Template Index metadata (required_by_skill_level, applies_when)

---

3) Outputs

A **Template Gate Report** (pass/fail) that includes:

- template_id + version
- output_path
- status: pass/fail
- issues[] with:
    - rule_id
    - pointer (section heading / placeholder key / field_path)
    - error_code
    - remediation

---

4) Validity Invariants (must always be true)

1. **Contract shape:** filled doc preserves required section structure and ordering.

2. **Required fields populated:** required placeholders are resolved, or allowed UNKNOWN policy is followed.
3. **No invention:** no content introduces entities not in canonical spec.
4. **Reference integrity:** all referenced IDs resolve to canonical spec entities.
5. **No contradictions:** doc does not contradict canonical spec, standards snapshot, or itself.
6. **Skill-level alignment:** required_by_skill_level rules are respected.

---

5) Completeness Rule Sets (Locked)

5.1 Structure Rules

TMP5-STRUCT-01

- **Rule:** All required top-level sections exist and are in the locked order (TMP-02).
- **Fail code:** `REQUIRED`
- **Pointer:** missing section heading

TMP5-STRUCT-02

- **Rule:** Required subsections defined by template's Output Format must exist.
- **Fail code:** `REQUIRED`

5.2 Placeholder Resolution Rules

TMP5-FILL-01

- **Rule:** No unresolved placeholders remain in the filled doc.
- **Fail code:** `REQUIRED`
- **Pointer:** placeholder token location

TMP5-FILL-02

- **Rule:** Required placeholders must not be empty unless marked OPTIONAL.
- **Fail code:** `REQUIRED`

TMP5-FILL-03

- **Rule:** UNKNOWN blocks only allowed where placeholders were marked `UNKNOWN_ALLOWED` and must include `unknown_id`.
- **Fail code:** `INVALID_FORMAT`

5.3 Reference Integrity Rules

TMP5-REF-01

- **Rule:** Any referenced entity IDs (role_id, feature_id, workflow_id, etc.) must resolve in canonical spec.
- **Fail code:** `INVALID_REFERENCE`

TMP5-REF-02

- **Rule:** Any cross-template references must be valid template_ids present in the Template Index.
- **Fail code:** `INVALID_REFERENCE`

5.4 No Duplicate Truth / No Invention Rules

TMP5-TRUTH-01

- **Rule:** Template must not define new roles/features/workflows/data objects outside canonical spec.
- **Fail code:** `INVALID_REFERENCE` *(or a dedicated code if you add one later)*

TMP5-TRUTH-02

- **Rule:** Template must not restate canonical truth in conflicting terms (e.g., feature list differs).
- **Fail code:** `INVALID_FORMAT` *(or dedicated contradiction code later)*

5.5 Standards Compliance Rules

TMP5-STD-01

- **Rule:** If template claims a standard (auth method, quality check, etc.), it must match the resolved standards snapshot.
- **Fail code:** `INVALID_REFERENCE`
- **Pointer:** section making conflicting claim

5.6 Skill-Level Rules (Requiredness + Thresholds)

TMP5-SKILL-01

- **Rule:** Sections marked “required” for the current skill level must be present and complete.
- **Fail code:** `REQUIRED`

TMP5-SKILL-02

- **Rule:** Minimum counts (if declared by the template) must be met (e.g., expert requires deeper detail).
- **Fail code:** `MIN_ITEMS`

5.7 Unknown Policy Rules

TMP5-UNK-01

- **Rule:** Any UNKNOWN in a core decision area (as defined by CAN-03 blocking policy) must be marked blocking and must trigger a gate failure if policy forbids it at this stage.
- **Fail code:** `DEPENDENCY_MISSING` or `MIN_ITEMS` *(depending on what’s missing)*

TMP5-UNK-02

- **Rule:** UNKNOWN blocks must be consolidated in the Unknown Handling section and must reference canonical unknown IDs.
- **Fail code:** `INVALID_FORMAT`

---

6) Contradiction Checks (minimal deterministic set)

The template gate must fail if:

- a list of features in the template does not match canonical feature IDs
- a role name/capability differs from canonical spec
- an auth requirement contradicts standards snapshot
- the template contradicts itself (e.g., “no auth” and “auth required”)

---

7) Failure Modes

- templates pass with unresolved placeholders → external agent guessing
- templates include invented scope → drift
- unknowns not linked to canonical unknowns → untraceable gaps
- skill-level rules ignored → inconsistent depth behavior

---

8) Definition of Done (TMP-05)

TMP-05 is complete when:

- rule sets cover structure, fill, references, truth integrity, standards compliance, skill-level requirements, unknown policy
- each rule has deterministic failure codes and pointers
- the output gate report format is consistent and usable by the pipeline

TMP-06

TMP-06 — Pack Hierarchy & Placement Contract (Outline + One End-to-End Path)

1) Purpose

Define the **locked standard layout** for how filled templates are packaged into a kit, across the hierarchy:
App → **Domain** → **Feature** → **Screen** → **Component**, so the agent always knows:

- where to find things
- what's required at each level
- how variability is expressed (filled vs N/A)

---

2) Core Concepts (locked)

2.1 Pack Levels

- **APP Pack** (entire product)
- **DOMAIN Pack** (major system area: Frontend/Backend/Data/Security/Ops/etc.)
- **FEATURE Pack** (capability slice)
- **SCREEN Pack** (UI surface for a workflow step)
- **COMPONENT Pack** (reusable UI building block; only when promoted)

2.2 Template Slots (universal categories)

Every pack uses the same slot folders (even if empty):

- `01_requirements/`
- `02_design/`
- `03_architecture/`
- `04_implementation/`
- `05_security/`
- `06_quality/`
- `07_ops/`
- `08_data/`
- `09_api_contracts/`
- `10_release/`
- `11_governance/`

- `12_analytics/`

Rule: If a slot isn't applicable, it must contain `00_NA.md` with reason.

---

3) Pack Root Required Files (locked)

Every pack folder must include:

1. `00_pack_meta.md` (required)
- pack_level: APP/DOMAIN/FEATURE/SCREEN/COMPONENT
- pack_id (stable)
- scope_refs[] (canonical spec IDs included)
- required_slots[] (computed)
- upstream_deps[] (pack-level dependencies)
- status (draft/complete)
- version stamps
2. `00_pack_index.md` (required)
- table of contents (links to files)
- "what to read first" for this pack
3. `00_gate_checklist.md` (required)
- pass/fail checklist for this pack being "complete"
- references Acceptance Map IDs if relevant

---

4) Required Slots by Pack Level (locked minimums)

4.1 APP Pack (minimum required slots)

- Requirements, Design, Architecture, Implementation, Security, Quality
- Data/API/ops/release/governance/analytics are conditional

4.2 DOMAIN Pack (minimum required slots)

- Architecture, Implementation, Quality
- Security conditional (if domain touches auth/data/control)
- Design conditional (frontend domain)
- Ops conditional (ops domain / deploy concerns)

4.3 FEATURE Pack (minimum required slots)

- Requirements slice, Implementation slice, Quality slice
- Data/API/Security conditional

4.4 SCREEN Pack (minimum required slots)

- Design (screen layout), Quality (screen checks)
- Security conditional (access rules)
- Implementation optional (if screen has tricky behavior)

4.5 COMPONENT Pack (minimum required slots)

- Design (component rules), Quality (a11y/tests)
- Implementation optional

---

5) Template Placement Rules (locked)

5.1 One Source of Truth

- Templates must reference canonical IDs; no redefining features/roles/workflows.

5.2 Where templates live

- Product/Requirements templates go in `01_requirements/`
- Design templates go in `02_design/`
- Architecture templates go in `03_architecture/`
- Implementation templates go in `04_implementation/`
- Security templates go in `05_security/`
- Quality templates go in `06_quality/`
- Operations templates go in `07_ops/`
- Data templates go in `08_data/`
- API/Interface templates go in `09_api_contracts/`
- Release templates go in `10_release/`
- Governance templates go in `11_governance/`
- Analytics templates go in `12_analytics/`

5.3 N/A rule (no silent omissions)

If a slot has no applicable templates:

- create `00_NA.md` with:

    - “Why N/A”
    - what would have triggered it
    - who decided (system default vs user decision)

---

6) End-to-End Example Path (locked “one path”)

This is the canonical sequence of packs the internal agent must be able to generate for a typical app feature.

Example Path: APP → Frontend Domain → Feature → Screen → Component

A) APP Pack

`/kit/app/`

- `00_pack_meta.md`
- `00_pack_index.md`
- `00_gate_checklist.md`
- `01_requirements/`
    - PRD-01, PRD-02, PRD-03, PRD-04, PRD-05, PRD-06
- `02_design/`
    - DES-01, DES-02, DES-03
- `03_architecture/`
    - ARC-01, ARC-02, ARC-03, ARC-04
- `04_implementation/`
    - IMP-01, IMP-02, IMP-03, IMP-04
- `05_security/`
    - SEC-01, SEC-02, SEC-03, SEC-05
- `06_quality/`
    - QA-01, QA-02, QA-03, QA-04
- Conditional slots: `08_data/`, `09_api_contracts/`, etc. (filled or N/A)

B) DOMAIN Pack (Frontend)

`/kit/domains/frontend/`

- pack root files (meta/index/gate)
- `02_design/`
    - DES-04 (screen specs set)
    - DES-05 (states/motion)

    - DES-06 (a11y requirements)
- `04_implementation/`
    - IMP-05 (frontend slice guide)
- `06_quality/`
    - QA-06 (a11y verification)
    - QA-04 (frontend regression checklist slice)
- non-applicable slots have `00_NA.md`

C) FEATURE Pack (Task CRUD)

`/kit/features/feat_task_crud/`

- pack root files (meta/index/gate)
- `01_requirements/`
    - PRD-03 feature excerpt (or feature slice doc)
    - PRD-04 workflow excerpt(s)
- `02_design/` *(optional but common)*
    - links to relevant screen specs
- `04_implementation/`
    - IMP-05 Feature Slice Implementation Guide (task CRUD)
- `06_quality/`
    - QA-01 acceptance items for this feature
    - QA-03 proof requirements for this feature
- `08_data/` *(if introduces/uses Task entity)*
    - DATA-02 excerpt for Task
- `09_api_contracts/` *(if has endpoints)*
    - API-02 excerpt for /tasks

D) SCREEN Pack (Task List Screen)

`/kit/screens/scr_task_list/`

- pack root files (meta/index/gate)
- `02_design/`
    - DES-04 screen layout spec (Task List)
- `05_security/` *(conditional)*
    - access rules (role access summary)
- `06_quality/`
    - screen acceptance checks (states, behaviors, a11y)

E) COMPONENT Pack (TaskCard)

`/kit/components/cmp_task_card/`

- pack root files (meta/index/gate)
- `02_design/`
  - component usage rules (variants, props, do/don't)
- `06_quality/`
  - a11y + render-state checks

---

7) Completion Definition (locked)

A pack is "complete" when:

- required root files exist
- required slots for that pack level are filled **or** explicitly N/A
- all templates inside pass their own completeness gates
- pack gate checklist passes

If you want, next we can write the **exact file tree standard** (the literal folder names and naming rules) as the final "locked layout" version.

Document Build Order & Gates

Document Build Order & Gates

- ORD-01 Build Order Graph (doc dependency graph)
- ORD-02 Gate DSL / Gate Rules (how compliance is checked)
- ORD-03 Per-Doc Gate Checklist (standard gate format)

ORD-01

ORD-01 — Build Order Graph (Doc Dependency Graph)

(Hardened Draft — Full)

1) Purpose

Define the authoritative **document dependency graph** and a valid **topological build order** for the Axion internal documentation and artifact system.

ORD-01 exists to enforce:

- “no forward progress without compliance” at the documentation and artifact level
- deterministic build sequencing (docs and artifacts are produced in the correct order)
- explicit upstream/downstream dependencies (no hidden coupling)

---

2) Inputs

- Locked doc sets (SYS, INT, CAN, STD, TMP, KIT, VER, STATE, GOV, EXEC, PLAN)
- Stage architecture (SYS-03)
- Gate model (SYS-07)

---

3) Outputs

- `doc_dependency_graph` (nodes + edges)
- `canonical_build_order[]` (topological order)
- `critical_path[]` (minimum backbone order)
- Rule: graph must be acyclic

---

4) Graph Model (Locked)

4.1 Node

A node represents a doc (or contract artifact) identified by:

- `doc_id` (e.g., SYS-01, INT-03)

- `category` (SYS/INT/CAN/STD/TMP/ORD/etc.)
- `lock_level` (LOCKED/CONFIGURABLE/ADVISORY) *(declared in each doc)*

4.2 Edge

An edge `A -> B` means:

- **B depends on A**
- B cannot be authored/locked until A is complete and compliant (gate pass)

---

5) Dependency Graph (High-Level Edges)

5.1 Foundations

- SYS-01 → SYS-02, SYS-03, SYS-04, SYS-05, SYS-06, SYS-07, SYS-08, SYS-09, SYS-10
- SYS-03 ↔ SYS-04 (must align; resolve by authoring SYS-03 then SYS-04)
- SYS-06 depends on SYS-03 + SYS-04
- SYS-07 depends on SYS-03
- SYS-10 depends on SYS-07 (enforcement references)

5.2 Intake

- INT-01 → INT-02 → INT-03 → INT-05
- INT-01 + INT-02 → INT-04
- INT-03 → INT-05

5.3 Canonical Model

- CAN-01 → CAN-02, CAN-03
- INT-02 → CAN-01 (schema alignment)
- INT-03 influences CAN-03 blocking policy thresholds

5.4 Standards

- STD-01 → STD-02 → STD-03
- CAN-01 influences what standards must be representable (constraints binding)

5.5 Templates

- TMP-01 → TMP-02 → TMP-04 → TMP-05
- TMP-01 → TMP-03 (selection uses index metadata)
- CAN-01/02/03 + STD-03 → TMP-04 (fill rules depend on truth + snapshot)
- TMP-05 aligns with SYS-07 template gate concept

5.6 Build Order & Gates

- ORD-01 depends on locked doc sets (it references them)
- ORD-02 depends on SYS-07 (gate model) + INT-05 (issue format patterns) + TMP-05 (template gate patterns)
- ORD-03 depends on ORD-02

5.7 Kit System (contracts)

- KIT docs depend on:
    - TMP (template outputs and placement)
    - CAN/STD (truth artifacts included)
    - SYS-06 traceability requirements
    - ORD gate enforcement (packaging gate)
      (*KIT-01..KIT-04 are included as active nodes (not reserved). Packaging gates must validate against KIT contracts.)*

5.8 Verification/State/Governance/Exec

- VER docs depend on SYS-07 + Acceptance Map rules (PLAN)
- STATE docs depend on PLAN + VER proof model
- GOV docs depend on SYS-08 + SYS-10
- EXEC docs depend on ORD + all core contracts

---

6) Canonical Build Order (Topological Order)

This is the authoritative order for authoring/locking docs (matches what we already used, formalized here):

Foundations

1. SYS-01
2. SYS-03
3. SYS-04
4. SYS-07
5. SYS-10
6. SYS-02
7. SYS-05
8. SYS-06
9. SYS-08
10. SYS-09 *(provisional; final compliance pass later)*

Intake

11. INT-01
12. INT-02
13. INT-03
14. INT-04
15. INT-05

Canonical Model

16. CAN-01
17. CAN-02
18. CAN-03

Standards

19. STD-01
20. STD-02
21. STD-03

Templates

22. TMP-01
23. TMP-02
24. TMP-03
25. TMP-04
26. TMP-05

Order & Gates

27. ORD-01

28. ORD-02
29. ORD-03

PLAN, VER, KIT, STATE, GOV, EXEC follow in the build order after ORD.

---

7) Critical Path (Minimum Backbone)

If only the minimal backbone must be locked to run the system:

- SYS-01, SYS-03, SYS-07, SYS-10
- INT-01..INT-05
- CAN-01..CAN-03
- STD-01..STD-03
- TMP-01..TMP-05
- KIT-01..KIT-04
- EXEC-01..EXEC-03

---

8) Failure Modes

- cyclic dependencies (graph invalid)
- hidden dependencies not represented as edges (causes drift)
- docs authored out of order (downstream contradictions)
- “provisional” docs treated as locked without compliance pass (especially SYS-09)

---

9) Definition of Done (ORD-01)

ORD-01 is complete when:

- node/edge model is defined
- dependency edges cover all authored doc sets so far
- canonical build order is a valid topological sort
- critical path is defined
- cycles are explicitly disallowed and detectable

ORD-02

ORD-02 — Gate DSL / Gate Rules (How Compliance Is Checked)

(Hardened Draft — Full)

1) Purpose

Define a **minimal, deterministic Gate DSL** and rule evaluation model so Axion can express and enforce compliance checks consistently across:

- docs (SYS/INT/CAN/STD/TMP/etc.)
- artifacts (submission/spec/snapshot/templates/kit)
- pipeline stages

ORD-02 does **not** define every domain's rules; it defines the **language and contract** for expressing them.

---

2) Inputs

- SYS-07 gate model (what gate types exist conceptually)
- INT-05 validator output contract (issue structure patterns)
- TMP-05 template gate expectations (template validity rules)
- Artifact IDs and version stamps (SYS-06 traceability)

---

3) Outputs

A gate rule representation that produces:

- deterministic pass/fail results
- standardized gate reports (issues with pointers and rule IDs)
- support for "no forward progress without compliance"

---

4) Gate DSL (Minimal Operators, Locked)

4.1 Gate Declaration

Each gate is declared as:

- `gate_id`
- `target` (artifact/doc ID or file path)
- `mode` (hard_stop | warn_only)
- `rules[]` (list of rule expressions)
- `on_fail` (stop | continue_with_warnings)
- `report_format` (must match gate report contract)

4.2 Rule Expression Types (allowed primitives)

A) Existence / Non-empty checks

- `exists(path)`
- `non_empty(path)` *(string/array/object has content)*

B) Equality / membership

- `eq(path, value)`
- `in(path, [values...])`

C) Count / threshold

- `count(path) >= N`
- `count(path) == N`
- `count(path) <= N`

D) Reference integrity

- `ref_exists(ref_path, index_path)`
  (e.g., each role_ref in permissions exists in roles_by_id)

E) Schema validation hook

- `schema_valid(artifact_id)`
  (expects INT-05 output exists and is_valid==true)

F) Hash/version pin checks

- `version_eq(path, value)`

- `hash_eq(path, value)` *(optional but supported)*

G) Composite logic

- `all([rule1, rule2, ...])`
- `any([rule1, rule2, ...])`
- `not(rule)`

Rule: Only these operators are allowed in the minimal DSL.

---

5) Rule Metadata (required for every rule)

Every rule must include:

- `rule_id`
- `description`
- `severity` (error|warning) *(must align to gate mode)*
- `pointer_paths[]` (what to point at when failing)
- `error_code` (from allowed error code catalog for that gate category)
- `expression` (one of the DSL expressions above)
- `remediation` (what to change)

---

6) Evaluation Model (Deterministic)

6.1 Input sources

Rules evaluate only against authoritative artifacts:

- submission record
- validation result
- normalized input
- standards snapshot
- canonical spec
- work breakdown / acceptance map
- filled templates
- kit manifest/index

- state snapshot + proof log

No chat context.

6.2 Ordering

Evaluate rules in deterministic order:

1. gate-level ordering (rules array order)
2. inside `all/any`, preserve listed order

6.3 Result aggregation

- gate passes if no `severity=error` rule fails
- warnings are included in report but do not block unless gate mode is hard_stop and policy requires

---

7) Gate Report Contract (produced by any gate)

A gate report must contain:

- `gate_id`
- `target`
- `status` (pass|fail)
- `executed_at`
- `issues[]`:
  - `issue_id`
  - `severity` (error|warning)
  - `error_code`
  - `rule_id`
  - `pointer` (field_path / template section / file path)
  - `message`
  - `remediation`
  - `meta` (optional)

Rule: rule_id is mandatory. No freeform failures.

---

8) Override Hook (policy)

ORD-02 supports overrides only as a record type (evaluation remains strict):

- a rule may declare `overridable: true|false`
- if overridable and override record exists, report still includes failure but status may be pass-with-override depending on policy

Default: for documentation and system law, overridable is false.

---

9) Failure Modes

- adding new operators makes evaluation inconsistent
- rule lacks pointer_paths so remediation is unclear
- rule lacks rule_id so compliance can't be audited
- rules evaluate against non-authoritative sources (drift risk)

---

10) Definition of Done (ORD-02)

ORD-02 is complete when:

- gate declaration contract is defined
- minimal DSL operators are locked
- rule metadata requirements are explicit
- evaluation model is deterministic
- gate report contract is explicit and consistent

ORD-03

ORD-03 — Per-Doc Gate Checklist (Standard Gate Format)

(Hardened Draft — Full)

1) Purpose

Define the standard way each internal doc/artifact declares its own **compliance gate checklist**, using the Gate DSL contract (ORD-02), so:

- every doc is lockable
- “no forward progress without compliance” applies consistently
- gate checks are machine-readable and auditable

ORD-03 defines the **format** and required fields for per-doc gates.

---

2) Inputs

- ORD-02 Gate DSL (operators + report contract)
- The doc being gated (e.g., SYS-10, INT-03, TMP-04)
- The doc’s Definition of Done criteria (DoD sections)

---

3) Outputs

A per-doc gate checklist object (stored in one of two allowed ways):

- embedded at the end of the doc under a dedicated heading, **and/or**
- stored as a separate gate file referenced by doc_id

Either way, the format must be identical.

---

4) Standard Gate Checklist Format (Locked)

4.1 Checklist Header (required)

- `doc_id`
- `doc_version`
- `gate_id`
- `gate_mode` (hard_stop | warn_only)
- `target` (doc_id or artifact path)
- `depends_on[]` (upstream doc_ids that must already be compliant)
- `rules[]` (list of gate rules)
- `outputs` (what artifacts/reports are produced)

4.2 Rule Entry (required)

Each rule must follow ORD-02 metadata requirements:

- `rule_id`
- `description`
- `severity` (error|warning)
- `error_code`
- `pointer_paths[]`
- `expression`
- `remediation`
- `overridable` (true|false; default false)

---

5) Required Rule Categories (minimum set for any doc)

Every doc's gate checklist must include rules for:

CAT-01 Structural Completeness

- required sections exist
- required headings present

CAT-02 Non-ambiguity

- no “should/usually/ideally” in enforceable sections *(only for system-law docs; optional elsewhere)*
- terms used are defined or referenced *(SYS-09 linking)*

CAT-03 Dependency Alignment

- doc does not contradict upstream dependencies (e.g., SYS-10 aligns with SYS-07)

CAT-04 Traceability / Versioning

- doc declares version and lock level
- references include doc_ids and versions where required

Rule: Some categories may be marked warning-only for provisional docs (e.g., SYS-09 during early drafting), but must be hard_stop at lock time.

---

6) Gate Location Rules (where checklist lives)

A doc’s gate checklist must appear under a fixed heading:

- `## Gate Checklist (ORD-03)`

If stored externally, the doc must include:

- `Gate Checklist Ref: <path>` in its header

---

7) How DoD maps to gate rules

DoD statements must be converted into explicit gate rules:

- “All boundaries map to gates” → `count(boundary_mappings) >= N`
- “No optional language” → pattern check (if supported) or manual checklist

If a DoD item cannot be expressed deterministically, it must be:

- explicitly marked as **Manual Gate Item**, with a required reviewer role.

---

8) Manual Gate Items (allowed but controlled)

Some checks may require human review. If so:

- rule entry must include `manual: true`
- required reviewer role must be declared
- pass/fail must still be recorded in gate report

Rule: manual checks must be rare for core contracts; the goal is deterministic gating.

---

9) Failure Modes

- docs declared “locked” without passing their gate checklist
- DoD remains narrative and cannot be enforced
- missing depends_on causes out-of-order locking
- override allowed on system-law docs (unsafe)

---

10) Definition of Done (ORD-03)

ORD-03 is complete when:

- standard checklist format is fully specified
- rule entry schema aligns with ORD-02
- minimum rule categories are defined
- gate location rules are explicit
- DoD-to-gate mapping guidance is explicit

Planning & Decomposition System

Planning & Decomposition System

- PLAN-01 Work Breakdown Rules (how spec becomes work units)
- PLAN-02 Acceptance Map Rules (how proof is attached to units)
- PLAN-03 Sequencing Heuristics (foundational first, vertical slices, etc.)

PLAN-01

PLAN-01 — Work Breakdown Rules (How Spec Becomes Work Units)

(Hardened Draft — Full)

1) Purpose

Define how Axion converts the **Canonical Spec** into a deterministic **Work Breakdown**:

- work units with stable unit IDs
- explicit scope per unit (spec ID references)
- dependencies and sequencing constraints
- deliverables per unit

This doc ensures decomposition is consistent, auditable, and resistant to drift.

---

2) Inputs

- Canonical Spec (CAN-01 entities + indexes)
- ID/reference rules (CAN-02)
- Unknowns policy (CAN-03)
- Resolved Standards Snapshot (STD-03) for constraints affecting sequencing
- (Optional) system policies for unit sizing and risk flags

---

3) Outputs

A Work Breakdown artifact containing:

- `work_breakdown_id`
- `spec_id`
- `units[]` (ordered list)
- `dependency_graph`
- `unit_index` (unit_id → unit)
- `created_at`

---

4) Planning Invariants (must always be true)

1. **Traceability:** every work unit must reference at least one spec entity ID (`scope_refs[]`).
2. **Deterministic:** same canonical spec + same standards snapshot ⇒ same unit set and dependency graph (structure-level).
3. **No cycles:** dependency graph must be acyclic.
4. **No "mega unit":** decomposition cannot be "build the whole app" in one unit.
5. **No missing foundations:** baseline units (setup/security/quality scaffolding) must exist when required by context.

---

5) Work Unit Contract (Locked Structure)

Each `unit` must include:

5.1 Identity

- `unit_id` (stable; format `unit_<slug>`)
- `title`
- `description`

5.2 Scope

- `scope_refs[]` (list of canonical IDs: feat_*, wf_*, scr_*, op_*, dat_*, rol_*, rule IDs)
- `in_scope` (short summary)
- `out_of_scope` (explicit exclusions)

5.3 Dependencies

- `depends_on[]` (unit_id list)
- `unblocks[]` (optional; derived)

5.4 Deliverables

- `deliverables[]` (explicit artifacts/behaviors that must exist after unit completion)

5.5 Risk/Constraints (optional but recommended)

- `risk_flags[]` (e.g., auth, data_migration, external_integration)
- `constraints_refs[]` (standards rule_ids relevant to unit)

6) Decomposition Rules (How units are created)

6.1 Baseline Unit Set (context-driven, required)

The planner must always produce baseline units based on context:

PLAN1-BASE-01 Project Setup

- always required for new builds
- scope_refs: architecture/implementation scaffolding (no feature refs)

PLAN1-BASE-02 Security Foundation (conditional)

Required if:

- auth.required == true OR security standards indicate baseline enforcement
  Scope refs include: roles/permissions model + auth constraints

PLAN1-BASE-03 Data Foundation (conditional)

Required if:

- data.enabled == true
  Scope refs include: dat_* entities and relationship model

PLAN1-BASE-04 Quality Foundation (always)

Required:

- verification scaffolding per quality standards (tests, lint, build checks)

PLAN1-BASE-05 Ops Foundation (conditional)

Required if:

- ops required by preset/scope (deployable system)
  Scope refs include: ops constraints + env expectations

6.2 Feature Slice Units (vertical slices)

For each **must-have feature** (feat_*), the planner must create at least one unit:

- `unit_feat_<feature_slug>_slice`
  Scope refs should include:
- feat_* (feature)
- wf_* (workflows supporting it)
- scr_* (screens supporting it, if UI)
- op_* (operations supporting it, if backend)
- dat_* (data objects, if data enabled)

Rule: must-have features cannot be left unassigned to units.

6.3 Workflow-first decomposition (allowed)

When spec is workflow-heavy, planner may create units by workflow:

- `unit_wf_<workflow_slug>_slice`
  Scope refs include workflow + mapped features/screens/ops

Rule: workflow-based units must still cover all must-have features.

6.4 Integration units (conditional)

If integrations exist, planner creates:

- `unit_integration_<service_slug>`
  Scope refs: integration + affected ops/data/features

6.5 Unknown constraints on decomposition

If there are **blocking unknowns** in core areas:

- planner must not create downstream dependent units that assume resolved unknowns
- planner may create a “resolution unit”:
  - `unit_resolve_unknowns_core`

    - scope_refs: unk_* items
    - deliverable: decisions/inputs required to unblock

---

7) Dependency Rules (How depends_on is assigned)

7.1 Foundational ordering (hard constraints)

- Setup precedes everything.
- Security foundation precedes any unit requiring auth/permissions enforcement.
- Data foundation precedes any unit that uses dat_* entities.
- API contract units (if modeled) precede dependent implementation units *(if you include API units as work units; optional)*.

7.2 Feature slice dependencies

A feature slice depends on:

- setup
- any needed foundations (security/data/ops)
- any prerequisite shared components (if modeled)

---

8) Unit Sizing Rules (anti-drift)

Units must be sized so that:

- a unit can be verified with acceptance checks
- a unit's deliverables are concrete and testable
- a unit does not span unrelated features

Heuristic bounds (non-binding but enforced as warnings unless you choose hard stops):

- max number of primary scope_refs per unit (e.g., 1–3 features/workflows)
- avoid units with >N screens and >N operations unless justified

---

9) Failure Modes

- units with no scope_refs (no traceability)
- dependency cycles
- features not mapped to units (missing work)
- units too broad (agent drift)
- units sequenced before prerequisites exist

---

10) Definition of Done (PLAN-01)

PLAN-01 is complete when:

- unit contract is locked (identity/scope/deps/deliverables)
- baseline unit rules exist and are context-driven
- feature/workflow slicing rules ensure coverage of must-have features
- dependency constraints are explicit and acyclic
- unknowns blocking policy is respected in decomposition

PLAN-02

PLAN-02 — Acceptance Map Rules (How Proof Is Attached to Units)

(Hardened Draft — Full)

1) Purpose

Define how Axion generates an **Acceptance Map** that attaches verifiable “done” criteria to every work unit, including:

- acceptance criteria statements
- required checks (build/test/lint/etc.)
- proof requirements (what evidence is needed)
- gating severity (hard stop vs soft)

This prevents “claims without evidence” and ensures units are objectively completable.

---

2) Inputs

- Work Breakdown (PLAN-01 output)
- Canonical Spec (for feature/workflow rules)
- Resolved Standards Snapshot (quality/security/ops constraints)
- Proof model (VER docs will define proof types; for now acceptance must reference allowed proof types)

---

3) Outputs

An Acceptance Map artifact containing:

- `acceptance_map_id`
- `work_breakdown_id`
- `spec_id`
- `acceptance_items[]`
- `unit_acceptance_index` (unit_id → acceptance_ids)
- `created_at`

---

4) Acceptance Invariants (must always be true)

1. **Coverage:** every work unit has acceptance items.
2. **Proof required for hard gates:** completion requires proof for all hard-gate acceptance items.
3. **Deterministic mapping:** same work breakdown + same standards ⇒ same acceptance structure (at least in count and categories).
4. **Traceability:** acceptance items link to unit_id and relevant spec IDs.
5. **Runnable verification:** acceptance items must specify how to verify (commands or manual steps).

---

5) Acceptance Item Contract (Locked Structure)

Each acceptance item must include:

5.1 Identity

- `acceptance_id` (stable; format `acc_<slug>`)
- `unit_id` (required)

5.2 Criterion

- `statement` (clear “done when…”)
- `criterion_type` (enum-like category)
  - `build_check`
  - `test_check`
  - `lint_check`
  - `functional_check`
  - `security_check`
  - `data_integrity_check`
  - `ui_check`
  - `performance_check`
  - `regression_check`
  - `manual_check`

5.3 Proof Requirement

- `proof_required` (boolean; default true for hard gates)
- `proof_type` (command_output | test_result | screenshot | log_excerpt | diff_ref | checklist)
- `proof_instructions` (how to capture/store)
- `proof_location_hint` (path/pointer key)

5.4 Verification Instructions

- `how_to_verify` (commands and/or steps)
- `expected_result` (what passing looks like)

5.5 Gating

- `gating` (hard_gate | soft_gate)

5.6 Scope References (required)

- `scope_refs[]` (spec IDs relevant to this criterion)

---

6) Acceptance Generation Rules

6.1 Minimum acceptance per unit (hard requirement)

Every unit must have at least:

- 1 **deliverable/functional** criterion (functional_check / ui_check / data_integrity_check)
- 1 **verification** criterion (build/test/lint or manual)
- 1 **proof** requirement attached to each hard_gate criterion

If a unit is purely planning/scaffolding, functional criterion may be replaced with:

- “artifact exists and matches contract” criterion (manual_check or functional_check with file verification)

6.2 Standards-driven checks (context required)

From Standards Snapshot:

- If quality standards require build/lint/tests:
  - include acceptance items for those checks on applicable units
- If security standards require auth/permission enforcement checks:
  - include security_check items on affected units
- If ops standards require deploy checks:
  - include relevant checks on ops-related units

6.3 Risk-based enrichment (required when flagged)

If a unit has risk_flags such as:

- `auth` → include security_check items (login, session handling, role restrictions)
- `data_migration` → include data_integrity_check + rollback considerations
- `external_integration` → include contract check + failure-mode behavior

6.4 UI units (if screens involved)

If a unit includes scr_* scope refs:

- include ui_check items for:
  - empty/loading/error states
  - role access gating
  - basic usability requirements
- if accessibility is required by standards:
  - include accessibility-related acceptance (can be `ui_check` or `manual_check` until VER defines specifics)

6.5 Unknown handling

If a unit depends on blocking unknowns:

- acceptance must include a blocking criterion:
  - “unknowns resolved and recorded” with proof (decision record / updated submission)

---

7) Completion Rules (unit done criteria)

A unit is complete only when:

- all hard_gate acceptance items are PASS
- all required proof artifacts are attached and linked

Soft gates may remain as warnings, but must be recorded.

---

8) Failure Modes

- units without acceptance criteria (agent improvisation)
- criteria vague (“works well”) (not verifiable)
- proof not required (claims without evidence)
- standards requirements omitted (quality drift)
- acceptance not linked to scope refs (no traceability)

---

9) Definition of Done (PLAN-02)

PLAN-02 is complete when:

- acceptance item contract is locked
- minimum acceptance rules per unit are explicit
- standards-driven check inclusion is explicit
- risk-based enrichment rules are explicit
- completion rules (hard_gate + proof) are explicit and enforceable
-

PLAN-03

PLAN-03 — Sequencing Heuristics (Foundational First, Vertical Slices, etc.)

(Hardened Draft — Full)

1) Purpose

Define the allowed sequencing heuristics the planner may use to order work units, while staying deterministic and aligned to dependencies.

PLAN-03 does not replace hard dependency rules (PLAN-01). It defines **tie-breakers and ordering principles** when multiple valid sequences exist.

---

2) Inputs

- Work Breakdown (units + dependency graph)
- Canonical Spec (feature priorities, workflows)
- Resolved Standards Snapshot (foundational constraints)
- Delivery preferences (priority bias: speed/balanced/quality)

---

3) Outputs

- An ordered `units[]` list in the Work Breakdown
- Optional: `sequencing_notes` (why an order was chosen)

---

4) Sequencing Invariants (must always be true)

1. **Dependencies first:** no unit can be scheduled before its prerequisites.
2. **Deterministic ties:** tie-breakers must be stable (no randomness).
3. **Early validation:** prefer sequences that produce early verifiable checkpoints.
4. **Risk isolation:** handle high-risk foundations before dependent feature work.

---

5) Sequencing Heuristics (Locked Set)

H-01 Foundations First (hard preference)

After setup, schedule foundational units early:

- security foundation (if auth/permissions)
- data foundation (if data.enabled)
- quality foundation (always)
- ops foundation (if required)

Purpose: avoid feature work built on unstable foundations.

H-02 Vertical Slices Over Horizontal Layers (default)

Prefer completing a small end-to-end feature slice (UI + API + data + checks) before building many unrelated layers.

Example:

- complete “Task CRUD” end-to-end before building “All screens” then “All endpoints.”

H-03 Follow Priority Rank for Must-Haves

Order feature-slice units by:

- must-have priority rank (from canonical spec)
  Tie-breakers:
- lower dependency fan-in first (simpler features earlier)
- higher user value first (if captured)

H-04 Reduce Rework (stabilize interfaces early)

If multiple units depend on shared contracts:

- schedule the contract-defining units early:
  - API contract docs (if present)
  - data model foundations
  - permission model definitions

H-05 Risk-Based Ordering

High-risk units should occur earlier when they unblock large work:

- auth/session implementation
- migration-heavy changes
- external integrations with unknowns

But not before prerequisites.

H-06 Optimize for Verification Cadence

Prefer sequences that regularly produce proof:

- early build/test/lint passing
- early smoke workflows
- early acceptance passes

H-07 Delivery Preference Modifier (speed vs quality)

Use `delivery.priority_bias` to modify tie-breakers only (never override dependencies):

- **Speed:** favor smallest verifiable slice first, defer non-critical hardening
- **Balanced:** standard ordering (foundations → vertical slices → polish)
- **Quality:** bring forward additional checks, hardening, and regression coverage earlier

---

6) Deterministic Tie-Breaker Order (Locked)

When two units are both eligible (all deps satisfied), order them by:

1. Unit category order:

    - setup → foundations → contract units → feature slices → integrations → polish/ops
2. Must-have priority rank (if unit maps to a must-have feature)
3. Risk flag priority (auth/data_migration/integration higher first)
4. Smaller scope_refs count first (simpler first)
5. Lexical order of unit_id (final deterministic tie-breaker)

---

7) Failure Modes

- planner invents a custom heuristic per project (non-deterministic)
- tie-breakers depend on agent mood/context (drift)
- speed bias causes skipping required foundations (violates PLAN-01 hard constraints)
- ordering produces long stretches without verification (late failure discovery)

---

8) Definition of Done (PLAN-03)

PLAN-03 is complete when:

- allowed heuristic set is locked
- dependency precedence is explicit
- tie-breaker order is deterministic and explicit
- delivery preference only modifies tie-breakers, not dependencies

Verification & Proof System

Verification & Proof System

- VER-01 Proof Types & Evidence Rules (what counts as proof)
- VER-02 Verification Command Policy (when/how to run checks)
- VER-03 Completion Criteria (global “done” rules)

VER-01

VER-01 — Proof Types & Evidence Rules (What Counts as Proof)

(Hardened Draft — Full)

1) Purpose

Define what qualifies as **proof** in Axion so completion is based on evidence, not statements. This establishes the proof taxonomy, evidence requirements, and disallowed evidence.

This doc governs:

- what proof types exist
- what minimum metadata must be recorded
- what is acceptable vs unacceptable proof
- how proof links to acceptance criteria and work units

---

2) Inputs

- Acceptance Map contract (PLAN-02) for proof requirements per acceptance item
- Traceability model (SYS-06) for linking proof → acceptance → unit → spec
- Gate model (SYS-07) for completion gating

---

3) Outputs

- A locked proof taxonomy and evidence rules used by:
  - Proof Log
  - Completion Gate (VER-03)
  - Any verification automation

---

4) Proof Invariants (must always be true)

1. **Proof is linked:** every proof must link to an `acceptance_id` and `unit_id`.
2. **Proof is reproducible:** proof must include enough context to rerun or re-check.
3. **Proof is bounded:** large outputs must be referenced by stored file pointer, not pasted endlessly.

4. **Proof is auditable:** proof must include timestamp and source context (command, file, screenshot, etc.).
5. **Proof is not narrative:** “I tested it” is never proof.

---

5) Proof Types (Locked Set)

P-01 Command Output Proof

Evidence from running a command (build, lint, tests, scripts).
Required fields:

- command string
- working directory
- exit code
- timestamp
- stdout/stderr (or file refs)

P-02 Test Result Proof

Evidence from a test runner result (unit/integration/e2e).
Required fields:

- test command / runner
- pass/fail summary
- report location (file ref)
- timestamp

P-03 Screenshot / UI Capture Proof

Evidence showing UI state or behavior.
Required fields:

- screenshot file ref
- what is being proven (brief)
- timestamp
- context (screen name, flow step)

P-04 Log Excerpt Proof

Evidence from application logs.
Required fields:

- log source (file/system)
- excerpt pointer or file ref
- timestamp
- correlation (what acceptance it supports)

P-05 Diff/Commit Reference Proof

Evidence of changes via diff/commit identifiers (when relevant).
Required fields:

- diff/commit reference
- files changed list (or pointer)
- timestamp
- what it proves (tie to acceptance)

P-06 Checklist Proof (Manual Verification)

Evidence from a structured checklist with pass/fail items.
Required fields:

- checklist ID or name
- itemized results
- reviewer identity (agent or human)
- timestamp

Rule: Manual checklist proof is allowed but should be minimized for core build correctness.

---

6) Proof Record Contract (Minimum Metadata)

Every proof entry must contain:

- `proof_id` (stable)
- `acceptance_id`
- `unit_id`
- `proof_type` (P-01..P-06)

- `location` (file path or storage ref)
- `summary` (one sentence)
- `created_at` (timestamp)
- `source_context` (object, depends on proof type)
    - command context, screenshot context, etc.

---

7) Evidence Quality Rules (What makes proof valid)

7.1 Must be directly relevant

Proof must support the exact acceptance statement, not a loosely related claim.

7.2 Must be current

Proof must be generated for the current build context/version. If the system version or relevant artifacts changed, old proof may be invalid.

7.3 Must show pass/fail objectively

- commands must include exit code
- tests must include pass/fail summary
- screenshots must correspond to the stated screen/state

7.4 Must be accessible

Proof location must be reachable from the kit/workspace environment; broken refs invalidate proof.

---

8) Disallowed “Proof” (Explicitly not acceptable)

These do **not** count as proof:

- narrative statements (“works now”, “should be fine”)
- inferred reasoning (“it must pass because…”)
- partial command output with no exit code
- screenshots with no context (what it shows/why it matters)
- references to chat logs as evidence

---

9) Failure Modes

- proofs not linked to acceptance_id (un-auditable)
- proofs stored only in chat (lost, non-deterministic)
- outputs too large to parse (need file refs)
- manual proofs replacing objective checks for core items

---

10) Definition of Done (VER-01)

VER-01 is complete when:

- proof types are locked and non-overlapping
- minimum proof record metadata is defined
- validity rules and disallowed evidence are explicit
- linkage requirements align with PLAN-02 and SYS-06

VER-02

VER-02 — Verification Command Policy (When/How to Run Checks)

(Hardened Draft — Full)

1) Purpose

Define the strict policy for running verification checks and capturing proof, including:

- when checks must be run
- which checks are mandatory vs conditional
- how command output is captured and stored
- how failures are handled (stop vs continue)
- how verification maps to acceptance items

This prevents “I’ll run tests later” drift and standardizes evidence collection.

---

2) Inputs

- Acceptance Map (PLAN-02): required checks + how_to_verify + proof requirements
- Resolved Standards Snapshot (STD-03): mandated checks (build/lint/test/perf)
- Proof Types (VER-01)
- Work Breakdown (unit boundaries)

---

3) Outputs

- Verification events (command runs) that create:
  - proof artifacts (VER-01)
  - acceptance status updates
  - state snapshot updates

---

4) Command Policy Invariants (must always be true)

1. **Checks are tied to acceptance:** every required verification command must map to an acceptance_id.
2. **Fail-fast on hard gates:** failures on hard-gate checks block unit completion.

3. **Capture is mandatory:** command, cwd, exit code, and stdout/stderr pointers are required for proof.
4. **No silent retries:** retries must be recorded as separate attempts with timestamps.
5. **Deterministic environment:** record enough context to reproduce (cwd, environment label).

---

5) When to Run Verification (Locked Rules)

V2-RUN-01 Per-unit completion (mandatory)

Before marking a unit complete:

- run all verification commands associated with that unit's hard-gate acceptance items
- capture proof artifacts
- update acceptance status

V2-RUN-02 After foundational units (mandatory)

After setup/security/data/quality foundation units:

- run baseline checks required by standards snapshot (build/lint/tests as applicable)

V2-RUN-03 Before packaging completion claim (mandatory)

Before claiming "project complete":

- run full required verification suite specified by standards snapshot and acceptance map
- produce final proof references for completion criteria

V2-RUN-04 On changes affecting many units (conditional)

If a change touches core foundations (auth, data model, routing, shared components):

- run at least regression checks that cover affected scopes (as defined in QA templates / acceptance map)

---

6) Verification Command Categories (Locked)

Commands are categorized as:

- `build`
- `lint`
- `unit_tests`
- `integration_tests`
- `e2e_tests`
- `typecheck`
- `security_checks` *(if defined)*
- `performance_checks` *(if defined)*

Rule: Each acceptance item that requires commands must specify category and exact command string(s).

---

7) Command Capture Rules (Proof discipline)

For every command run, record:

- `command_id` (unique)
- `acceptance_id` (required link)
- `unit_id` (required link)
- `category`
- `command` (full string)
- `cwd`
- `env_label` (optional but recommended: dev/stage/prod)
- `started_at`, `finished_at`
- `exit_code`
- `stdout_ref` (file pointer or bounded inline)
- `stderr_ref` (file pointer or bounded inline)

Bounded inline rule: if inline capture is used, cap length and store full output as a file ref.

---

8) Failure Handling Policy (Locked)

V2-FAIL-01 Hard-gate failure

If a command tied to a hard-gate acceptance item fails:

- mark acceptance status FAIL
- mark unit status BLOCKED
- record blocker with pointer to acceptance_id and proof refs
- unit cannot be completed until resolved and re-verified

V2-FAIL-02 Soft-gate failure

If a soft-gate check fails:

- mark acceptance status FAIL
- unit may proceed only if policy allows “pass with warnings”
- must be recorded in state snapshot warnings

V2-FAIL-03 Flaky checks

If a check is flaky:

- retries are allowed but must be recorded as separate proof attempts
- a unit cannot be marked complete without a PASS proof for hard-gate checks

---

9) Mapping Commands to Acceptance Items (Required)

Every verification command must belong to a specific acceptance item.
If a command is “global” (e.g., full test suite), it still must map to:

- a completion acceptance item (global), or

- a unit-level acceptance item defined for that purpose

No “free-floating commands” without acceptance linkage.

---

10) Failure Modes

- commands run without linking to acceptance_id (un-auditable)
- pass/fail claimed without exit code evidence
- unit marked complete with failing hard-gates
- retries overwrite evidence (should be separate attempts)

---

11) Definition of Done (VER-02)

VER-02 is complete when:

- run timing rules are explicit (per unit, foundations, completion)
- command categories are locked
- capture contract is explicit and produces VER-01 proof artifacts
- failure handling is explicit (hard vs soft gating)
- command-to-acceptance linkage is mandatory and enforceable

VER-03

VER-03 — Completion Criteria (Global “Done” Rules)

(Hardened Draft — Full)

1) Purpose

Define the system-wide rules for declaring:

- a **work unit complete**
- the **overall project complete**
  using acceptance criteria and proof artifacts—not claims.

VER-03 is the final authority for “done.”

---

2) Inputs

- Work Breakdown (units + dependencies)
- Acceptance Map (acceptance items + gating)
- Proof Log (proof artifacts linked to acceptance)
- State Snapshot (unit/acceptance status tracking)
- Standards Snapshot (global required checks)

---

3) Outputs

- Deterministic completion status:
  - unit completion status
  - overall completion status
- A completion report structure (high-level; detailed template may exist elsewhere)

---

4) Completion Invariants (must always be true)

1. **No proof, no done:** completion claims require proof IDs.
2. **Hard gates are absolute:** all hard-gate acceptance items must pass.
3. **Dependency-respecting:** a unit cannot be complete unless its dependencies are complete (unless a recorded override exists).
4. **State is authoritative for progress:** completion status must be reflected in State Snapshot.
5. **Auditability:** completion must be reconstructable from artifacts (SYS-06).

---

5) Unit Completion Rules (Locked)

VER3-UNIT-01 Hard-gate pass required

A unit is complete only if:

- all acceptance items with `gating == hard_gate` for that unit are `PASS`
- each such acceptance item has required proof artifacts attached

VER3-UNIT-02 Proof linkage required

For each hard-gate acceptance item:

- at least one proof_id is recorded
- proof_id resolves to a proof record
- proof record references same unit_id and acceptance_id

VER3-UNIT-03 Failure blocks completion

If any hard-gate acceptance item is FAIL or NOT_RUN:

- unit status must be `blocked` or `in_progress` (not `done`)

VER3-UNIT-04 Soft-gates policy

Soft-gate acceptance items:

- may remain FAIL only if global policy allows “complete with warnings”
- all soft failures must be listed in completion report and state snapshot warnings

---

6) Overall Project Completion Rules (Locked)

VER3-PROJ-01 All required units done

Project completion requires:

- every unit marked as required for scope is DONE
- no required unit remains not_started/in_progress/blocked

VER3-PROJ-02 Global required checks satisfied

If standards snapshot mandates global checks (build/lint/tests/perf), then:

- there must exist proof artifacts for those checks
- mapped to a global completion acceptance item (or final unit)

VER3-PROJ-03 No blocking unknowns

Project cannot be complete if:

- any `unknown` with `blocking == true` remains `open`
  unless an override policy explicitly permits it (rare and must be recorded)

VER3-PROJ-04 No failed hard gates anywhere

If any hard-gate acceptance item across any unit is FAIL:

- project is not complete

---

7) Completion Report Contract (Minimum)

A completion report must include:

- `project_status` (complete|incomplete)
- `completed_at` (if complete)
- `spec_id`, `work_breakdown_id`, `acceptance_map_id`
- `units_summary`:
  - total units
  - completed units
  - blocked units
- `hard_gate_failures[]` (if any)

- `soft_gate_warnings[]` (if any)
- `proof_index`:
    - list of proof_ids used for final completion
- `open_blocking_unknowns[]` (should be empty if complete)

Rule: Any “complete” report without proof_ids is invalid.

---

8) Overrides and Exceptions (Strict)

Overrides that allow bypassing completion conditions are:

- strongly discouraged
- must be explicitly recorded with:
    - what was bypassed
    - why
    - risk acknowledged
    - who approved
    - downstream impacts

Default policy: no overrides for completion hard gates.

---

9) Failure Modes

- unit marked done without proof IDs
- unit done while dependencies incomplete
- project marked complete while blocking unknowns exist
- global checks never run but completion claimed

---

10) Definition of Done (VER-03)

VER-03 is complete when:

- unit completion rules are explicit and proof-based
- project completion rules are explicit and proof-based

- completion report minimum contract is defined
- unknowns and overrides policies are explicitly integrated

Packaging & Kit Format

Packaging & Kit Format

- KIT-01 Kit Folder Structure Contract
- KIT-02 Manifest/Index Format (IDs → paths, reading order)
- KIT-03 Entrypoint Contract (single start file rules)
- KIT-04 Version Stamping Rules (what versions get recorded)

KIT-01

KIT-01 — Kit Folder Structure Contract

(Hardened Draft — Full)

1) Purpose

Define the **locked folder/file structure** of the Agent Kit so:

- every kit is predictable and machine-navigable
- the external agent always knows where to find truth artifacts, filled docs, and proof/state
- packaging can be validated via a deterministic gate (no silent omissions)
- the kit remains stable across versions (changes require version stamping + compatibility notes)

---

2) Inputs

- Kit file tree standard (locked)
- Core artifacts (A2): normalized input, standards snapshot, canonical spec, work breakdown, acceptance map, state snapshot
- Filled templates (TMP outputs)
- Pack hierarchy model (App → Domain → Feature → Screen → Component)
- Traceability requirements (SYS-06)

---

3) Outputs

A kit folder that:

- conforms to the structure below
- includes required root files
- includes required core artifacts
- includes filled docs in standardized placement
- includes pack folders (domain/feature/screen/component) as needed
- includes N/A stubs where applicable (no silent absence)

---

4) Locked Kit Root Structure

agent_kit/

```
├── 00_START_HERE.md
├── 00_KIT_MANIFEST.md
├── 00_KIT_INDEX.md
├── 00_VERSIONS.md
├── 00_RUN_RULES.md
├── 00_PROOF_LOG.md
│
├── 01_core_artifacts/
│   ├── 01_normalized_input_record.json
│   ├── 02_resolved_standards_snapshot.json
│   ├── 03_canonical_spec.json
│   ├── 04_work_breakdown.json
│   ├── 05_acceptance_map.json
│   └── 06_state_snapshot.json
│
├── 10_app/
│   ├── 00_pack_meta.md
│   ├── 00_pack_index.md
│   ├── 00_gate_checklist.md
│   ├── 01_requirements/
│   ├── 02_design/
│   ├── 03_architecture/
│   ├── 04_implementation/
│   ├── 05_security/
│   ├── 06_quality/
│   ├── 07_ops/
│   ├── 08_data/
│   ├── 09_api_contracts/
│   ├── 10_release/
│   ├── 11_governance/
│   └── 12_analytics/
│
├── 20_domains/
│   └── <domain_slug>/
│       ├── 00_pack_meta.md
│       ├── 00_pack_index.md
│       ├── 00_gate_checklist.md
│       └── <slot folders...>
│
├── 30_features/
│   └── feat_<feature_slug>/
│       ├── 00_pack_meta.md
│       ├── 00_pack_index.md
│       ├── 00_gate_checklist.md
```

```
│       └── <slot folders...>
│
├── 40_screens/
│   └── scr_<screen_slug>/
│       ├── 00_pack_meta.md
│       ├── 00_pack_index.md
│       ├── 00_gate_checklist.md
│       └── <slot folders...>
│
└── 50_components/
    └── cmp_<component_slug>/
        ├── 00_pack_meta.md
        ├── 00_pack_index.md
        ├── 00_gate_checklist.md
        └── <slot folders...>
```

---

5) Required Root Files (Locked)

5.1 `00_START_HERE.md` (required)

- single entrypoint for external agent
- must reference:
  - manifest/index
  - canonical spec
  - work breakdown + acceptance map
  - proof log + state snapshot

5.2 `00_KIT_MANIFEST.md` (required)

- IDs → paths mapping and kit metadata (KIT-02 defines format)

5.3 `00_KIT_INDEX.md` (required)

- human-readable table of contents (mirrors manifest)

5.4 `00_VERSIONS.md` (required)

- pinned versions (KIT-04 defines format)

5.5 `00_RUN_RULES.md` (required)

- minimal execution rules:
  - “no claims without proof”
  - update state snapshot
  - follow work breakdown order unless blocked

5.6 `00_PROOF_LOG.md` (required)

- proof record surface (VER-01 contract)

---

6) Core Artifacts Folder (01_core_artifacts) — Required Contents

These files are required in every kit (no exceptions):

- `01_normalized_input_record.json`
- `02_resolved_standards_snapshot.json`
- `03_canonical_spec.json`
- `04_work_breakdown.json`
- `05_acceptance_map.json`
- `06_state_snapshot.json`

Rule: missing any core artifact is a Packaging Gate failure.

---

7) Pack Folder Contract (Applies to 10_app and all pack folders)

7.1 Pack Root Files (required)

Every pack folder must include:

- `00_pack_meta.md`
- `00_pack_index.md`
- `00_gate_checklist.md`

7.2 Slot Folder Set (required to exist)

Every pack must include the full slot folder set:

- `01_requirements/`
- `02_design/`
- `03_architecture/`
- `04_implementation/`
- `05_security/`
- `06_quality/`
- `07_ops/`
- `08_data/`
- `09_api_contracts/`
- `10_release/`
- `11_governance/`
- `12_analytics/`

7.3 N/A Rule (no silent omission)

If a slot has no applicable filled templates, it must contain:

- `00_NA.md` with reason and trigger condition

---

8) Naming Rules (Locked)

- Domain pack folder: `20_domains/<domain_slug>/`
- Feature pack folder: `30_features/feat_<feature_slug>/`
- Screen pack folder: `40_screens/scr_<screen_slug>/`
- Component pack folder: `50_components/cmp_<component_slug>/`

Slug rules:

- lowercase, underscore
- deterministic from canonical IDs/names
- stable across regeneration unless entity is new

---

9) Mutability Rules (for drift control)

- Core artifacts are **append-only** only where specified:
  - `06_state_snapshot.json` updates over time
  - `00_PROOF_LOG.md` grows over time

- Filled templates are **immutable once template gate passes** *(policy can be enforced by packaging/gate tooling)*

---

10) Failure Modes

- inconsistent kit trees cause external agent drift
- missing core artifacts causes guessing
- silent omission of slot docs hides missing constraints
- pack folders without meta/index/gates break navigation and auditing

---

11) Definition of Done (KIT-01)

KIT-01 is complete when:

- root file set is locked and required
- core artifacts folder required contents are locked
- pack folder contract (root files + slot folders + N/A rule) is locked
- naming rules for domain/feature/screen/component packs are locked
- mutability expectations for state/proof are explicit and compatible with VER/STATE docs

KIT-02

KIT-02 — Manifest/Index Format (IDs → Paths, Reading Order)

(Hardened Draft — Full)

1) Purpose

Define the **Kit Manifest and Index contracts** so every kit can be navigated deterministically by:

- machines (agents, validators, packagers)
- humans (you auditing or reading)

KIT-02 defines:

- the machine-readable manifest format (authoritative IDs → paths)
- the human-readable index format (mirrors manifest)
- reading order rules (what to read first, then next)

---

2) Inputs

- Kit folder structure (KIT-01)
- Core artifact IDs (submission_id/spec_id/work_breakdown_id/acceptance_map_id)
- Template Selection Result (TMP-03)
- Filled template outputs + their output paths
- Pack folders and their meta/index/gate files

---

3) Outputs

Two required root files:

1. `00_KIT_MANIFEST.md` *(machine-readable block required)*
2. `00_KIT_INDEX.md` *(human-friendly table of contents)*

Both must be consistent. If inconsistent → Packaging Gate fails.

---

4) Manifest Contract (Authoritative)

4.1 File: `00_KIT_MANIFEST.md`

Must contain two parts in this order:

A) Manifest Header (required)

- `kit_id`
- `created_at`
- `submission_id`
- `spec_id`
- `work_breakdown_id`
- `acceptance_map_id`
- `state_id`
- `schema_version_used`
- `standards_version_used`
- `template_library_version_used`
- `system_version`

B) Manifest Body (required; machine-readable)

A fenced JSON block named `manifest`:

```
{
  "manifest_version": "v1",
  "reading_order": ["00_START_HERE.md", "..."],
  "core_artifacts": {
    "normalized_input": "01_core_artifacts/01_normalized_input_record.json",
    "standards_snapshot": "01_core_artifacts/02_resolved_standards_snapshot.json",
    "canonical_spec": "01_core_artifacts/03_canonical_spec.json",
    "work_breakdown": "01_core_artifacts/04_work_breakdown.json",
    "acceptance_map": "01_core_artifacts/05_acceptance_map.json",
    "state_snapshot": "01_core_artifacts/06_state_snapshot.json"
  },
  "packs": {
    "app": "10_app/",
    "domains": ["20_domains/frontend/", "20_domains/backend/"],
    "features": ["30_features/feat_task_crud/"],
    "screens": ["40_screens/scr_task_list/"],
    "components": ["50_components/cmp_task_card/"]
  },
  "docs": [
```

```
    { "doc_id": "PRD-01", "path":
"10_app/01_requirements/PRD-01_Product_Requirements_Document.md" },
    { "doc_id": "DES-04", "path": "10_app/02_design/DES-04_Screen_Layout_Specs.md" }
  ],
  "proof": {
    "proof_log": "00_PROOF_LOG.md"
  },
  "run_rules": "00_RUN_RULES.md",
  "versions": "00_VERSIONS.md"
}
```

Rule: The JSON block is the authoritative mapping; the markdown around it is commentary.

4.2 Required Manifest Fields (locked)

- `manifest_version`
- `reading_order[]`
- `core_artifacts` map (must include all required core artifacts)
- `packs` lists (may be empty lists but keys must exist)
- `docs[]` list with `doc_id` and `path`
- `proof.proof_log`
- `run_rules`
- `versions`

4.3 Reading Order Rules (locked)

`reading_order[]` must always begin with:

1. `00_START_HERE.md`
2. `00_KIT_MANIFEST.md`
3. `01_core_artifacts/02_resolved_standards_snapshot.json`
4. `01_core_artifacts/03_canonical_spec.json`
5. `01_core_artifacts/04_work_breakdown.json`
6. `01_core_artifacts/05_acceptance_map.json`
7. `01_core_artifacts/06_state_snapshot.json`

After core artifacts, the order may list:

- app-level filled docs in template type order (Product → Design → Architecture → …)
- then domain packs
- then feature packs
- then screen packs
- then component packs

Rule: reading_order must be deterministic and must not omit core artifacts.

4.4 Doc ID Rules

- `doc_id` must match Template Index `template_id` for filled templates.
- Pack root docs may use reserved IDs:
  - `PACK_META`, `PACK_INDEX`, `PACK_GATE`

---

5) Index Contract (Human-Friendly)

5.1 File: `00_KIT_INDEX.md`

Must include:

A) Quick Start

- link to `00_START_HERE.md`
- link to core artifacts

B) Core Artifacts Links

List the six core artifacts with paths.

C) Filled Docs Table of Contents

Organize by:

- App-level docs (by template type)
- Domain packs
- Feature packs

- Screen packs
- Component packs

Rule: The index must not list anything that isn't in the manifest JSON, and must list everything the manifest includes (at least as a link group).

---

6) Consistency Rules (Manifest ↔ Index)

KIT2-CON-01

Every `docs[].path` in manifest must appear in index.

KIT2-CON-02

Every link listed in index must exist in manifest (or be a pack folder path listed under `packs`).

KIT2-CON-03

All referenced paths must exist in the kit file tree.

Violations → Packaging Gate failure.

---

7) Failure Modes

- missing reading order causes external agent confusion
- index differs from manifest (two truths)
- doc_id doesn't match template_id (breaks automation)
- paths don't exist (broken kit)

---

8) Definition of Done (KIT-02)

KIT-02 is complete when:

- manifest header fields are defined

- manifest JSON contract is fully specified
- reading order rules are locked
- index structure is defined and mirrors manifest
- consistency rules are explicit and gateable

KIT-03

KIT-03 — Entrypoint Contract (Single Start File Rules)

(Hardened Draft — Full)

1) Purpose

Define the single external-agent entrypoint file (`00_START_HERE.md`) so every kit can be executed consistently without the external agent guessing:

- what to read first
- what artifacts are authoritative
- what order to work in
- how to record progress and proof
- what is prohibited (claims without proof, skipping gates)

---

2) Inputs

- Kit structure (KIT-01)
- Manifest/index contract (KIT-02)
- Core artifacts (canonical spec, work breakdown, acceptance map, state snapshot, standards snapshot)
- Proof rules (VER-01/02/03)
- Boundaries (SYS-10), especially no guessing / no proofless claims

---

3) Outputs

A single file: `00_START_HERE.md` that:

- is complete and self-contained as instructions
- references all authoritative artifacts by path
- defines a deterministic execution loop

---

4) Entrypoint Invariants (must always be true)

1. **Single start:** external agent must start here; no alternate entrypoints.
2. **Artifact authority:** must explicitly declare which artifacts are truth.
3. **Execution loop:** must define “read → select unit → implement → verify → record proof → update state.”
4. **No skipping:** must prohibit skipping required steps without recorded override/decision.
5. **No proof, no done:** must require proof IDs for completion claims.

---

5) Required Sections (Locked Order)

`00_START_HERE.md` must contain these sections in order:

1. **Purpose & How to Use This Kit**
2. **Authoritative Artifacts (Truth Sources)**
3. **Reading Order (Minimal Required Reading)**
4. **Execution Loop (Do This Repeatedly)**
5. **How to Verify (Commands + Proof Rules)**
6. **How to Record Progress (State Snapshot + Proof Log)**
7. **What Not To Do (Hard Prohibitions)**
8. **When Blocked (What to Output)**
9. **Completion Definition (What “Done” Requires)**
10. **Pointers (Links to Key Files)**

Missing any section → Packaging Gate fail.

---

6) Required Content (Per Section)

6.1 Purpose & How to Use This Kit

Must state:

- this kit is execution-ready instructions
- the agent must follow work breakdown + acceptance map

6.2 Authoritative Artifacts

Must list and link:

- `01_core_artifacts/02_resolved_standards_snapshot.json`
- `01_core_artifacts/03_canonical_spec.json`
- `01_core_artifacts/04_work_breakdown.json`
- `01_core_artifacts/05_acceptance_map.json`
- `01_core_artifacts/06_state_snapshot.json`

Must explicitly say:

- chat memory is not authoritative
- only these artifacts define truth

6.3 Reading Order (minimal)

Must include:

- `00_KIT_MANIFEST.md`
- the core artifacts above
- the app-level implementation guidance docs (or where to find them)

6.4 Execution Loop

Must define the loop steps:

1. Read `06_state_snapshot.json` to find current unit
2. If no current unit, select next eligible unit from `04_work_breakdown.json`
3. Read unit scope + acceptance criteria
4. Implement unit deliverables
5. Run required verification commands
6. Attach proof to proof log and acceptance status
7. Update state snapshot (unit status, acceptance status, blockers)
8. Repeat

6.5 How to Verify

Must instruct:

- run verification commands specified by Acceptance Map
- capture proof per VER-01/02
- fail-fast on hard-gate failures

6.6 How to Record Progress

Must require updates to:

- `00_PROOF_LOG.md` (proof entries)
- `01_core_artifacts/06_state_snapshot.json` (progress)
  And specify:
- unit cannot be marked done without proof refs

6.7 What Not To Do (Hard Prohibitions)

Must list at minimum:

- do not invent requirements
- do not skip acceptance checks
- do not mark done without proof
- do not change standards/constraints unless explicitly allowed and recorded

6.8 When Blocked

Must instruct:

- mark unit blocked in state snapshot
- record blocker with acceptance_id pointers
- state what input/decision is needed

6.9 Completion Definition

Must reference VER-03 completion rules:

- all required units done
- all hard-gate acceptance pass with proof
- no blocking unknowns

6.10 Pointers

Must provide direct links to:

- manifest
- key artifacts
- app docs folder
- domain/feature/screen packs (if present)

---

7) Failure Modes

- entrypoint too vague → agent improvises
- truth sources not explicit → agent uses chat memory
- no execution loop → agent drifts
- no recording rules → progress lost and continuity breaks

---

8) Definition of Done (KIT-03)

KIT-03 is complete when:

- required sections are locked and complete
- authoritative artifacts are explicitly listed and linked
- execution loop is explicit and references work breakdown + acceptance map + state snapshot
- proof and progress recording rules align with VER-01/02/03
- prohibitions align with SYS-10 boundaries

KIT-04

KIT-04 — Version Stamping Rules (What Versions Get Recorded)

(Hardened Draft — Full)

1) Purpose

Define the mandatory version stamps recorded in every kit so:

- kits are reproducible (determinism)
- audits can identify exactly which rules/templates/contracts were used
- compatibility can be managed across system upgrades
- “silent change” drift is prevented

---

2) Inputs

- Submission Record metadata (INT-04): form/schema versions
- Standards Snapshot (STD-03): standards library version + pack versions
- Template Index (TMP-01): template library version + template versions used
- System version (overall Axion version)
- Kit contract version (KIT-01/KIT-02/KIT-03 contracts)

---

3) Outputs

A required root file: `00_VERSIONS.md` containing:

- a machine-readable version block
- a human-readable summary

The kit manifest (KIT-02) must also repeat key version fields for cross-checking.

---

4) Version Stamps (Locked Minimum Set)

Every kit must record these version categories:

V-01 System Version

- `system_version` (Axion core rules/docs/contracts version)

V-02 Intake Versions

- `form_version_used`
- `schema_version_used`
- `int_ruleset_version_used` *(INT-03 version/hash)*

V-03 Standards Versions

- `standards_library_version_used`
- `standards_packs_used[]` (pack_id@pack_version list)
- `standards_resolver_version_used` *(STD-02 version/hash)*

V-04 Template Versions

- `template_library_version_used`
- `templates_used[]` (template_id@template_version list)
- `template_index_version_used` *(TMP-01 index schema version)*
- `template_fill_rules_version_used` *(TMP-04 version/hash)*

V-05 Canonical Model Versions

- `canonical_spec_model_version` *(CAN-01/02/03 version set or a single model version)*
- `id_rules_version_used` *(CAN-02 version/hash)*
- `unknowns_model_version_used` *(CAN-03 version/hash)*

V-06 Planning & Verification Versions

- `planning_rules_version_used` *(PLAN-01/02/03 version/hash)*
- `proof_rules_version_used` *(VER-01/02/03 version/hash)*

V-07 Kit Contract Versions

- `kit_folder_structure_version` *(KIT-01 version)*

- `manifest_format_version` *(KIT-02 manifest_version)*
- `entrypoint_contract_version` *(KIT-03 version)*

---

5) `00_VERSIONS.md` File Contract (Locked)

`00_VERSIONS.md` must contain two parts:

A) Machine-readable block

A fenced JSON block named `versions`:

```
{
  "versions_format": "v1",
  "system_version": "Axion-v2.0.0",
  "intake": {
    "form_version_used": "form-vX",
    "schema_version_used": "schema-vY",
    "int_ruleset_version_used": "int03-vZ"
  },
  "standards": {
    "standards_library_version_used": "std-lib-vA",
    "standards_resolver_version_used": "std02-vB",
    "standards_packs_used": ["pack1@1.2.0", "pack2@3.0.1"]
  },
  "templates": {
    "template_library_version_used": "tmp-lib-vC",
    "template_index_version_used": "tmp-index-vD",
    "template_fill_rules_version_used": "tmp04-vE",
    "templates_used": ["PRD-01@1.0", "DES-04@1.1"]
  },
  "canonical_model": {
    "canonical_spec_model_version": "can-model-vF",
    "id_rules_version_used": "can02-vG",
    "unknowns_model_version_used": "can03-vH"
  },
  "planning_verification": {
    "planning_rules_version_used": "plan-vI",
    "proof_rules_version_used": "ver-vJ"
```

```
  },
  "kit_contracts": {
    "kit_folder_structure_version": "kit01-vK",
    "manifest_format_version": "v1",
    "entrypoint_contract_version": "kit03-vL"
  }
}
```


B) Human-readable summary

A short summary of the above:

- “This kit was generated with…”
- list of key versions

---

6) Consistency Rules (must always be true)

KIT4-CON-01 Manifest match

Key versions in `00_KIT_MANIFEST.md` must match `00_VERSIONS.md` (system/schema/standards/template versions).

KIT4-CON-02 Snapshot match

- `standards_library_version_used` must match the one inside the standards snapshot.
- templates_used must match what exists in the kit doc folders.

KIT4-CON-03 No missing stamps

If any required version stamp is missing → Packaging Gate fail.

---

7) Failure Modes

- kits can’t be reproduced (“why did output change?”)
- older kits become uninterpretable without knowing contracts
- silent template/standards changes break determinism
- manifest and versions disagree (two truths)

---

8) Definition of Done (KIT-04)

KIT-04 is complete when:

- the minimum version set is locked
- `00_VERSIONS.md` machine-readable format is defined
- consistency rules are explicit and gateable
- required cross-checks with manifest and snapshot are defined

Continuity & State

Continuity & State

- STATE-01 State Snapshot Format
- STATE-02 Resume Rules (how to restart without drift)
- STATE-03 Handoff Rules (agent-to-agent continuity)

STATE-01

STATE-01 — State Snapshot Format

(Hardened Draft — Full)

1) Purpose

Define the **State Snapshot** as the authoritative, resumable progress record so an agent can stop and restart without relying on chat memory. The State Snapshot is the continuity anchor that prevents drift by recording:

- which unit is current
- what's complete vs blocked
- acceptance pass/fail status
- proof references
- blockers and required decisions
- last-known-good checkpoint

---

2) Inputs

- Work Breakdown (units + dependencies)
- Acceptance Map (acceptance items per unit + gating)
- Proof Log (proof artifacts linked to acceptance items)
- Unknowns (CAN-03) and their resolution status
- Gate reports (optional but recommended) for audit

---

3) Outputs

A single machine-readable file in the kit:

- `01_core_artifacts/06_state_snapshot.json`

This file is updated over time.

---

4) State Invariants (must always be true)

1. **Single source for progress:** State Snapshot is authoritative for progress, not chat.
2. **Completion requires proof:** unit completion status must align with acceptance/proof linkage (VER-03).

3. **Deterministic pointers:** state must reference units/acceptance/proof by IDs.
4. **Append-safe:** updates must not erase history; use event log or history pointers.
5. **Resume-ready:** snapshot must include enough context to select next unit without re-planning.

---

5) State Snapshot Contract (Top-Level Structure)

```
state_snapshot = {
  meta: {...},
  pointers: {...},
  unit_status: [...],
  acceptance_status: [...],
  proofs_index: [...],
  blockers: [...],
  decisions: [...],
  overrides: [...],
  unknowns_status: [...],
  last_known_good: {...},
  history: {...}
}
```

5.1 meta (required)

- `state_id` (string)
- `submission_id`
- `spec_id`
- `work_breakdown_id`
- `acceptance_map_id`
- `created_at`
- `updated_at`
- `state_format_version`

5.2 pointers (required)

- `status` (enum): in_progress | blocked | complete

- `current_unit_id` (string|null)
- `next_unit_candidates[]` (optional; list of unit_ids)
- `blocked_on` (optional; list of blocker_ids)

---

6) Unit Progress Tracking

6.1 unit_status[] (required)

Each entry:

- `unit_id`
- `status` (enum): not_started | in_progress | blocked | done
- `started_at` (optional)
- `completed_at` (optional)
- `notes` (optional)
- `blocker_refs[]` (optional)
- `acceptance_refs[]` (optional; cached from acceptance map for convenience)

Invariant: `done` requires VER-03 conditions.

---

7) Acceptance Tracking

7.1 acceptance_status[] (required)

Each entry:

- `acceptance_id`
- `unit_id`
- `status` (enum): not_run | pass | fail
- `last_run_at` (optional)
- `proof_refs[]` (list of proof_ids)
- `attempt_count` (optional)
- `last_failure_summary` (optional)

---

8) Proof Index

8.1 proofs_index[] (optional but recommended)

Each entry:

- `proof_id`
- `acceptance_id`
- `unit_id`
- `proof_type`
- `location`
- `created_at`

(This mirrors proof log entries for quick lookup.)

---

9) Blockers / Unknowns / Decisions

9.1 blockers[] (required if any)

Each blocker:

- `blocker_id`
- `unit_id` (optional)
- `acceptance_id` (optional)
- `description`
- `severity` (low|med|high)
- `created_at`
- `resolved_at` (optional)
- `resolution_notes` (optional)
- `needs` (what input/decision fixes it)

9.2 decisions[] (optional)

Each decision:

- `decision_id`
- `description`
- `rationale` (optional)
- `made_by` (agent/user)
- `made_at`
- `refs[]` (spec IDs, template IDs, acceptance IDs)

9.3 overrides[] (optional)

Each override:

- `override_id`
- `scope` (standards/template/gate)
- `field_path` or `rule_id`
- `before`
- `after`
- `approved_by`
- `approved_at`
- `reason`

9.4 unknowns_status[] (optional but recommended)

Each entry:

- `unknown_id`
- `status` (open|resolved)
- `blocking` (boolean)
- `resolution_ref` (optional)

---

10) Last Known Good (LKG)

10.1 last_known_good (required)

- `timestamp`

- `unit_id` (optional)
- `verification_summary` (string)
- `proof_refs[]` (proof_ids supporting LKG)

Purpose: allow safe resume point and rollback reasoning.

---

11) History / Event Log (recommended)

To avoid overwriting important context, include either:

Option A (preferred): `history.events[]`

- append-only list of state events:
  - unit started/completed
  - acceptance pass/fail
  - proof attached
  - blocker created/resolved

Each event:

- `event_id`
- `event_type`
- `timestamp`
- `refs` (unit_id, acceptance_id, proof_id)

Option B: `history.log_ref`

- pointer to an external event log file

---

12) Failure Modes

- unit marked done without proofs
- acceptance results missing proof refs
- current_unit_id missing and no way to choose next
- blockers exist but not linked to unit/acceptance
- history overwritten (loss of audit trail)

13) Definition of Done (STATE-01)

STATE-01 is complete when:

- state snapshot structure is locked and machine-readable
- unit/acceptance/proof linkage rules are explicit
- blockers/decisions/overrides/unknown status fields are defined
- last-known-good is required
- history/event log approach is defined to prevent overwrites

STATE-02

STATE-02 — Resume Rules (How to Restart Without Drift)

(Hardened Draft — Full)

1) Purpose

Define the deterministic procedure an agent must follow to resume work using the Agent Kit and State Snapshot, without relying on chat memory, assumptions, or re-planning drift.

---

2) Inputs

- `01_core_artifacts/06_state_snapshot.json` (STATE-01)
- `01_core_artifacts/04_work_breakdown.json` (PLAN-01)
- `01_core_artifacts/05_acceptance_map.json` (PLAN-02)
- `01_core_artifacts/03_canonical_spec.json` (CAN)
- `01_core_artifacts/02_resolved_standards_snapshot.json` (STD)
- `00_KIT_MANIFEST.md` / `00_VERSIONS.md` (KIT-02/KIT-04)

---

3) Outputs

- A deterministic “resume decision”:
  - which unit to work on next
  - what prerequisites must be satisfied
  - whether the run is blocked (and why)
- Updated State Snapshot (timestamps, events)

---

4) Resume Invariants (must always be true)

1. **Artifacts over memory:** resume decisions come only from kit artifacts, never chat history.
2. **No skipping:** units cannot be skipped past hard dependencies without an override record.
3. **No proofless progress:** a unit cannot be marked done unless VER-03 conditions are met.
4. **Version awareness:** if kit versions changed, resume must treat this as a compatibility event.

5. **Blocking unknowns block:** open blocking unknowns prevent progress in affected scope.

---

5) Resume Procedure (Locked Steps)

Step 1 — Verify Kit Integrity

- Confirm required core artifacts exist (KIT-01).
- Confirm manifest and versions are present (KIT-02/KIT-04).

If missing → stop and record blocker: `KIT_INTEGRITY_MISSING`.

Step 2 — Load State Snapshot

Read:

- current_unit_id
- unit_status list
- blockers list
- acceptance_status
- open blocking unknowns

If snapshot missing or corrupted → stop and record blocker.

Step 3 — Validate State Consistency

Check:

- every unit_id referenced exists in work breakdown
- every acceptance_id referenced exists in acceptance map
- every proof_ref resolves to proof log (if available)

If inconsistencies exist:

- mark system blocked
- record blocker `STATE_INTEGRITY_FAIL` with pointers

Step 4 — Determine Resume Target

Decision order:

1. If `current_unit_id` exists and unit not done:

    - resume that unit
2. Else if any unit is `in_progress`:
    - choose the oldest `in_progress` unit (deterministic tie-breaker: earliest started_at, else lexical unit_id)
3. Else choose next eligible unit:
    - eligible means: status not_started AND all depends_on units are done
    - apply sequencing heuristics (PLAN-03) only as tie-breaker
4. If no eligible units:
    - if any units are blocked → overall status blocked
    - else if all required units done → overall status complete
    - else → record blocker `NO_ELIGIBLE_UNIT` (graph inconsistency)

Step 5 — Check Blocking Unknowns

If the chosen unit's scope_refs intersect any open blocking unknowns:

- do not proceed
- set unit status blocked
- record blocker `BLOCKING_UNKNOWN` referencing unknown_id(s)

Step 6 — Load Unit Execution Context

For selected unit:

- read unit scope, deliverables, acceptance items
- read relevant standards constraints (snapshot)
- identify required verification commands and proof types

Step 7 — Execute Unit Loop (by kit rules)

Proceed with implementation per entrypoint loop:

- implement deliverables
- run acceptance checks
- record proofs
- update acceptance_status
- update unit_status
- append history events

---

6) Resume Tie-Breakers (Deterministic)

When multiple candidates exist, order by:

1. dependency eligibility (hard)
2. foundational priority (setup/foundations first)
3. must-have priority rank (if linked)
4. risk flags (higher earlier)
5. lexical unit_id

---

7) Compatibility / Version Change Handling

If the kit's versions changed since last known snapshot event:

- record a `COMPATIBILITY_EVENT` in history
- do not assume prior proofs remain valid
- rerun required global checks if standards or template versions changed materially (policy-driven)

(This does not modify the kit; it modifies state/proof requirements.)

---

8) Failure Modes

- agent resumes using chat memory and skips artifacts
- unit chosen without dependency eligibility
- continuing despite open blocking unknowns
- state snapshot references IDs that don't exist (corruption)
- compatibility changes ignored (stale proofs)

---

9) Definition of Done (STATE-02)

STATE-02 is complete when:

- resume procedure is explicit and deterministic

- candidate selection and tie-breakers are defined
- blocking unknown policy is integrated
- integrity checks for state/work/acceptance/proof links are defined
- compatibility/version-change handling is defined without relying on memory

STATE-03

STATE-03 — Handoff Rules (Agent-to-Agent Continuity)

(Hardened Draft — Full)

1) Purpose

Define how work is safely handed off between agents (or sessions) without drift by ensuring the next agent can resume using only artifacts, not chat memory.

STATE-03 governs:

- what must be recorded before handoff
- what the receiving agent must do first
- what constitutes a valid handoff package

---

2) Inputs

- State Snapshot (STATE-01)
- Resume rules (STATE-02)
- Work Breakdown + Acceptance Map (PLAN)
- Proof Log (VER-01)
- Kit manifest/index/versions (KIT-02/KIT-04)
- Unknowns (CAN-03)

---

3) Outputs

A deterministic **handoff state** consisting of:

- updated state snapshot (complete, consistent)
- proof log pointers (up to date)
- explicit blockers/unknowns/decisions
- last-known-good checkpoint

---

4) Handoff Invariants (must always be true)

1. **No hidden context:** all critical context must be in artifacts (state/proof/unknowns/decisions), not chat.
2. **Unambiguous "what's next":** current unit or next eligible unit must be determinable.

3. **Proof-linked progress:** any completed work claims must have proof refs.
4. **Integrity:** all referenced IDs resolve (unit_id, acceptance_id, proof_id, unknown_id).
5. **Version awareness:** handoff must include the kit's pinned versions.

---

5) Pre-Handoff Requirements (Sender Must Do)

H-01 Update State Snapshot

Sender must ensure:

- `updated_at` reflects latest work
- `current_unit_id` is correct (or null if none)
- unit statuses are accurate
- acceptance_status updated for any checks run
- blockers recorded for any inability to proceed
- decisions/overrides recorded if made
- history events appended for changes made

H-02 Update Proof Log

Sender must ensure:

- proofs are recorded with proof_id, acceptance_id, unit_id
- proof locations are valid and accessible
- no "proof" exists only in chat

H-03 Last Known Good (LKG) Updated

Sender must record:

- last successful verification checkpoint
- supporting proof refs

H-04 Unknowns Updated

Sender must:

- mark resolved unknowns as resolved with resolution_ref
- record open blocking unknowns as blockers where they impact current work

H-05 Integrity Check Before Handoff (mandatory)

Before handoff, sender must validate:

- state snapshot references resolve to work breakdown and acceptance map
- proofs referenced exist and are linked correctly
- no unit is marked done without required proofs (VER-03)

If integrity fails → handoff is invalid; record blocker `HANDOFF_INTEGRITY_FAIL`.

---

6) Receiving Agent Procedure (Must Do First)

Step 1 — Read Entrypoint + Manifest + Versions

Receiving agent must read:

- `00_START_HERE.md`
- `00_KIT_MANIFEST.md`
- `00_VERSIONS.md`

Step 2 — Load Core Artifacts

Must load:

- standards snapshot
- canonical spec
- work breakdown
- acceptance map
- state snapshot

Step 3 — Run Resume Algorithm

Must execute STATE-02 to determine:

- resume target unit
- blockers/unknown constraints
- required verification commands

Step 4 — Confirm No Drift

Receiving agent must not proceed if:

- state integrity check fails
- versions changed unexpectedly without recorded compatibility event
- blocking unknowns intersect current unit scope

---

7) Handoff Package Definition (What must exist)

A valid handoff requires these artifacts to be present and current:

- `01_core_artifacts/06_state_snapshot.json` (updated)
- `00_PROOF_LOG.md` (updated)
- core artifacts and manifest/versions (present)
- any referenced proof files exist at recorded locations

---

8) Failure Modes

- sender forgets to record blockers/decisions, next agent improvises
- sender claims completion without proof refs
- receiver uses chat memory rather than state snapshot
- missing proof files break auditability
- versions mismatch causes stale assumptions

---

9) Definition of Done (STATE-03)

STATE-03 is complete when:

- pre-handoff requirements are explicit and enforceable
- receiving procedure is explicit and references STATE-02
- handoff package contents are explicit
- integrity checks and failure handling are specified

Governance & Evolution

Governance & Evolution

- GOV-01 Versioning Policy (templates/standards/system)
- GOV-02 Change Control Rules (what can change, approval requirements)
- GOV-03 Deprecation & Migration Rules
- GOV-04 Audit / Traceability Rules

GOV-01

GOV-01 — Versioning Policy

(Hardened Draft — Full)

1) Purpose

Define the strict versioning rules that keep Axion stable and reproducible over time, including:

- what is versioned
- when versions must change
- how versions are recorded in kits
- what counts as breaking vs non-breaking change
- how compatibility is maintained across system upgrades

---

2) Inputs

- System boundaries (SYS-10) — no uncontrolled evolution
- Traceability model (SYS-06) — provenance and stamps
- Kit version stamping (KIT-04)
- Library structures (STD-01, TMP-01)
- Canonical model evolution rules (CAN-01/02/03)

---

3) Outputs

A locked policy that governs versioning for:

- system docs/contracts
- standards library
- template library
- intake schema and rules
- canonical model
- kit contracts
- generated artifacts (as stamped outputs)

---

4) Versioning Invariants (must always be true)

1. **No silent change:** any change that affects behavior/output must change a version stamp.

2. **Pinned outputs:** every kit records versions required to reproduce generation (KIT-04).
3. **Compatibility declared:** breaking changes require compatibility notes/migrations.
4. **Scope-appropriate versioning:** libraries and contracts version separately (no single giant version for everything).
5. **Determinism preserved:** versions exist to explain output differences.

---

5) Versioned Surfaces (What MUST be versioned)

V-SURF-01 System Version

Covers: SYS/ORD/GOV/EXEC contract docs and system-wide behavior.

- `system_version`

V-SURF-02 Intake Versions

- `form_version`
- `schema_version`
- `int_ruleset_version` (INT-03)

V-SURF-03 Standards Library Versions

- `standards_library_version`
- per-pack versions (`pack_id@pack_version`)
- resolver ruleset version (`STD-02`)

V-SURF-04 Template Library Versions

- `template_library_version`
- per-template versions (`template_id@template_version`)
- template index version (`TMP-01`)
- fill rules version (`TMP-04`)

V-SURF-05 Canonical Model Versions

- canonical spec model version (`CAN-01/02/03`)
- ID rules version (`CAN-02`)
- unknown model version (`CAN-03`)

V-SURF-06 Planning & Proof Versions

- planning rules version (`PLAN-01/02/03`)
- proof rules version (`VER-01/02/03`)

V-SURF-07 Kit Contract Versions

- kit folder structure version (`KIT-01`)
- manifest format version (`KIT-02`)
- entrypoint contract version (`KIT-03`)

---

6) When Versions MUST Change (Trigger Rules)

6.1 System version triggers

System version must change when:

- pipeline stage behavior changes
- gate DSL/rules change (ORD-02/03)
- system boundaries change (SYS-10)
- any contract that affects generation semantics changes

6.2 Intake version triggers

- `form_version` changes when UI fields/pages change.
- `schema_version` changes when field paths/types/base constraints change.
- `int_ruleset_version` changes when dependency/threshold logic changes (INT-03).

6.3 Standards version triggers

- standards library version changes when:
    - packs added/removed
    - applies_when filters change
    - rule defaults/constraints change
- pack_version changes when any rule inside pack changes.

6.4 Template version triggers

- template_library_version changes when:
    - templates added/removed
    - template applies_when/requiredness changes
    - template index schema changes
- template_version changes when:
    - template structure changes
    - required fields change
    - placeholders change
    - output format changes materially

6.5 Canonical model triggers

- canonical model version changes when:
    - entity shape changes
    - reference rules change
    - ID rules change
    - unknown model changes

6.6 Kit contract triggers

- kit structure version changes when:
    - required root files change
    - paths for core artifacts change
    - pack folder contract changes
- manifest version changes when JSON schema changes
- entrypoint contract version changes when required sections/order changes

---

7) Breaking vs Non-Breaking Changes (Locked Definitions)

7.1 Breaking change

A change is breaking if it causes:

- old kits to be unreadable by the runner/builder
- old manifests/indexes to be incompatible
- canonical spec references to no longer resolve
- template filler to fail on old templates/specs without migration

Breaking changes require:

- version bump
- compatibility notes
- migration guidance (GOV-03)

7.2 Non-breaking change

A change is non-breaking if:

- it is additive (new optional fields) and defaults preserve meaning
- old kits still run without reinterpretation
- no required paths/fields change

Still requires version bump if behavior/output changes.

---

8) Version Recording Rules

- all relevant versions must be stamped into:
  - `00_VERSIONS.md` (KIT-04)
  - `00_KIT_MANIFEST.md` (KIT-02)
  - standards snapshot (STD-03 includes pack versions)
  - template selection result (if stored)

---

9) Failure Modes

- outputs change but version didn't → impossible to audit
- breaking change without migration notes → older kits become unusable

- single “global version only” → cannot isolate what changed
- versions recorded inconsistently across manifest/snapshot/versions file

---

10) Definition of Done (GOV-01)

GOV-01 is complete when:

- versioned surfaces are enumerated
- version bump triggers are explicit per surface
- breaking vs non-breaking definitions are locked
- recording rules align with KIT-04 and STD-03
- policy prevents silent changes under SYS-10 B-08

GOV-02

GOV-02 — Change Control Rules

(Hardened Draft — Full)

1) Purpose

Define the strict process and rules for changing Axion system assets so evolution is controlled, auditable, and compatible with existing kits. This prevents “junkyard growth” and silent drift.

GOV-02 governs changes to:

- SYS/ORD/GOV/EXEC contracts
- intake form/schema/rules
- canonical model
- standards library
- template library
- kit contracts

---

2) Inputs

- Versioning policy (GOV-01)
- System boundaries (SYS-10), especially “no uncontrolled evolution”
- Traceability model (SYS-06)
- Gate model (SYS-07) and Gate DSL (ORD-02/03)

---

3) Outputs

A change control process that produces:

- a recorded change request
- an impact assessment
- version bumps (as required)
- compatibility notes/migrations when breaking
- a governance gate pass report

---

4) Change Control Invariants (must always be true)

1. **No untracked edits:** system assets cannot be modified without a change record.

2. **Versioning required:** any change that affects behavior/output must bump versions per GOV-01.
3. **Breaking changes require migration:** no breaking change without GOV-03 artifacts.
4. **Read-only runs:** normal kit generation runs must treat system assets as read-only (SYS-10).
5. **Auditability:** every change must have who/why/what/when recorded.

---

5) Change Classes (Locked)

CC-01 Documentation-only (advisory text)

- Clarifications that do not change behavior, contracts, or gating.
- Requires: change record + doc version bump if doc is LOCKED.

CC-02 Contract change (structure/format)

- Changes to schemas, template contracts, kit layout, manifest schema, gate DSL.
- Requires: version bump + compatibility notes; often breaking.

CC-03 Rule change (behavioral)

- Changes to validation thresholds, selection rules, standards precedence, gate logic.
- Requires: version bump; may be breaking depending on output impact.

CC-04 Library change (standards/templates)

- Adding/removing/modifying template or standards packs.
- Requires: library version bump + updated indexes.

CC-05 Model evolution (canonical spec)

- Changes to entity shapes, references, unknown model.
- Requires: canonical model version bump + migration policy (GOV-03).

---

6) Required Change Record (Locked Fields)

Every change must have a record containing:

- `change_id`
- `change_class` (CC-01..CC-05)
- `scope` (which surfaces are impacted: SYS/INT/CAN/STD/TMP/KIT/ORD/PLAN/VER/STATE)
- `summary`
- `rationale`
- `proposed_by`
- `approved_by`
- `created_at`
- `approved_at`
- `files_changed[]` (paths)
- `versions_bumped[]` (what and new values)
- `breaking_change` (boolean)
- `compatibility_notes_ref` (required if breaking)
- `migration_ref` (required if breaking model/kit/schema)
- `risk_assessment` (what might break)

---

7) Approval Rules (Who can approve what)

- LOCKED contracts (SYS, ORD, KIT, CAN model rules) require maintainer approval.
- Library changes (STD/TMP) require maintainer approval.
- Advisory docs can be approved by maintainer or delegated role (policy-defined).

Rule: external agent cannot approve system changes.

---

8) Governance Gates (Must Pass Before Release)

Before a change becomes active, run governance gate checks:

GOV2-GATE-01 Version bump check

- all impacted surfaces have required version bumps per GOV-01.

GOV2-GATE-02 Index update check

- if STD or TMP changed, corresponding index/registry updated.

GOV2-GATE-03 Compatibility check (breaking)

- compatibility notes present and complete
- migration guidance present (if required)

GOV2-GATE-04 No self-modifying behavior

- confirm the change does not enable system assets to be mutated during normal runs.

If any gate fails → change is rejected.

---

9) Release Discipline (How changes become active)

A change becomes active only when:

- change record exists and is approved
- required version bumps are applied
- governance gate report is PASS
- changelog updated

---

10) Failure Modes

- “quick edits” without change record → drift
- version not bumped → outputs change silently
- breaking change without migration → old kits unusable
- ambiguous approvals → unauthorized modifications

11) Definition of Done (GOV-02)

GOV-02 is complete when:

- change classes are defined
- required change record fields are locked
- approval rules are explicit
- governance gates are explicit and map to GOV-01 triggers
- activation rules prevent untracked changes

GOV-03

GOV-03 — Deprecation & Migration Rules

(Hardened Draft — Full)

1) Purpose

Define how Axion evolves without breaking existing kits by establishing:

- how assets are deprecated (templates/standards/contracts)
- how breaking changes are migrated (schemas/models/kit formats)
- how backward compatibility is documented and enforced

---

2) Inputs

- Versioning policy (GOV-01)
- Change control rules (GOV-02)
- Kit version stamping rules (KIT-04)
- Canonical model rules (CAN-01/02/03)
- Template and standards registries (TMP-01, STD-01)

---

3) Outputs

A deterministic deprecation/migration discipline including:

- deprecation records
- replacement mappings
- migration guides for breaking changes
- compatibility notes for old kits

---

4) Invariants (must always be true)

1. **No sudden removal:** deprecated assets remain readable and indexed until a declared removal milestone.
2. **Replacement mapping required:** deprecation must specify what replaces it (or “none”).
3. **Breaking changes require migration path:** any breaking change must have either:
    - a migration script/process, or
    - explicit “not migratable” declaration (rare, requires maintainer sign-off)

4. **Kits remain interpretable:** old kits must remain understandable using pinned versions and compatibility notes.
5. **Migration is traceable:** migrations must record before/after versions and what changed.

---

5) Deprecation Rules (Templates/Standards/Docs)

5.1 Deprecation Record (required fields)

- `deprecated_asset_id` (template_id / pack_id / doc_id)
- `asset_type` (template | standards_pack | contract_doc)
- `deprecated_in_version`
- `deprecated_at`
- `reason`
- `replacement_asset_id` (optional)
- `sunset_policy` (enum):
  - `supported_indefinitely`
  - `supported_until_date`
  - `supported_until_version`
- `sunset_target` (date or version; required if not indefinite)

5.2 Deprecation Behavior

- Deprecated assets must remain in their registry with:
  - status: deprecated
  - replacement pointer if any
- New kit generation must not select deprecated assets unless:
  - explicitly pinned by policy for compatibility mode
  - or no replacement exists and omission would break baseline coverage

---

6) Migration Rules (Breaking Changes)

6.1 When a migration is required

A migration is required when a breaking change affects:

- intake schema field paths/types (INT-02)
- canonical spec entity shapes/references (CAN-01/02/03)
- standards snapshot schema (STD-03)
- template index schema or placeholder variables (TMP)
- kit folder structure / manifest schema (KIT-01/KIT-02)

6.2 Migration Artifact (required)

Every breaking change must include a Migration Guide containing:

- `migration_id`
- `from_version` → `to_version`
- `assets_affected[]` (what changes)
- `mapping_rules` (old → new fields/paths)
- `step_by_step_procedure`
- `validation_checks` (how to confirm migration success)
- `rollback_strategy` (if migration fails)
- `known_risks`

6.3 Migration Modes

- **Automated migration:** script/tool transforms artifacts deterministically
- **Manual migration:** step-by-step instructions with required proof checks

Rule: manual migrations must still produce proof of completion and record it.

---

7) Compatibility Notes (Required for Breaking Changes)

Compatibility notes must include:

- what breaks
- what remains compatible
- how old kits should be interpreted

- which version stamps determine behavior
- what the agent must do differently when encountering older kits

---

8) Enforcement (Governance Gates)

A breaking change cannot be released unless:

- deprecation record exists (if applicable)
- migration guide exists (if required)
- compatibility notes exist
- version bumps are correct per GOV-01
- registries updated (TMP/STD)

Failing any of these → change rejected.

---

9) Failure Modes

- assets removed without replacement → older kits fail
- breaking changes shipped without migration → unusable legacy artifacts
- ambiguous compatibility notes → agents misinterpret old kits
- deprecated assets still selected by default → drift and inconsistency

---

10) Definition of Done (GOV-03)

GOV-03 is complete when:

- deprecation record contract is locked
- migration guide requirements are locked
- compatibility notes requirements are locked
- enforcement conditions are explicit and gateable
- rules guarantee old kits remain interpretable via pinned versions

GOV-04

GOV-04 — Audit / Traceability Rules

(Hardened Draft — Full)

1) Purpose

Define the audit rules that ensure Axion outputs are fully traceable and reviewable. This document turns SYS-06 “traceability model” into enforceable governance requirements across:

- intake → spec → plan → templates → kit → proof → completion
- system evolution (changes, versions, migrations)

---

2) Inputs

- Traceability model (SYS-06)
- Gate model (SYS-07) + Gate DSL (ORD-02/03)
- Version stamping rules (KIT-04)
- Deprecation/migration rules (GOV-03)
- Proof system rules (VER-01/02/03)

---

3) Outputs

A set of locked audit requirements that:

- specify what must be recorded
- specify minimum metadata and linkage
- define audit failure conditions (what breaks auditability)

---

4) Audit Invariants (must always be true)

1. **End-to-end provenance exists:** every derived artifact links to its upstream inputs.
2. **Single authoritative mapping:** manifest/index + version stamps make the kit navigable and reproducible.
3. **Decisions are recorded:** overrides/decisions/blockers are recorded with IDs and timestamps.
4. **Proof links to acceptance:** no completion claim without acceptance-linked proof.
5. **Change history is recorded:** system updates are traceable through change records, versions, and migrations.

5) Required Audit Records (Locked)

5.1 Project Run Audit Spine (must exist per kit)

The kit must include and link:

- submission_id + schema/form versions (INT-04)
- validation result (INT-05 stored or referenced)
- normalized input record
- resolved standards snapshot
- canonical spec
- work breakdown
- acceptance map
- kit manifest/index
- versions file
- state snapshot
- proof log

Rule: missing any of these breaks auditability.

5.2 Gate Reports (recommended; required for strict mode)

For each stage gate (intake/spec/template/packaging/completion):

- store gate report or include pointers to it
- include rule_ids + pointers + timestamps

5.3 Overrides / Decisions / Blockers (required if any occur)

If any override/decision occurs, it must be recorded in:

- standards snapshot overrides section (for standards overrides)
- state snapshot overrides/decisions (for execution decisions)
- change control record (for system updates)

6) Linkage Rules (must always be true)

GOV4-LINK-01 Artifact linkage

Every artifact must include:

- submission_id
- its own id (spec_id, work_breakdown_id, etc.)
- upstream ids (where applicable)

GOV4-LINK-02 Proof linkage

Every proof entry must include:

- proof_id → acceptance_id → unit_id
  and acceptance_id must exist in acceptance map.

GOV4-LINK-03 Manifest linkage

Every important artifact path must appear in manifest JSON:

- core artifacts
- proof log
- key filled docs
  and must physically exist.

GOV4-LINK-04 Version linkage

Versions recorded in:

- `00_VERSIONS.md`
- `00_KIT_MANIFEST.md`
- `resolved_standards_snapshot.json`
  must agree.

---

7) Audit Failure Conditions (hard fails)

Audit is considered failed if:

- any core artifact is missing
- any required id link is missing or inconsistent
- proof references are broken
- unit marked done without proof for hard gates
- versions mismatch across manifest/snapshot/versions
- system changes lack change record/version bump when applicable

---

8) Audit Output (what an audit should be able to answer)

A valid audit must be able to answer:

- What did the user submit (raw)?
- What ruleset was applied (standards snapshot)?
- What is the canonical truth model (spec)?
- What work was planned (work breakdown)?
- What proved it was done (acceptance + proof)?
- What changed over time (state snapshot events)?
- What versions produced this kit (versions file)?
- If something differs from another run, which version changed?

---

9) Failure Modes

- kits exist but cannot be reproduced due to missing versions
- “done” status is narrative without proof
- decisions made in chat and not recorded
- compatibility changes shipped without traceable migrations

---

10) Definition of Done (GOV-04)

GOV-04 is complete when:

- required audit records are enumerated

- linkage rules are explicit and enforceable
- audit failure conditions are explicit
- version consistency requirements align with KIT-04
- proof and completion linkage requirements align with VER-03

Execution Instructions

Execution Instructions

- EXEC-01 Internal Agent Runbook (how internal agent runs the pipeline)
- EXEC-02 External Agent Prompt Template *(the one external-facing piece)*
- EXEC-03 Failure Handling Playbook (what to do when blocked)

EXEC-01

EXEC-01 — Internal Agent Runbook (How the Internal Agent Runs Axion)

(Hardened Draft — Full)

1) Purpose

Define the deterministic procedure the **internal agent** must follow to run Axion end-to-end, including:

- what to read first
- what artifacts to produce at each stage
- what gates to run and when to stop
- what outputs must be packaged into the kit
- what is prohibited (no invention, no self-modifying runs)

This is the operational “how to run the system” contract.

---

2) Inputs

- System contracts: SYS-01..SYS-10
- Build order graph + gate DSL: ORD-01..ORD-03
- Intake system: INT-01..INT-05
- Canonical model: CAN-01..CAN-03
- Standards system: STD-01..STD-03
- Template system: TMP-01..TMP-05
- Planning system: PLAN-01..PLAN-03
- Proof system contracts: VER-01..VER-03
- Kit system: KIT-01..KIT-04
- State system: STATE-01..STATE-03
- Governance rules: GOV-01..GOV-04

---

3) Outputs

- Validated submission boundary
- Core artifacts:
    - normalized input record
    - resolved standards snapshot
    - canonical spec

    - work breakdown
    - acceptance map
    - initial state snapshot
- Filled templates per selection result
- Packaged Agent Kit (folder + manifest/index + entrypoint + versions)
- Gate reports (recommended) and error reports (required on failure)

---

4) Runbook Invariants (must always be true)

1. **Read-only system assets:** templates/standards/contracts are read-only during normal runs (SYS-10).
2. **No invention:** missing info becomes UNKNOWN or gate fail; never guessed (SYS-01/SYS-10).
3. **No forward progress without compliance:** stop on hard gate failures (SYS-07).
4. **Traceability:** all artifacts are linked by IDs and version-stamped (SYS-06, KIT-04).
5. **Deterministic outputs:** same inputs + versions yield same structure (SYS-02).

---

5) Execution Modes (Locked)

- **Mode A: Kit Generation Mode (default)**
  Produces a new agent kit from a submission. System assets are read-only.
- **Mode B: System Update Mode (maintainer-only)**
  Not allowed in this runbook. Governed by GOV change control.

---

6) Pipeline Procedure (Locked Steps)

Step 0 — Initialize Run Context

Create `run_context`:

- submission_id (or pending)
- pinned versions:
    - system_version
    - schema_version (from intake)
    - standards_library_version

    - template_library_version
- environment label (optional)

Gate: fail if versions cannot be pinned.

Step 1 — Intake Submission Record

- Create INT-04 Submission Record from raw user form payload + uploads
- Stamp `form_version`, `schema_version`

Gate: record creation success; payload hash integrity if enabled.

Step 2 — Validate Submission (INT-05)

- Validate INT-04 raw payload against:
    - INT-02 schema (types/base constraints)
    - INT-03 validation rules (dependencies/thresholds/references)
- Produce INT-05 Validation Result

Gate: if `is_valid == false` → stop and return validation report (no downstream artifacts).

Step 3 — Normalize Input (A2 Normalized Input Record)

- Normalize validated submission deterministically:
    - canonical naming
    - list normalization/deduping
    - enum standardization
    - extraction where rules allow
- Produce `01_normalized_input_record.json`

Gate: normalization integrity (no invention) and schema consistency.

Step 4 — Resolve Standards (STD-03)

- Build resolver_context from normalized input

- Select packs, merge rules, apply/deny overrides (STD-02)
- Emit `02_resolved_standards_snapshot.json`

Gate: conflict-free snapshot; version stamps present; blocked overrides recorded.

Step 5 — Build Canonical Spec (CAN)

- Build `03_canonical_spec.json` according to CAN-01
- Assign stable IDs (CAN-02)
- Create unknowns (CAN-03) as needed

Gate: referential integrity; no duplicate truth; blocking unknown policy respected.

Step 6 — Work Breakdown (PLAN-01)

- Decompose canonical spec into units:
  - baseline units
  - feature/workflow slices
  - integration units (if needed)
- Emit `04_work_breakdown.json`

Gate: all must-have features mapped to units; no cycles; dependencies valid.

Step 7 — Acceptance Map (PLAN-02)

- Create acceptance items for each unit:
  - minimum required items
  - standards-driven checks
  - risk-based enrichment
- Emit `05_acceptance_map.json`

Gate: every unit has hard-gate acceptance with proof requirements.

Step 8 — Initialize State Snapshot (STATE-01)

- Create initial `06_state_snapshot.json`:
    - status in_progress
    - current_unit_id set to first eligible unit (per sequencing)
    - acceptance_status initialized
    - unknowns_status initialized
    - LKG empty or initial baseline

Gate: state references resolve; no unit marked done.

Step 9 — Select Templates (TMP-03)

- Compute selection context from routing + gates
- Select templates deterministically from Template Index
- Emit selection result (stored in packaging metadata or as internal artifact)

Gate: baseline coverage satisfied; no missing upstream dependencies.

Step 10 — Fill Templates (TMP-04)

- Fill each selected template using authoritative artifacts only
- Emit filled docs into correct kit locations (KIT-01)
- Ensure unknowns inserted map to canonical unknown objects

Gate: TMP-05 template gate must pass for each filled doc.

Step 11 — Package Kit (KIT)

- Create kit folder structure (KIT-01)
- Place core artifacts under `01_core_artifacts/`
- Place filled docs under `10_app/` and packs (`20_domains/` etc.) as applicable
- Create `00_KIT_MANIFEST.md` + `00_KIT_INDEX.md` (KIT-02)
- Create `00_START_HERE.md` (KIT-03)
- Create `00_VERSIONS.md` (KIT-04)
- Create `00_RUN_RULES.md` and blank `00_PROOF_LOG.md`

Gate: packaging gate: required files exist; manifest/index consistent; N/A stubs present.

Step 12 — Final Output

- Produce zipped kit (optional)
- Return kit path + manifest summary + any warnings

---

7) Failure Handling (Hard Rules)

If any hard gate fails:

- stop immediately
- emit a gate report with:
  - rule_id
  - pointers
  - remediation steps
- do not generate downstream artifacts beyond the failure point

---

8) Definition of Done (EXEC-01)

EXEC-01 is complete when:

- steps cover intake → validation → normalization → standards → spec → plan → acceptance → state → templates → packaging
- stop conditions are explicit and match SYS-07/ORD-02
- read-only system asset rule is explicit
- outputs align with KIT-01..KIT-04 contracts

EXEC-02

EXEC-02 — External Agent Prompt Template (Builder Instructions)

(Hardened Draft — Full)

1) Purpose

Define the standard prompt/instruction template the user (or platform) gives to the **external builder agent** so it can execute the Agent Kit correctly with minimal drift.

This template must:

- force the agent to treat kit artifacts as truth
- define an execution loop tied to work units + acceptance criteria + proof
- require state/proof updates
- forbid invention and proofless completion claims

---

2) Inputs

- Entrypoint contract (KIT-03)
- Manifest/index contract (KIT-02)
- Proof rules (VER-01/02/03)
- State rules (STATE-01/02)
- System boundaries (SYS-10) relevant to external agent behavior

---

3) Outputs

A reusable prompt template that:

- can be pasted into any vibecode platform
- instructs the agent to read the kit and follow it
- standardizes reporting and proof logging

---

4) Prompt Template (Copy/Paste)

You are the External Builder Agent. Your job is to build the project using the provided Agent Kit. You must treat the kit artifacts as the only authoritative truth. Do not invent requirements.

ABSOLUTE RULES
1) Truth sources: Only use these as truth:
- 01_core_artifacts/02_resolved_standards_snapshot.json
- 01_core_artifacts/03_canonical_spec.json
- 01_core_artifacts/04_work_breakdown.json
- 01_core_artifacts/05_acceptance_map.json
- 01_core_artifacts/06_state_snapshot.json
- 00_KIT_MANIFEST.md, 00_VERSIONS.md

2) No guessing: If something is missing or ambiguous, create/append a BLOCKER entry in 06_state_snapshot.json and stop on that unit.
3) No proof, no done: You may not mark a unit DONE unless all hard-gate acceptance items pass and proof IDs are recorded in 00_PROOF_LOG.md and referenced in 06_state_snapshot.json.
4) Do not change constraints: Do not modify standards, canonical spec, or kit contracts. If you believe a change is required, record it as a decision request in state snapshot and stop.

STARTUP (do this first)
A) Open and read 00_START_HERE.md fully.
B) Open and read 00_KIT_MANIFEST.md and 00_VERSIONS.md.
C) Load 06_state_snapshot.json and identify current_unit_id. If none, pick next eligible unit strictly using dependencies in 04_work_breakdown.json.

EXECUTION LOOP (repeat until complete)
1) Select the current unit:
- Read its scope_refs, deliverables, and depends_on from 04_work_breakdown.json.
- Read its acceptance items from 05_acceptance_map.json (hard_gate vs soft_gate).

2) Implement ONLY this unit's deliverables.
3) Run the verification commands specified by acceptance items (how_to_verify).
4) Capture proof:
- For each hard_gate acceptance item, create a proof entry in 00_PROOF_LOG.md with proof_id, acceptance_id, unit_id, proof_type, location, timestamp.

5) Update 06_state_snapshot.json:
- acceptance_status for each acceptance_id (pass/fail) + proof_refs
- unit_status for the unit (in_progress/blocked/done)
- blockers if anything fails or is missing

6) If all hard_gate acceptance items pass with proof refs, mark the unit DONE and move to the next eligible unit.

REPORTING FORMAT (each time you finish a unit or get blocked)
Output a short report:
- unit_id

- what changed (files)
- acceptance results (pass/fail)
- proof_ids added
- next unit_id OR blocker description

STOP CONDITIONS
- If a hard_gate acceptance fails, stop and record blocker + proof of failure.
- If required info is missing, stop and record blocker.
- If dependencies aren't satisfied, do not proceed; choose another eligible unit or stop if none.

FINISH
When all required units are DONE:
- Confirm VER-03 conditions are met (no blocking unknowns, all hard gates passed with proof).
- Produce a completion report listing proof_ids for final completion.

---

5) Invariants (must always be true)

- External agent starts from kit entrypoint
- External agent updates state/proof as it works
- External agent does not invent requirements or change constraints
- Completion claims are proof-linked

---

6) Definition of Done (EXEC-02)

EXEC-02 is complete when:

- prompt enforces truth sources and no guessing
- defines deterministic execution loop tied to work breakdown + acceptance map
- mandates proof logging and state snapshot updates
- defines stop conditions and reporting format consistent with VER/STATE/KIT contracts

EXEC-03

EXEC-03 — Failure Handling Playbook (What to Do When Blocked)

(Hardened Draft — Full)

1) Purpose

Define the standard failure-handling procedure for both internal and external agents so blockers are handled deterministically, recorded properly, and do not produce drift through improvisation.

This playbook defines:

- failure categories
- required response steps
- what to record (state/proof/gate report)
- when to stop vs continue
- how to request missing input

---

2) Inputs

- Gate model + DSL (SYS-07, ORD-02)
- Validation output contract (INT-05)
- Unknowns model (CAN-03)
- Proof rules (VER-01/02/03)
- State snapshot contract (STATE-01)

---

3) Outputs

- Blocker entries (STATE-01 blockers[])
- Gate reports (for internal agent stage failures)
- Proof of failure where applicable (command outputs, logs)
- A deterministic “what is needed” request

---

4) Failure Handling Invariants (must always be true)

1. **No improvisation:** when blocked, do not guess; record blocker and stop that scope.

2. **Record before stop:** every failure must produce an artifact update (gate report or state blocker).
3. **Pointer precision:** every failure must include pointers (unit_id/acceptance_id/field_path/file_path).
4. **Actionable remediation:** failures must state exactly what's needed to proceed.
5. **No silent continuation:** continuing with warnings is allowed only where policy explicitly permits.

---

5) Failure Categories (Locked)

F-01 Intake Validation Failure (internal agent)

Examples: missing required fields, dependency violations, thresholds not met.

F-02 Standards Resolution Failure (internal agent)

Examples: unresolved conflicts, blocked override attempts that are required.

F-03 Spec Integrity Failure (internal agent)

Examples: broken references, duplicate truth, blocking unknowns violate stage policy.

F-04 Template Fill/Completeness Failure (internal agent)

Examples: unresolved placeholders, invented content, contradictions with standards/spec.

F-05 Packaging Failure (internal agent)

Examples: missing required files, manifest/index mismatch, silent omission of slots.

F-06 Verification Failure (external agent)

Examples: tests fail, build fails, lint fails, acceptance hard gate fails.

F-07 Missing Information / Ambiguity (either agent)

Examples: unresolved decisions required for progress; contradictions; missing constraints.

F-08 Environment/Tooling Failure (external agent)

Examples: dependency install fails; environment mismatch; command not available.

---

6) Required Response Steps (By Category)

6.1 Internal Agent Failures (F-01..F-05)

Step A — Stop progression

- Do not produce downstream artifacts beyond failure point.

Step B — Emit Gate Report

- gate_id
- target artifact
- status fail
- issues[] with rule_id, pointers, remediation

Step C — If fixable by user input

- output a precise request:
  - which field paths missing
  - what constraints to satisfy
  - minimum required content

Step D — Preserve traceability

- record that run stopped at stage X with submission_id and versions.

6.2 External Agent Failures (F-06..F-08)

Step A — Record blocker
In `06_state_snapshot.json` blockers[]:

- blocker_id

- unit_id
- acceptance_id (if applicable)
- description
- severity
- needs (exact next input/action)

Step B — Capture failure proof

- command output proof (exit code + stderr/stdout ref)
- log excerpt proof if relevant

Step C — Mark acceptance status FAIL

- update acceptance_status entry
- link proof_id(s)

Step D — Set unit status BLOCKED

- unit_status.status = blocked
- current_unit_id stays or moves per resume rules

---

7) “What is Needed” Request Format (Locked)

When requesting help (from user or maintainer), the agent must output:

- context: submission_id/spec_id/unit_id
- failure category (F-0X)
- exact rule_id(s) failing
- pointers (field_path/template section/file_path)
- minimal required input to proceed (bullet list)
- what was attempted (if external; commands run + exit codes)
- proof_ids for failure evidence

No vague “it didn’t work” reports.

---

8) Continue-With-Warnings Policy (Strict)

Allowed only if:

- acceptance item is soft_gate
- policy permits “pass with warnings”
- warnings are recorded in state snapshot and completion report

Hard gates never allow “continue as done.”

---

9) Failure Modes Prevented

- agent guessing to “push through”
- losing context between sessions
- untraceable failures
- repeated failures without recorded evidence
- marking done despite failed checks

---

10) Definition of Done (EXEC-03)

EXEC-03 is complete when:

- failure categories are locked
- required response steps are explicit for internal vs external failures
- blocker + proof capture requirements align with STATE and VER
- request format is precise and pointer-based
- warning continuation policy is strict and enforceable

Doc Mapping

Document Dependency Map (Interaction + Order)

Below is the **dependency graph** for the internal doc system: what each doc relies on, and what it unlocks. Format:

- **Doc** → depends on → produces/enables

A) System Foundations

- **SYS-01 System Purpose & Guarantees** → (none) → sets non-negotiables used by all gates
- **SYS-02 High-Level Architecture** → SYS-01 → defines major stages and artifact flow
- **SYS-03 Glossary / Core Terms** → SYS-01 → standard language used everywhere

---

B) Intake & Validation (Front Door)

- **INT-01 Intake Form Spec** → SYS-03 → defines what users can submit
- **INT-02 Intake Schema Spec** → INT-01 → defines valid submissions (types/requiredness)
- **INT-03 Intake Validation Rules** → INT-02 + skill-level rules → defines dependencies/thresholds
- **INT-04 Submission Record Format** → INT-01 + INT-02 → defines immutable raw storage
- **INT-05 Validator Output Format** → INT-03 → defines pass/fail + error payload

Enables: a validated submission boundary for downstream stages.

C) Canonical Data Model (Truth Layer)

- **CAN-01 Canonical Spec Model** → SYS-03 + INT-02 → defines the canonical spec structure
- **CAN-02 ID & Reference Rules** → CAN-01 → defines stable IDs + referential integrity rules
- **CAN-03 Unknowns Model** → CAN-01 → defines unknown representation + rules

Enables: deterministic spec building + no duplicate truth.

D) Standards System (Ruleset Layer)

- **STD-01 Standards Library Structure** → SYS-01 → defines standards categories + storage
- **STD-02 Standards Resolution Rules** → STD-01 + routing fields → defines how snapshot is produced
- **STD-03 Standards Snapshot Format** → STD-02 → defines the emitted resolved standards artifact

Enables: deterministic constraints for templates/spec/plans.

E) Template System (Manufacturing Layer)

- **TMP-01 Template Types & Template Index** → SYS-03 → defines template inventory + registry
- **TMP-02 Template File Contract** → TMP-01 → enforces consistent template structure
- **TMP-03 Template Selection Rules** → TMP-01 + routing + gates → decides which templates apply
- **TMP-04 Template Fill Rules** → CAN-01/02/03 + STD-03 → defines how placeholders get filled
- **TMP-05 Template Completeness Rules** → TMP-02 + TMP-04 → defines when a filled template is valid

Enables: filling templates without guessing.

F) Document Build Order & Gates (Compliance Backbone)

- **ORD-01 Build Order Graph** → all doc inventories (INT*, CAN*, STD*, TMP*, PLAN*, VER*, KIT*, STATE*, GOV*, EXEC*) → defines canonical order
- **ORD-02 Gate DSL / Gate Rules** → SYS-01 + TMP-05 + INT-05 → defines how gates are expressed and evaluated
- **ORD-03 Per-Doc Gate Checklist** → ORD-02 → defines pass/fail checks per doc

Enables: “can’t proceed unless compliant.”

G) Planning & Decomposition System (Build Plan Layer)

- **PLAN-01 Work Breakdown Rules** → CAN-01/02 + STD-03 → defines how work units are created

- **PLAN-02 Acceptance Map Rules** → PLAN-01 + VER-01 + STD-03 → defines per-unit done/proof mapping
- **PLAN-03 Sequencing Heuristics** → PLAN-01 → defines ordering principles

Enables: work units + acceptance map generation.

H) Verification & Proof System (Evidence Layer)

- **VER-01 Proof Types & Evidence Rules** → SYS-01 → defines what counts as proof
- **VER-02 Verification Command Policy** → VER-01 + STD-03 → defines when/how checks run
- **VER-03 Completion Criteria** → VER-01 + PLAN-02 → defines global done rules

Enables: objective proof gating.

I) Packaging & Kit Format (Delivery Layer)

- **KIT-01 Kit Folder Structure Contract** → SYS-02 → defines kit layout
- **KIT-02 Manifest/Index Format** → KIT-01 + CAN-02 → defines IDs → paths + reading order
- **KIT-03 Entrypoint Contract** → KIT-01 + EXEC-02 → defines single start doc for external agent
- **KIT-04 Version Stamping Rules** → GOV-01 + STD-03 + TMP-01 → defines what versions get recorded

Enables: deterministic kit packaging.

J) Continuity & State (Resume Layer)

- **STATE-01 State Snapshot Format** → PLAN-01 + PLAN-02 + VER-01 → defines progress/proof tracking
- **STATE-02 Resume Rules** → STATE-01 + CAN-02 → defines safe restart behavior
- **STATE-03 Handoff Rules** → STATE-02 → defines multi-agent continuity rules

Enables: no drift across sessions/agents.

K) Governance & Evolution (Stability Over Time)

- **GOV-01 Versioning Policy** → SYS-01 → version rules for templates/standards/system
- **GOV-02 Change Control Rules** → GOV-01 + ORD-02 → governs what can change and how
- **GOV-03 Deprecation & Migration Rules** → GOV-01 → old→new compatibility rules
- **GOV-04 Audit Log Rules** → INT-04 + KIT-04 → traceability requirements

Enables: clean evolution without junkyard growth.

L) Execution Instructions (Runbooks)

- **EXEC-01 Internal Agent Runbook** → all above (especially ORD-01/02/03) → step-by-step internal agent procedure
- **EXEC-02 External Agent Prompt Template** → KIT-03 + CAN/PLAN/VER outputs → the external agent entry instructions
- **EXEC-03 Failure Handling Playbook** → VER-02 + STATE-02 + GOV-02 → what to do when blocked

---

Minimal “Critical Path” (if you want the shortest backbone)

SYS → INT → CAN → STD → TMP → ORD → PLAN → VER → KIT → STATE → EXEC

If you want, I can rewrite this as a **single ordered list** (topological order) instead of dependencies.

Build Order

Build Order (Topological Order) — Internal Doc System

This is the **recommended canonical build order** (generate/lock in this order). Each step assumes upstream docs are compliant before proceeding.

0) Foundations

1. **SYS-01 System Purpose & Guarantees**
2. **SYS-03 Glossary / Core Terms**
3. **SYS-02 High-Level Architecture (end-to-end flow)**

1) Intake & Validation

4. **INT-01 Intake Form Spec**
5. **INT-02 Intake Schema Spec**
6. **INT-03 Intake Validation Rules**
7. **INT-05 Validator Output Format**
8. **INT-04 Submission Record Format**

2) Canonical Truth Model

9. **CAN-01 Canonical Spec Model**
10. **CAN-02 ID & Reference Rules**
11. **CAN-03 Unknowns Model**

3) Standards System

12. **STD-01 Standards Library Structure**
13. **STD-02 Standards Resolution Rules**

14. **STD-03 Standards Snapshot Format**

4) Template System

15. **TMP-01 Template Types & Template Index**
16. **TMP-02 Template File Contract**
17. **TMP-04 Template Fill Rules**
18. **TMP-05 Template Completeness Rules**
19. **TMP-03 Template Selection Rules**

5) Planning & Verification Systems

20. **PLAN-01 Work Breakdown Rules**
21. **PLAN-03 Sequencing Heuristics**
22. **VER-01 Proof Types & Evidence Rules**
23. **VER-02 Verification Command Policy**
24. **PLAN-02 Acceptance Map Rules**
25. **VER-03 Completion Criteria**

6) Packaging & Continuity

26. **KIT-01 Kit Folder Structure Contract**
27. **KIT-02 Manifest/Index Format**
28. **KIT-03 Entrypoint Contract**
29. **STATE-01 State Snapshot Format**
30. **STATE-02 Resume Rules**

31. **STATE-03 Handoff Rules**

7) Governance

32. **GOV-01 Versioning Policy**
33. **GOV-03 Deprecation & Migration Rules**
34. **GOV-02 Change Control Rules**
35. **GOV-04 Audit Log Rules**

8) Execution Runbooks

36. **EXEC-01 Internal Agent Runbook**
37. **EXEC-03 Failure Handling Playbook**
38. **EXEC-02 External Agent Prompt Template**